

Spis treści

1. Strona tytułowa i informacje o dokumencie.....	5
1.1. Dane podstawowe.....	5
1.2. Autorzy	5
1.3. Opieka nad projektem.....	5
1.4. Informacja o dokumencie.....	5
2. Wprowadzenie	6
2.1. Podstawowe pojęcia.....	6
2.2. Odniesienie do implementacji.....	7
3. Cel i zakres projektu.....	8
3.1. Cel projektu	8
3.2. Zakres funkcjonalny wykonanej platformy.....	8
3.3. Audytowalność procesu	8
3.4. Zakres techniczny	8
3.5. Elementy poza wykonanym zakresem	9
3.6. Odniesienie do implementacji.....	9
4. Wymagania	10
4.1. Wymagania funkcjonalne	10
4.2. Wymagania нефunkcjonalne	12
4.3. Macierz realizacji wymagań.....	13
4.4. Podsumowanie oceny.....	15
5. Źródła danych i zakres informacyjny.....	16
5.1. Systemy źródłowe.....	16
5.2. Zbiory plikowe	17
5.3. Modele PERSON i PARTY	17
5.4. Mapowanie kolumn.....	18
5.5. Demonstracyjne źródło Oracle	18
5.6. Dane referencyjne TERYT	19
5.7. Automatyczne rozpoznawanie źródła w Airflow	19
5.8. Dane syntetyczne	20
5.9. Odniesienie do implementacji.....	20
6. Architektura systemu.....	21
6.1. Architektura logiczna	21
6.2. Architektura aplikacji FastAPI.....	22
6.3. Główne komponenty	22
6.4. Komunikacja pomiędzy komponentami	23
6.5. Konfiguracja.....	23
6.6. Środowisko lokalne.....	24
6.7. Przygotowane zasoby OpenShift	24
6.8. Ograniczenia architektury	25
6.9. Odniesienie do implementacji.....	25
7. Technologie i narzędzia	26
7.1. Zestawienie technologii.....	26
7.2. Backend	27
7.3. Dostęp do danych.....	27
7.4. Orkiestracja.....	27
7.5. Biblioteki przetwarzania danych.....	27
7.6. Frontend	28
7.7. Neo4j	28
7.8. Konteneryzacja i wdrożenie	28
7.9. Narzędzia developerskie.....	28

7.10. Odniesienie do implementacji.....	28
8. Model danych.....	30
8.1. Organizacja schematów.....	30
8.2. Schemat META	30
8.3. Schemat RAW	31
8.4. Schemat STG.....	31
8.5. Schemat GOLD.....	34
8.6. Audyt zmian i lineage	36
8.7. Relacje między schematami	37
8.8. Integralność i inicjalizacja	38
8.9. Odniesienie do implementacji.....	38
9. Przepływ i orkiestracja danych	40
9.1. Ogólny przebieg procesu	40
9.2. Parametry uruchomienia DAG-u	40
9.3. Komunikacja Airflow z FastAPI	41
9.4. Przekazywanie identyfikatorów	42
9.5. Zakres zadań DAG-u.....	42
9.6. Statusy i logowanie.....	43
9.7. Transakcje i powtarzalność.....	44
9.8. Obsługa błędów	44
9.9. Ograniczenia orkiestracji	45
9.10. Odniesienie do implementacji.....	45
10. Pobieranie i składowanie danych RAW	46
10.1. Interfejs API	46
10.2. Import plików	46
10.3. Import ze źródła relacyjnego	47
10.4. Wspólny zapis materiału RAW	48
10.5. Struktura RawFile	48
10.6. Kontrola duplikatów	48
10.7. Obsługa błędów	49
10.8. Uzasadnienie użycia VARBINARY(MAX).....	49
10.9. Bezpieczeństwo i integralność.....	49
10.10. Ograniczenia	49
10.11. Realizacja w kodzie	50
11. Staging i mapowanie danych	51
11.1. Interfejs API	51
11.2. Przebieg ładowania	51
11.3. Parsowanie materiału RAW	51
11.4. Mapowania kolumn.....	52
11.5. Raport mapowania	52
11.6. Budowa rekordu stagingowego.....	52
11.7. Model PERSON	53
11.8. Model PARTY	53
11.9. Zakres konwersji.....	53
11.10. Statusy, błędy i powtarzalność	54
11.11. Ograniczenia	54
11.12. Odniesienie do implementacji.....	54
12. Preprocessing i standaryzacja	55
12.1. Uruchomienie procesu	55
12.2. Przebieg przetwarzania	55
12.3. Reguły wspólne.....	55
12.4. Standaryzacja danych osoby	56
12.5. Standaryzacja danych podmiotu	56

12.6. Rozdzielanie i składanie adresu	56
12.7. Informacja o zastosowanych regułach	57
12.8. Przygotowanie danych do dopasowywania	57
12.9. Ograniczenia	57
12.10. Odniesienia do implementacji.....	57
13. Walidacja	59
13.1. Uruchomienie procesu	59
13.2. Przebieg walidacji	59
13.3. Model wyniku	59
13.4. Walidacja osoby.....	60
13.5. Walidacja podmiotu	60
13.6. Kontrola adresu z wykorzystaniem TERYT	61
13.7. Walidacja adresu e-mail	61
13.8. Wykorzystanie wyników	61
13.9. Ograniczenia	61
13.10. Odniesienia do implementacji.....	61
14. Matching, grupowanie i budowa Golden Record	63
14.1. Uruchamianie procesu	63
14.2. Model reguł dopasowania	63
14.3. Etap Levenshteina	64
14.4. Ograniczenie liczby porównań.....	65
14.5. Etap Jaro-Winklera	65
14.6. Znaczenie decyzji matchingowych	65
14.7. Grupowanie rekordów	65
14.8. Budowa Golden Record osoby	66
14.9. Budowa Golden Record podmiotu	66
14.10. Reguły survivorship	67
14.11. Adresy i powiązania adresowe	67
14.12. Odrzucanie grup	67
14.13. Transakcyjność etapu GOLD	68
14.14. Ograniczenia	68
14.15. Odniesienia do implementacji.....	68
15. Audytowalność, lineage i historia zmian	69
15.1. Logi przebiegu procesu.....	69
15.2. Logowanie etapu GOLD	69
15.3. Czym jest lineage w tym systemie.....	70
15.4. Tabele lineage.....	70
15.5. Lineage osób, podmiotów i adresów.....	71
15.6. Lineage relacji adresowych.....	71
15.7. Wykorzystanie walidacji w lineage	71
15.8. Rejestr zmian wymiarów	72
15.9. Co dokładnie daje historia zmian	72
15.10. Ograniczenia historii lineage i zmian	72
15.11. Przykłady informacji możliwych do odtworzenia	73
15.12. Wykorzystanie przez warstwę serving	73
15.13. Odniesienia do implementacji.....	73
16. REST API	75
16.1. Organizacja aplikacji	75
16.2. Styl komunikacji API	75
16.3. Endpointy techniczne poza /layers.....	75
16.4. Endpointy statusowe warstw	76
16.5. Endpointy sterujące pipeline'em.....	76
16.6. Endpointy odczytowe warstwy serving	77

16.7. Typowe odpowiedzi warstwy serving	78
16.8. Konwencja błędów	79
16.9. CORS i współpraca z frontendem	79
16.10. Swagger i OpenAPI	79
16.11. Zakres i ograniczenia API	80
16.12. Odniesienia do implementacji	80
17. Interfejs użytkownika.....	81
17.1. Technologia i sposób uruchomienia	81
17.2. Struktura aplikacji.....	81
17.3. Komunikacja z backendem	81
17.4. Główny układ aplikacji.....	82
17.5. Widok walidacji	82
17.6. Widok matchingu	83
17.7. Porównanie pary rekordów	83
17.8. Komponenty wspólne.....	84
17.9. Formatowanie i stałe	84
17.10. Warstwa stylów	85
17.11. Zakres funkcjonalny obecnej wersji	85
17.12. Miejsce frontendu w architekturze	85
17.13. Ograniczenia	86
17.14. Odniesienia do implementacji.....	86
18. Środowisko uruchomieniowe, konfiguracja i wdrożenie	87
18.1. Środowisko lokalne Docker Compose	87
18.2. Kontenery i obrazy.....	87
18.3. Wolumeny i trwałość danych	88
18.4. Zależności startowe i healthchecki	89
18.5. Konfiguracja backendu	89
18.6. Konfiguracja frontendu	90
18.7. Konfiguracja Airflow	90
18.8. Lokalne uruchomienie	90
18.9. Inicjalizacja baz danych.....	91
18.10. OpenShift.....	91
18.11. Różnice między Docker Compose i OpenShift.....	92
18.12. Bezpieczeństwo konfiguracji	92
18.13. Ograniczenia środowiskowe.....	92
18.14. Odniesienia do implementacji.....	92
19. Weryfikacja rozwiązania	94
19.1. Zakres przyjętej weryfikacji	94
19.2. Testy automatyczne backendu	94
19.3. Weryfikacja lokalnego uruchomienia	96
19.4. Zakres nieobjęty pełną automatyzacją	96
19.5. Odniesienia do implementacji.....	96
20. Podsumowanie realizacji.....	98
20.1. Końcowe odniesienie do implementacji	98

1. Strona tytułowa i informacje o dokumencie

1.1. Dane podstawowe

Tytuł projektu:

Platforma integracji danych podstawowych podmiotów i osób

Rodzaj dokumentu:

Dokumentacja techniczna projektu

Wersja dokumentu:

1.0

Data:

17.06.2026

Główne repozytorium (Github):

[Github](#)

Kopia repozytorium (Gitlab):

[Gitlab](#)

1.2. Autorzy

Projekt został przygotowany przez zespół w składzie:

- inż. Aleksandra Bamberska
- inż. Julia Nowakowska
- inż. Sebastian Wróbel
- inż. Adam Biernacki

1.3. Opieka nad projektem

Opiekun z ramienia uczelni:

dr hab. inż. Robert Wrembel, prof. PP

Opiekun z ramienia UKNF:

Andrzej Burzyński, Urząd Komisji Nadzoru Finansowego

1.4. Informacja o dokumencie

Dokument opisuje stan rozwiązania faktycznie zaimplementowanego w repozytorium.

2. Wprowadzenie

Dane opisujące osoby i podmioty są wykorzystywane przez wiele systemów i procesów, lecz często występują w różnych strukturach, formatach i poziomach jakości. Ten sam podmiot może być zapisany pod kilkoma nazwami, a informacje o jego identyfikatorach, adresach lub danych kontaktowych mogą być rozproszone pomiędzy niezależnymi źródłami. Analogiczny problem dotyczy osób, dla których poszczególne systemy mogą przechowywać różne warianty imienia i nazwiska, adresu, dokumentów tożsamości lub danych kontaktowych.

Samo zgromadzenie takich danych w jednym miejscu nie rozwiązuje problemu ich niespójności. Konieczne jest zachowanie danych źródłowych, ujednolicenie ich struktury, kontrola poprawności, rozpoznanie rekordów odnoszących się do tej samej encji oraz wybór wartości, które powinny utworzyć jej wynikową reprezentację. Proces ten powinien być powtarzalny i audytowalny, tak aby możliwe było ustalenie źródła każdej istotnej informacji oraz przyczyny podjęcia decyzji o połączeniu rekordów.

W ramach projektu wykonano platformę integracji i goldenizacji danych podstawowych osób i podmiotów. Rozwiązanie przyjmuje dane z wielu źródeł, zapisuje ich oryginalną zawartość, mapuje je do wspólnego modelu, wykonuje preprocessing i walidację, a następnie przeprowadza matching oraz grupowanie rekordów. Ostatnim etapem jest utworzenie lub aktualizacja Golden Record, stanowiącego spójną reprezentację osoby albo podmiotu.

Platforma została zaprojektowana warstwowo. Poszczególne etapy przetwarzania są rozdzielone pomiędzy moduły pobierania danych, stagingu, preprocessingu, walidacji oraz integracji. Ułatwia to niezależne rozwijanie i testowanie reguł właściwych dla każdego etapu. Operacje warstw są dostępne przez interfejs REST aplikacji FastAPI, natomiast ich wykonanie w pełnym procesie koordynuje Apache Airflow. Podstawowym repozytorium danych procesowych i wynikowych jest Microsoft SQL Server.

Istotną cechą rozwiązania jest rozdzielenie dwóch typów encji. Model PERSON obejmuje dane osób, takie jak imiona, nazwiska, PESEL, dokumenty tożsamości, data urodzenia, dane kontaktowe i adresowe. Model PARTY służy do reprezentowania podmiotów i zawiera między innymi nazwę, formę prawną, identyfikatory NIP, REGON, KRS i LEI, dane rejestrowe oraz adresy. Rozdzielenie modeli pozwala stosować różne reguły walidacji, priorytety źródeł i kryteria dopasowania.

Goldenizacja jest w projekcie rozumiana jako proces składający się z trzech głównych działań. Pierwszym jest wyszukanie kandydatów do połączenia na podstawie identyfikatorów oraz podobieństwa pozostałych atrybutów. Drugim jest grupowanie rekordów uznanych za reprezentację tej samej encji. Trzecim jest wybór wartości wynikowych, określane również jako survivorship. Wybór uwzględnia między innymi wynik walidacji, priorytet źródła dla danego pola, poziom zaufania do źródła oraz aktualność importu.

System zachowuje informacje niezbędne do prześledzenia procesu. Rejestrowane są systemy źródłowe, partie importu, pliki RAW, wyniki kolejnych kroków, reguły walidacyjne, kandydaci matchingu oraz grupy encji. Dla atrybutów zapisanych w warstwie GOLD przechowywane jest ich pochodzenie, zastosowana reguła wyboru oraz informacje o jakości i zaufaniu. Aktualizacje danych osoby i podmiotu są dodatkowo odnotowywane w rejestrze zmian.

Dokumentacja opisuje zarówno sposób działania platformy, jak i jej realizację techniczną. Przedstawia źródła danych, architekturę, model bazodanowy, kolejne etapy pipeline'u, algorytmy matchingu, reguły budowy Golden Record oraz mechanizmy audytowe. Uwzględnia również uruchomienie lokalne, interfejsy API, testy, nieprzetestowane manifesty OpenShift oraz pozostałe ograniczenia wykonanej wersji. Odniesienia do plików, klas, funkcji i testów umożliwiają powiązanie opisu z kodem przechowywanym w repozytorium.

2.1. Podstawowe pojęcia

Golden Record to wynikowa reprezentacja osoby lub podmiotu zbudowana z wartości wybranych spośród rekordów źródłowych należących do tej samej grupy.

Goldenizacja oznacza proces identyfikacji odpowiadających sobie rekordów, ich grupowania oraz budowy Golden Record.

Matching jest procesem oceny, czy dwa rekordy mogą opisywać tę samą osobę lub ten sam podmiot. W projekcie wykorzystywane są zarówno silne identyfikatory, jak i podobieństwo pozostałych pól.

Survivorship określa reguły wyboru wartości, która zostanie zapisana w Golden Record, gdy rekordy źródłowe zawierają różne warianty tego samego atrybutu.

Lineage opisuje pochodzenie danych. Pozwala powiązać atrybut Golden Record z systemem źródłowym, rekordem wejściowym, partią importu i regułą wyboru.

Partia importu jest jednostką organizującą pojedyncze pobranie danych ze wskazanego źródła. Jej identyfikator umożliwia powiązanie danych i logów powstających w kolejnych warstwach.

Warstwa RAW przechowuje oryginalną zawartość danych przyjętych przez platformę, zanim zostaną one przekształcone.

Staging jest warstwą pośrednią, w której dane źródłowe są mapowane do wspólnej struktury osoby lub podmiotu.

Preprocessing obejmuje standaryzację i normalizację wartości przed walidacją oraz matchingiem.

2.2. Odniesienie do implementacji

Główna aplikacja FastAPI jest konfigurowana w `app/main.py`, a routery warstw są łączone w `app/layers/router.py`. Logika procesu znajduje się w katalogach:

- `app/layers/ingestion`,
- `app/layers/staging_validation`,
- `app/layers/preprocessing`,
- `app/layers/validation`,
- `app/layers/integration_golden`.

Orkiestrację pełnego przebiegu realizuje DAG `goldenizacja_pipeline` zdefiniowany w `airflow/dags/goldenizacja_pipeline.py`. Modele danych są zapisane w plikach `models.py` poszczególnych warstw, a odpowiadający im skrypt Microsoft SQL Server znajduje się w `scripts/init_proposed_mssql_schema.sql`.

3. Cel i zakres projektu

3.1. Cel projektu

Celem projektu było wykonanie autonomicznej platformy integracji danych podstawowych osób i podmiotów. Rozwiązanie miało umożliwiać przyjmowanie danych z różnych źródeł, zachowanie ich oryginalnej postaci, kontrolę jakości, integrację rekordów oraz udostępnienie spójnej reprezentacji danych. Założenia przekazane przez opiekuna wskazywały na potrzebę budowy rozwiązania odpowiadającego koncepcji centralnego repozytorium osób i podmiotów, w którym możliwe jest prześledzenie pochodzenia danych oraz procesu prowadzącego do utworzenia Golden Record.

Projekt obejmował analizę wymagań, zaprojektowanie architektury warstwowej, implementację modułów przetwarzania, przygotowanie środowiska uruchomieniowego oraz testowanie rozwiązania. Szczególny nacisk położono na centralną część procesu: przejście od danych heterogenicznych do ujednoliconych, zwalidowanych i pogrupowanych rekordów osoby lub podmiotu.

3.2. Zakres funkcjonalny wykonanej platformy

Platforma przyjmuje pliki CSV, JSON, XML i XLSX oraz dane z demonstracyjnego źródła Oracle. Każdy import jest przypisywany do systemu źródłowego i partii, a zawartość wejściowa jest zachowywana w warstwie RAW wraz z metadanymi i skrótem SHA-256.

Dane są następnie mapowane do modeli PERSON albo PARTY, normalizowane i poddawane regułom jakości, w tym kontroli identyfikatorów i adresów z użyciem TERYT. Po walidacji system wykonuje dwuetapowy matching, grupuje rekordy zakwalifikowane do połączenia oraz tworzy lub aktualizuje osoby, podmioty, adresy i identyfikatory w warstwie GOLD.

Wartości Golden Record są wybierane według reguł survivorship uwzględniających jakość, priorytet i poziom zaufania źródła oraz aktualność partii. Pełny proces może zostać uruchomiony przez DAG Apache Airflow, który wywołuje operacje aplikacji FastAPI od importu RAW do materializacji danych GOLD.

Warstwa serving udostępnia przez REST listy i szczegóły Golden Record, wyszukiwanie osób po PESEL oraz podmiotów po identyfikatorach lub nazwie. Pozwala także odczytywać lineage, historię zmian, wyniki walidacji, kandydatów matchingu i liczniki etapów procesu.

3.3. Audytowalność procesu

System rejestruje źródła, partie importu, przebieg etapów, wyniki walidacji, kandydatów matchingu i grupy encji. Dla aktualnych wartości GOLD utrzymywane jest lineage wskazujące ich źródło, partię oraz regułę wyboru.

Zmiany atrybutów istniejących osób i podmiotów są zapisywane w EntityChangeLog wraz ze starą i nową wartością. Pozwala to ustalić, na podstawie jakich danych powstał aktualny Golden Record oraz jak zmieniał się w kolejnych importach.

3.4. Zakres techniczny

Backend wykonano w Pythonie z użyciem FastAPI i SQLAlchemy. Microsoft SQL Server przechowuje dane procesowe i wynikowe, Oracle jest demonstracyjnym źródłem relacyjnym, a Apache Airflow orkiestruje pipeline. Środowisko lokalne jest uruchamiane przez Docker Compose i obejmuje również frontend React oraz demonstracyjną konfigurację Neo4j.

Repozytorium zawiera manifesty OpenShift, ale nie zostały one zweryfikowane na docelowym klastrze. Neo4j również nie jest częścią przetestowanego procesu goldenizacji. Elementy te należy traktować jako przygotowaną konfigurację, a nie potwierdzoną ścieżkę działania systemu.

Pierwotna koncepcja przewidywała SQL Server FILESTREAM. W wykonanej wersji zastosowano VARBINARY(MAX) mapowany jako LargeBinary. W ograniczonym czasie projektu wybrano wariant możliwy do uruchomienia i zweryfikowania bez dodatkowej konfiguracji FILESTREAM. Decyzja nie wynika z założenia, że środowisko docelowe nie obsługuje tego mechanizmu.

3.5. Elementy poza wykonanym zakresem

Matching nie wykorzystuje ML ani DL, lecz jawne reguły, wagi, progi i deterministyczne miary podobieństwa. Nie wykonano także konektorów strumieniowych oraz wejść z zewnętrznych usług REST lub SOAP.

Warstwa analytics pozostaje szkieletem i nie udostępnia raportów ani metryk. Warstwa serving realizuje odczyt danych, ale nie zawiera mechanizmu PUSH ani webhooków. Frontend korzysta już z odczytowych endpointów servingowych dla walidacji i matchingu, ale nie udostępnia jeszcze pełnego profilu Golden Record ani obsługi manualnych decyzji integracyjnych. Neo4j nie przechowuje relacji wynikowych głównego procesu.

Brakuje również interfejsu operatora do ręcznej obsługi decyzji REVIEW oraz uwierzytelniania i autoryzacji API. Szczegółowa ocena realizacji tych elementów znajduje się w macierzy wymagań.

3.6. Odniesienie do implementacji

Poszczególne części zakresu są realizowane w następujących modułach:

Obszar	Główna lokalizacja
Pobieranie i RAW	app/layers/ingestion
Staging i mapowanie	app/layers/staging_validation
Preprocessing	app/layers/preprocessing
Walidacja i TERYT	app/layers/validation
Matching, grupowanie i Golden Record	app/layers/integration_golden
Udostępnianie danych	app/layers/serving
Orkiestracja	airflow/dags/goldenizacja_pipeline.py
Model SQL Server	scripts/init_proposed_mssql_schema.sql
Źródło demonstracyjne Oracle	scripts/init_oracle_insurance_core.sql
Środowisko lokalne	docker-compose.yml
Zasoby OpenShift	openshift/
Frontend	frontend/src/App.jsx

Testy w katalogu tests obejmują główne etapy procesu, reguły integracji, budowę GOLD i audytowalność.

4. Wymagania

Wymagania dla platformy wynikają przede wszystkim z dokumentu „Projekt studencki - Platforma integracyjna CRP”, przekazanego przez opiekuna projektu. Dokument określa cel rozwiązania oraz główne bloki funkcjonalne, lecz nie definiuje szczegółowych kryteriów odbioru ani zamkniętej listy przypadków użycia. Materiały przygotowane przez zespół na początku prac rozwijają tę koncepcję o proponowane mechanizmy techniczne. Nie wszystkie opisane w nich rozwiązania zostały ostatecznie wykorzystane.

W poniższym zestawieniu wymagania pierwotne zostały uporządkowane i odniesione do wykonanej implementacji. Ocena dotyczy stanu kodu analizowanego podczas przygotowania dokumentacji. Funkcje obecne wyłącznie jako szkieleł, demonstrator albo nieprzetestowana konfiguracja nie są traktowane jako pełna realizacja wymagań.

4.1. Wymagania funkcjonalne

Przyjmowanie danych z wielu źródeł

Platforma powinna przyjmować dane o osobach i podmiotach pochodzące z różnych systemów i zapisane w różnych formatach. Dokument wejściowy wymienia pliki wsadowe, relacyjne bazy danych, strumienie danych, interfejsy REST oraz zewnętrzne i wewnętrzne źródła referencyjne.

W wykonanym rozwiązaniu zakres wejścia obejmuje pliki CSV, JSON, XML i XLSX oraz dane relacyjne pobierane z demonstracyjnego systemu Oracle. Rejestrowane są źródła odpowiadające między innymi PESEL, CEIDG, KRS, REGON, VAT, GLEIF oraz wybranym rejestrom KNF. Nie wykonano konektorów wejściowych dla strumieni danych i zewnętrznych usług REST lub SOAP.

Zachowanie danych surowych

Dane przyjęte przez platformę powinny być przechowywane w oryginalnej postaci, bez ingerencji wynikającej z późniejszych transformacji. Każdy import powinien posiadać informacje o źródle, czasie pobrania, rozmiarze i stanie przetwarzania.

Warstwa RAW zapisuje pełną zawartość wejścia jako dane binarne. Dodatkowo przechowywane są nazwa i typ pliku, jego rozmiar, skrót SHA-256, źródło oraz partia importu. Dane relacyjne są najpierw serializowane do dokumentu JSON, który staje się niezmiennym obrazem pobrania i jest przetwarzany dalej tak samo jak pozostałe dane RAW.

Mapowanie do wspólnego modelu

Platforma powinna przekształcać dane o różnej strukturze do ujednoliconego modelu umożliwiającego wykonywanie wspólnych reguł walidacyjnych i integracyjnych.

Zaimplementowano dwa modele kanoniczne: PERSON dla osób oraz PARTY dla podmiotów. Mapowanie uwzględnia różnice w nazewnictwie kolumn, wielkości liter, strukturach zagnieżdżonych i formatach plików. Konfiguracja mapowania jest powiązana z systemem źródłowym i typem encji.

Standaryzacja danych

Przed integracją dane powinny zostać oczyszczone i doprowadzone do porównywalnej postaci. Wymaganie obejmuje między innymi normalizację tekstu, dat, wartości logicznych, danych kontaktowych, identyfikatorów i adresów.

Preprocessing tworzy oddzielne rekordy przygotowane do matchingu. Normalizowane są między innymi nazwy, imiona i nazwiska, identyfikatory, numery telefonów, adresy e-mail, kody pocztowe, formy prawne i elementy adresu. Zastosowane reguły są zapisywane przy rekordzie preprocessingowym.

Walidacja i kontrola jakości

Platforma powinna wykonywać reguły jakości właściwe dla danych źródłowych, znakować błędne rekordy oraz umożliwiać ich dalszą analizę bez zatrzymywania całego procesu.

Wyniki walidacji są zapisywane osobno dla każdej reguły i rekordu. Kontrole obejmują między innymi PESEL, NIP, REGON, KRS, LEI, numer dowodu osobistego, adres e-mail, nazwy osób, zakresy dat i zgodność danych wynikających z numeru PESEL. Błąd pojedynczego rekordu nie przerywa przetwarzania całej partii.

Wzbogacanie danymi referencyjnymi

Wymagania zakładają wykorzystanie danych referencyjnych do kontroli lub uzupełniania danych źródłowych.

W wykonanej wersji dane TERYT są używane do sprawdzania występowania miejscowości i ulic. Mechanizm wpływa również na reguły wyboru adresu Golden Record. Nie wykonano szerszego mechanizmu automatycznego uzupełniania brakujących wartości na podstawie wielu źródeł referencyjnych.

Identyfikacja odpowiadających sobie rekordów

Platforma powinna rozpoznawać rekordy dotyczące tej samej osoby lub tego samego podmiotu. Identyfikacja powinna wykorzystywać zarówno klucze silne, jak i podobieństwo atrybutów niebędących jednoznaczными identyfikatorami.

Matching uwzględnia identyfikatory takie jak PESEL, NIP, REGON, KRS i LEI, a także dane opisowe, kontaktowe i adresowe. Pierwszy etap wykorzystuje podobieństwo Levenshteina, a drugi Jaro-Winklera. Reguły przypisują połom role i wagi, rozpoznają konflikty danych stabilnych oraz klasyfikują wynik zgodnie z progami decyzyjnymi.

Grupowanie i budowa Golden Record

Rekordy uznane za reprezentacje tej samej encji powinny otrzymać wspólny klucz integracji i zostać wykorzystane do budowy wynikowej reprezentacji osoby lub podmiotu.

Kandydaci z decyzją AUTO_MERGE są grupowani, a rekordy bez połączeń tworzą grupy jednoelementowe. Na podstawie grup tworzone lub aktualizowane są osoby, podmioty, adresy i identyfikatory w schemacie gold. Grupy niespełniające minimalnych wymagań mogą zostać zapisane w rejestrze odrzuceń.

Ranking źródeł i wybór wartości wynikowych

W przypadku konfliktu danych platforma powinna wybrać wartość zgodnie z jakością, istotnością i wiarygodnością źródła oraz aktualnością danych.

Reguły survivorship uwzględniają obecność wartości, status walidacji, potwierdzenie adresu w TERYT, priorytet źródła dla danego atrybutu, poziom zaufania oraz czas rozpoczęcia importu. Informacja o zastosowanym kryterium jest zapisywana w lineage.

Audytowalność

Proces powinien umożliwiać ustalenie pochodzenia danych, przebiegu przetwarzania, podstaw decyzji integracyjnych oraz zmian Golden Record.

System zapisuje systemy źródłowe, partie importu, pliki RAW, logi kroków, wyniki walidacji, kandydatów matchingu i grupy encji. Lineage wskazuje pochodzenie aktualnych wartości Golden Record oraz regułę ich wyboru. EntityChangeLog rejestruje zmiany atrybutów osoby i podmiotu wraz ze starą i nową wartością.

Orkiestracja procesu

Kolejne etapy powinny tworzyć powtarzalny proces możliwy do uruchomienia dla wybranego źródła i typu encji.

DAG Apache Airflow wykonuje pełną sekwencję od pobrania danych do utworzenia Golden Record. Obsługuje źródła plikowe i relacyjne oraz pozwala konfigurować typ encji, kontrolę DNS i progi matchingu.

Udostępnianie danych

Założenia przewidują udostępnianie pełnych i dedykowanych widoków danych przez REST, a także przekazywanie aktualizacji do innych systemów.

Wykonane API udostępnia operacje sterujące etapami pipeline'u oraz konsumenckie endpointy odczytowe warstwy serving. Obejmują one listy i szczegóły osób oraz podmiotów GOLD, wyszukiwanie po identyfikatorach i nazwie, lineage, historię zmian, wyniki walidacji, kandydatów matchingu, porównanie rekordów i liczniki etapów. Nie wykonano mechanizmu PUSH ani webhooków.

Interfejs użytkownika

Platforma powinna zapewniać użytkownikowi dostęp do danych i wyników integracji.

Frontend React udostępnia widoki wyników walidacji i matchingu oparte na endpointach warstwy serving. Pozwala filtrować wyniki walidacji, przeglądać kandydatów Levenshteina i Jaro-Winklera oraz porównywać szczegóły wybranej pary rekordów. Nie zawiera jeszcze widoku pełnego profilu Golden Record, wyszukiwarki biznesowej ani interfejsu ręcznej obsługi przypadków REVIEW, dlatego wymaganie pozostaje zrealizowane częściowo.

Mechanizm samouczący

Wymagania szczegółowe wskazują na zastosowanie nadzorowanych modeli ML lub DL do automatyzacji integracji danych.

W wykonanym rozwiązaniu nie zastosowano mechanizmu samouczącego. Matching jest deterministyczny i opiera się na jawnych regułach, wagach, progach oraz algorytmach podobieństwa tekstowego. Materiał zespołu opisujący Random Forest należy traktować jako koncepcję, a nie element implementacji.

Grafowa reprezentacja relacji

Wymagania wskazują potrzebę rozważenia technologii grafowej dla złożonych relacji i widoku 360 stopni.

Neo4j został dodany do konfiguracji środowiska, a moduł demonstracyjny zapisuje do niego dokumenty. Baza grafowa nie została jednak wykorzystana ani przetestowana w głównym procesie goldenizacji i nie zawiera grafowej reprezentacji wynikowych relacji osób i podmiotów.

4.2. Wymagania niefunkcjonalne

Modularność

Kod powinien umożliwiać rozdzielenie odpowiedzialności poszczególnych etapów. Każda warstwa posiada własne API, serwis, repozytorium, modele oraz schematy odpowiedzi. Wspólny router łączy warstwy zgodnie z kolejnością przepływu danych.

Powtarzalność i idempotencja

Ponowne wykonanie procesu nie powinno prowadzić do niekontrolowanego powielania danych. Kod blokuje ponowne załadowanie tego samego pliku do tego samego stagingu, zastępuje wyniki kandydatów matchingu dla danego zakresu, stabilnie tworzy grupy i ponownie wykorzystuje istniejące adresy, identyfikatory oraz relacje. Zmiana adresu zamyka wcześniejsze aktywne powiązanie danego typu i tworzy nowe z datą obowiązywania.

Diagnostyka i obsługa błędów

Warstwy powinny zwracać jednoznaczne błędy wejścia i logować stan operacji. API rozróżnia błędy danych zwracane jako HTTP 400 od nieoczekiwanych awarii zwracanych jako HTTP 500. ProcessLog przechowuje statusy, liczniki i komunikaty błędów dla etapów procesu.

Konfigurowalność

Parametry połączeń, hasła, CORS oraz ustawienia usług są pobierane ze zmiennych środowiskowych. Progi matchingu i część zachowania walidacji mogą być ustawiane podczas uruchamiania DAG.

Przenośność środowiska

Podstawowe środowisko uruchomieniowe zostało przygotowane w Docker Compose. Repozytorium zawiera również manifesty OpenShift, ale nie zostały one przetestowane na docelowym klastrze. Nie można więc traktować wdrożenia OpenShift jako potwierdzonej właściwości wykonanej wersji.

Trwałość i integralność danych

Dane bazowe są przechowywane w trwałych wolumenach. Model SQL Server stosuje klucze obce, ograniczenia unikalności, ograniczenia kontrolne i indeksy. Oryginalna zawartość RAW jest identyfikowana skrótem SHA-256.

Bezpieczeństwo

Hasła i parametry połączeń są przekazywane przez zmienne środowiskowe, a dla OpenShift przygotowano zasoby Secrets. API nie implementuje jednak uwierzytelniania, autoryzacji ani ograniczeń dostępu do danych, dlatego wymaganie bezpieczeństwa jest spełnione jedynie na poziomie podstawowej separacji konfiguracji.

Wydajność i skalowalność

Matching posiada limit maksymalnej liczby porównywanych par oraz wykorzystuje wyszukiwanie kandydatów ograniczające pełne porównanie zbioru, gdy repozytorium udostępnia taką możliwość. Model bazy zawiera indeksy dla pól używanych podczas integracji. Nie wykonano jednak testów wydajnościowych ani pomiarów dla dużych wolumenów, dlatego wydajność nie została formalnie potwierdzona.

Testowalność

Logika biznesowa jest oddzielona od API i dostępu do danych, co umożliwia testowanie jej z repozytoriami zastępczymi. Testy obejmują import relacyjny, mapowanie, preprocessing, walidację, matching, survivorship, grupowanie, Golden Record, lineage, idempotencję, API servingowe i jakość danych syntetycznych.

4.3. Macierz realizacji wymagań

Id	Wymaganie	Stan realizacji	Realizacja i dowód w kodzie	Testy lub ograniczenia
F-01	Import plików CSV, JSON, XML i XLSX	Zrealizowane	SUPPORTED_FILE_TYPES, import_raw_file() w app/layers/ingestion/service.py; parsery w app/layers/staging_validation/service.py	Parsowanie i mapowanie sprawdza tests/test_staging_mapper.py
F-02	Import z relacyjnej bazy danych	Zrealizowane	import_relational_source(), extract_relational_records() i definicje zapytań Oracle w app/layers/ingestion/service.py	tests/test_relational_ingestion.py
F-03	Import ze strumieni danych i zewnętrznych REST/SOAP	Niezrealizowane	Brak konektorów wejściowych tego typu	Wzmianki w materiałach i README warstwy nie stanowią implementacji
F-04	Zachowanie oryginalnych danych RAW	Zrealizowane	RawFile.File_Content, File_Hash, File_Size; persist_raw_content()	Dane przechowywane jako VARBINARY(MAX), nie FILESTREAM
F-05	Rejestracja źródła i partii importu	Zrealizowane	SourceSystem, ImportBatch, ProcessLog w app/layers/ingestion/models.py	Logowanie kolejnych etapów jest rozproszone pomiędzy warstwy
F-06	Mapowanie do wspólnego modelu osoby i podmiotu	Zrealizowane	map_record_to_canonical(), PersonStaging, PartyStaging	tests/test_staging_mapper.py
F-07	Standaryzacja i preprocessing	Zrealizowane	build_preprocessed_record() oraz funkcje normalizacji w app/layers/preprocessing/service.py	tests/test_preprocessing.py
F-08	Walidacja identyfikatorów i reguł biznesowych	Zrealizowane	build_person_validation_results(), build_party_validation_results() i walidatory w app/layers/validation/service.py	tests/test_validation.py
F-09	Wzbogacanie lub weryfikacja danymi referencyjnymi	Zrealizowane z ograniczeniem	Ładowanie oraz walidacja miast i ulic TERYT w app/layers/validation	tests/test_teryt_validation.py; brak szerszego mechanizmu uzupełniania danych
F-10	Matching po silnych identyfikatorach	Zrealizowane	Reguły pól i score_match() w app/layers/integration_golden/service.py	tests/test_integration_golden_matching.py
F-11	Matching rozmyty	Zrealizowane	find_match_candidates() i refine_match_candidates_with_jaro_winkler()	Levenshtein i Jaro-Winkler; brak Jaccarda opisanego w koncepcji
F-12	Scoring i klasyfikacja decyzji	Zrealizowane	classify_match(), classify_jaro_winkler_match(), MatchDecision	Progi są konfigurowalne w Airflow
F-13	Grupowanie rekordów	Zrealizowane	group_auto_merge_candidates(), build_entity_groups(), add_singleton_entity_groups()	Testy grupowania i stabilności kluczy w tests/test_integration_golden_matching.py
F-14	Budowa Golden Record osoby i podmiotu	Zrealizowane	golden_load_dimensions(), create_or_update_golden_person(), create_or_update_golden_party()	tests/test_integration_golden_dimensions.py, tests/test_integration_golden_load.py
F-15	Ranking źródeł i reguły survivorship	Zrealizowane	select_survivor_value() oraz priorytety źródeł per atrybut	Testy wyboru wartości w tests/test_integration_golden_matching.py
F-16	Rejestr przypadków, z których nie można utworzyć Golden Record	Zrealizowane	GoldenRecordReject, record_golden_record_reject()	tests/test_integration_golden_load.py
F-17	Audytywalność pochodzenia Golden Record	Zrealizowane	Tabele Golden*Lineage, write_dimension_lineage(), lineage relacji adresowych	tests/test_integration_golden_dimensions.py
F-18	Rejestr zmian Golden Record	Zrealizowane	EntityChangeLog, record_dimension_changes(), record_entity_change()	Obejmuje aktualizowane atrybuty DimPerson i DimParty
F-19	Orkiestracja pełnego procesu	Zrealizowane	DAG goldenizacja_pipeline w airflow/dags/goldenizacja_pipeline.py	Nie wykonano automatycznego harmonogramu; DAG jest uruchamiany ręcznie
F-20	REST API do sterowania pipeline'em	Zrealizowane	Endpointy warstw w app/layers/*/api.py, router w app/layers/router.py	Dokumentacja OpenAPI generowana przez FastAPI
F-21	REST Out: pełne i dedykowane widoki Golden Record	Zrealizowane	Endpointy w app/layers/serving/api.py udostępniają listy, szczegóły, wyszukiwanie, lineage, historię i wyniki procesu	tests/test_serving_api.py; API ma charakter odczytowy
F-22	PUSH lub webhooki po zmianie danych	Niezrealizowane	Brak implementacji	Wymienione tylko w założeniach i README warstwy
F-23	Interfejs użytkownika do danych	Częściowo zrealizowane	frontend/src/App.jsx, frontend/src/features/validation/ValidationView.jsx, frontend/src/features/matching/MatchingView.jsx, frontend/src/features/matching/MatchingComparisonPanel.jsx	Brak profilu Golden Record, operacji zapisu i obsługi REVIEW

Id	Wymaganie	Stan realizacji	Realizacja i dowód w kodzie	Testy lub ograniczenia
F-24	Mechanizm samouczący ML/DL	Niezrealizowane	Brak bibliotek, modelu, danych uczących i procesu treningowego	Matching jest regułą; Random Forest występuje wyłącznie w materiale koncepcyjnym
F-25	Grafowa reprezentacja relacji i widok 360 stopni	Niezrealizowane w głównym systemie	Neo4j jest skonfigurowany, a <code>app/api/routes.py</code> zawiera demonstracyjny zapis dokumentów	Nieprzetestowane w pipeline'ie; brak grafu osób i podmiotów
F-26	Warstwa analityczna	Częściowo zrealizowane	Istnieje model wymiarowy gold; moduł <code>app/layers/analytics</code> jest szkieletem	Brak raportów, metryk i projekcji analitycznych
N-01	Architektura modułowa i warstwowa	Zrealizowane	Oddzielne katalogi warstw oraz podział <code>api/service/repository/models/schemas</code>	Struktura jest używana w głównym pipeline'ie
N-02	Idempotencja wybranych operacji	Zrealizowane	Ochrona przed ponownym stagingiem, zastępowanie kandydatów, stabilne grupowanie i ponowne użycie wymiarów	Testy idempotencji w plikach <code>test_staging_mapper.py</code> i <code>test_integration_golden_*.py</code>
N-03	Konfigurowalność środowiska	Zrealizowane	<code>app/core/config.py</code> , <code>.env</code> , parametry DAG, ConfigMap i Secrets	Wartości sekretów nie powinny być wersjonowane ani publikowane
N-04	Lokalne środowisko kontenerowe	Zrealizowane	<code>docker-compose.yml</code> , obrazy aplikacji i trwałe wolumeny	Dotyczy środowiska developerskiego
N-05	Wdrożenie OpenShift	Przygotowane, niezwerifikowane	Manifesty w katalogu <code>openshift/</code>	Nie zostały przetestowane na docelowym klastrze; zestaw nie odzwierciedla w pełni aktualnego Compose
N-06	Bezpieczeństwo dostępu do API	Niezrealizowane	Konfiguracja i sekrety są oddzielone od kodu, ale brak mechanizmu użytkowników i uprawnień	Brak uwierzytelniania i autoryzacji
N-07	Testy funkcjonalne	Zrealizowane w kodzie testów	Katalog <code>tests/</code> obejmuje główne reguły biznesowe	W repozytorium znajduje się 143 testów jednostkowych i integracyjnych
N-08	Testy wydajnościowe	Niezrealizowane	Brak scenariuszy obciążeniowych i pomiarów	Istnieje jedynie limit bezpieczeństwa liczby par matchingu
N-09	Diagnostyka i obsługa błędów	Zrealizowane	Wyjątki warstw, odpowiedzi HTTP 400/500, ProcessLog	Zakres logowania zależy od etapu procesu

4.4. Podsumowanie oceny

Najpełniej zrealizowana jest centralna część platformy: pobieranie plików i danych relacyjnych, przechowywanie RAW, mapowanie, preprocessing, walidacja, dwuetapowy matching, grupowanie, budowa Golden Record, audytowalność oraz odczyt wyników przez API. Elementy te posiadają odpowiadające modele danych, endpointy API i testy.

Warstwa servingowa oferuje odczyt Golden Record i danych audytowych, ale nie obsługuje aktywnego przekazywania zmian do systemów zewnętrznych. Warstwa analityczna pozostaje szkieletem, frontend udostępnia już odczyt walidacji i matchingu, lecz nadal nie obejmuje pełnego widoku Golden Record, a Neo4j nie jest częścią pipeline'u.

Nie wykonano również mechanizmu samouczącego, wejść strumieniowych, integracji z zewnętrznymi usługami REST lub SOAP, procesu manualnej obsługi decyzji REVIEW, zabezpieczeń API i testów wydajnościowych. Ograniczenia te nie wpływają na działanie wykonanego rdzenia integracyjnego, lecz oznaczają, że platforma nie realizuje pełnego zakresu rozwiązania docelowego opisanego w materiałach wejściowych.

5. Źródła danych i zakres informacyjny

Platforma została przygotowana do integracji danych o osobach i podmiotach pochodzących z wielu źródeł. W repozytorium znajdują się zestawy plikowe odwzorowujące struktury wybranych rejestrów publicznych i branżowych oraz demonstracyjny system relacyjny Oracle. Dane służą do sprawdzania mapowania, walidacji, matchingu, grupowania i budowy Golden Record.

Wszystkie dane plikowe używane w projekcie są syntetyczne. Nazwy zbiorów i układ kolumn nawiązują do rzeczywistych rejestrów, jednak rekordy nie stanowią kopii danych produkcyjnych ani oficjalnych eksportów. Pozwala to wykonywać testy integracji bez przetwarzania rzeczywistych danych osobowych i biznesowych. Syntetyczny charakter ma również baza Oracle utworzona na potrzeby demonstracji importu relacyjnego.

5.1. Systemy źródłowe

Kod warstwy ingestion rozpoznaje jedenaście systemów źródłowych. Dziesięć z nich odpowiada zestawom plikowym, natomiast INSURANCE_CORE jest relacyjnym źródłem demonstracyjnym.

Kod źródła	Zakres danych	Typ encji	Najważniejsze informacje
PESEL	dane osób	PERSON	PESEL, imiona, nazwiska, data i miejsce urodzenia, płeć, obywatelstwo, dokumenty tożsamości, adresy i dane kontaktowe
CEIDG	działalność gospodarcza i właściciel	PARTY, PERSON	nazwa firmy, NIP, REGON, dane działalności, adresy, kontakt oraz dane właściciela
KRS	podmioty rejestrowe i ich relacje	PARTY, PERSON	KRS, NIP, REGON, nazwa, forma prawna, status, adres, osoby reprezentujące, wspólnicy i okresy relacji
REGON	dane podmiotów	PARTY	REGON, NIP, KRS, nazwa, forma prawna, forma własności, adres, PKD i data powstania
VAT	dane podatników VAT	PARTY	nazwa, NIP, REGON, KRS, status VAT, adresy, rachunki bankowe i daty rejestracyjne
GLEIF	dane podmiotów posiadających LEI	PARTY	LEI, nazwa prawna, forma prawna, jurysdykcja, adres, status rejestracji oraz informacje o podmiocie dominującym
KNF_AGENT	pośrednicy ubezpieczeniowi	PARTY, PERSON	numer agenta, firma, NIP, KRS, zakład ubezpieczeń, typ agenta, adres oraz dane osoby
KNF_PRACOWNIK_AGENTA	pracownicy agentów i dane agentów	PARTY, PERSON	numer pracownika, dane osoby, PESEL, dane agenta, NIP, KRS i okres wpisu
KNF_FIRMY_INWESTYCYJNE	firmy inwestycyjne i członkowie zarządu	PARTY, PERSON	nazwa, KRS, NIP, REGON, adres, zakres czynności, decyzja oraz dane członka zarządu
KNF_PIENIADZ_ELEKTRONICZNY	dostawcy i wydawcy pieniądza elektronicznego	PARTY	numer UKNF, nazwa, typ podmiotu, status, daty, KRS, NIP i adres
INSURANCE_CORE	demonstracyjny system ubezpieczeniowy Oracle	PARTY, PERSON	dane klientów, identyfikatory, adresy, kontakty, rachunki, umowy, role osób i relacje pomiędzy klientami

Lista dozwolonych kodów znajduje się w stałej SUPPORTED_SOURCE_SYSTEMS w `app/layers/ingestion/service.py`. Kod źródła jest zapisywany w tabeli `meta.SourceSystem` i wiąże późniejsze dane z poziomem zaufania, mapowaniem kolumn oraz regułami survivorship.

5.2. Zbiory plikowe

Katalog data zawiera dziesięć logicznych zestawów danych w czterech formatach:

- CSV,
- JSON,
- XML,
- XLSX.

Każdy zestaw zawiera po 200 rekordów w każdym formacie, czyli łącznie 800 rekordów na logiczne źródło. Manifest `data/manifest.json` opisuje liczebność plików. Dla dziesięciu źródeł daje to łącznie 8000 syntetycznych rekordów plikowych, przy czym rekordy w poszczególnych formatach stanowią różne dane, a nie cztery identyczne kopie tego samego zbioru.

Pliki CSV są kodowane w UTF-8 z BOM. W plikach XML nazwy pól są zapisywane w atrybutach `name`, co pozwala bezpiecznie obsługiwać nazwy zawierające spacje i polskie znaki. Pliki XLSX mają prostą strukturę tabelaryczną. Dokumenty JSON mogą zawierać pojedynczy obiekt albo listę obiektów.

Warstwa RAW sprawdza rozszerzenie oraz podstawową poprawność zawartości. Pełne parsowanie następuje dopiero podczas ładowania do stagingu. Wszystkie cztery formaty są ostatecznie przekształcane do listy rekordów słownikowych, dzięki czemu późniejsze mapowanie nie zależy od formatu wejściowego.

Obsługa formatów jest realizowana przez:

- `SUPPORTED_FILE_TYPES` i `import_raw_file()` w `app/layers/ingestion/service.py`,
- `parse_raw_file_records()` w `app/layers/staging_validation/service.py`,
- `parse_xlsx_records()` i `parse_xml_records()` w tym samym module.

5.3. Modele PERSON i PARTY

Źródła są mapowane do jednego albo dwóch typów encji. `PERSON` reprezentuje osobę fizyczną, natomiast `PARTY` reprezentuje podmiot, organizację albo działalność gospodarczą.

Model `PERSON` obejmuje w szczególności:

- PESEL,
- numer dowodu osobistego i paszportu,
- pierwsze i drugie imię,
- nazwisko i nazwisko rodowe,
- datę i miejsce urodzenia,
- płeć i obywatelstwo,
- telefon i adres e-mail,
- elementy adresu.

Model `PARTY` obejmuje w szczególności:

- nazwę i nazwę skróconą,
- formę prawną,
- kraj rejestracji i datę powstania,
- identyfikatory NIP, REGON, KRS i LEI,
- dane rejestrowe i statusy,
- zakres działalności,
- telefon, adres e-mail i stronę internetową,
- rachunki bankowe,
- elementy adresu,
- informacje o osobach i podmiotach powiązanych.

CEIDG, KRS, rejestr agentów, rejestr pracowników agentów i rejestr firm inwestycyjnych zawierają informacje dotyczące obu modeli. Przykładowo rekord CEIDG opisuje działalność jako PARTY, a dane właściciela jako PERSON. Rekord KRS może zawierać główny podmiot oraz osoby reprezentujące i inne podmioty pozostające z nim w relacji.

Rozdzielenie encji następuje na etapie stagingu. Dla jednego pliku można wykonać osobne ładowanie do `Person_Staging` i `Party_Staging`, jeżeli dla danego źródła istnieją mapowania obu typów.

5.4. Mapowanie kolumn

Każde źródło może używać innego nazewnictwa i struktury danych. Mapowania są przechowywane w tabeli `meta.ColumnMapping` i wiążą:

- system źródłowy,
- typ encji,
- nazwę kolumny źródłowej,
- nazwę pola kanonicznego.

Przykładowo pole `firma.wlasciciel.pesel` z CEIDG jest mapowane na PESEL encji PERSON, a pole `firma.nip` na identyfikator encji PARTY. W źródle Oracle pola `TAX_REF`, `STAT_REG_REF` i `COURT_REF` są przekształcane odpowiednio do identyfikatorów NIP, REGON i KRS.

Mapowanie nie ogranicza się do prostego przypisania jednej kolumny. `app/layers/staging_validation/mapper.py`:

- wyszukuje wartości bez rozróżniania wielkości liter,
- obsługuje zagnieżdżone ścieżki dokumentów,
- ujednolica nazwy identyfikatorów,
- składa wiele kolumn do `Identifiers_JSON`,
- zachowuje pierwszą niepustą wartość przy powtarzających się polach,
- zapisuje oryginalny rekord w `Raw_Record_JSON`.

Szczególną obsługę posiada KRS. Szerokie zestawy kolumn opisujące członków zarządu, prokurentów, wspólników i inne powiązania są grupowane do struktur JSON. Struktura plików przewiduje do dziesięciu slotów dla danego rodzaju relacji, natomiast generator danych wypełnia maksymalnie cztery i pozostawia pozostałe puste. Dzięki temu dane relacyjne nie muszą być przechowywane jako duża liczba osobnych kolumn stagingowych.

Definicje systemów źródłowych i mapowań znajdują się w `scripts/init_proposed_mssql_schema.sql`. Samo automatyczne utworzenie tabel przez SQLAlchemy nie wypełnia `ColumnMapping`, dlatego do działania mapowania na przygotowanej bazie wymagane jest wykonanie skryptu inicjalizacyjnego.

5.5. Demonstracyjne źródło Oracle

Oracle Insurance Core symuluje relacyjny system dziedziny. Struktura jest tworzona przez `scripts/init_oracle_insurance_core.sql` i obejmuje między innymi:

- klientów,
- jednostki organizacyjne,
- identyfikatory klientów,
- adresy,
- historię kontaktów,
- rachunki płatnicze,
- umowy i strony umów,
- przypisania agentów,
- relacje pomiędzy klientami.

Warstwa ingestion wykonuje przygotowane zapytania dla encji PARTY i PERSON. Wynik zapytania jest wzbogacany o role osób i relacje pomiędzy podmiotami, a następnie serializowany do JSON. Taki snapshot zostaje zapisany w

`raw.RawFile` i przechodzi przez ten sam pipeline co dane dostarczone w plikach.

Ogólna nazwa zapytania `insurance_core_export` jest rozwijana do właściwego zapytania zależnie od wybranego typu encji. Implementacja znajduje się w:

- `RELATIONAL_QUERY_DEFINITIONS`,
- `resolve_relational_query_definition()`,
- `extract_relational_records()`,
- `import_relational_source()`

w pliku `app/layers/ingestion/service.py`.

Poprawność importu i tworzenia snapshotów sprawdzają testy w `tests/test_relational_ingestion.py`.

5.6. Dane referencyjne TERYT

Pliki `SIMC.csv` i `ULIC.csv` są wykorzystywane przez warstwę `validation` do sprawdzania miejscowości oraz ulic. Dane są ładowane do katalogu dostępnego dla API, a następnie przetwarzane do indeksów używanych podczas walidacji.

Walidacja TERYT tworzy między innymi wyniki reguł:

- `ADDR_TERYT_CITY_EXISTS`,
- `ADDR_TERYT_STREET_EXISTS`.

Wynik weryfikacji adresu jest wykorzystywany nie tylko jako informacja o jakości. Podczas wyboru wartości `Golden Record` adres potwierdzony w TERYT może otrzymać pierwszeństwo przed innym wariantem.

Obsługę danych TERYT realizują:

- endpoint `POST /layers/validation/teryt-load`,
- funkcje `load_teryt_index()`, `validate_teryt_city_exists()` i `validate_teryt_street_exists()` w `app/layers/validation/service.py`,
- zadanie `teryt_load` w DAG Airflow.

Testy znajdują się w `tests/test_teryt_validation.py`.

5.7. Automatyczne rozpoznawanie źródła w Airflow

DAG może automatycznie wyznaczyć kod źródła na podstawie nazwy pliku. Słownik `SOURCE_SYSTEM_BY_FILE_STEM` mapuje nazwę bazową, na przykład `pesel` albo `regon`, na kod systemu źródłowego.

Airflow określa również typ encji:

- `GLEIF`, rejestr pieniądza elektronicznego, `PESEL`, `REGON` i `VAT` mają jeden domyślny typ encji,
- `CEIDG`, `KRS`, rejestr agentów, rejestr pracowników agentów i rejestr firm inwestycyjnych są przetwarzane zarówno jako `PARTY`, jak i `PERSON`,
- źródło relacyjne może zwrócić oba typy encji,
- użytkownik może nadpisać automatyczny wybór parametrem DAG.

Automatyczne rozpoznawanie wymaga zachowania znanej nazwy pliku. Dla innej nazwy należy jawnie podać `source_system_code`. Podobnie nieznanym lub niestandardowym układem kolumn wymaga dodania odpowiednich rekordów do `meta.ColumnMapping`.

5.8. Dane syntetyczne

Zbiory syntetyczne zostały przygotowane tak, aby umożliwiały sprawdzanie zarówno poprawnych przypadków, jak i kontrolowanych anomalii. Zawierają między innymi:

- literówki w nazwach i nazwiskach,
- niepełne adresy,
- błędne lub brakujące identyfikatory,

- warianty tego samego rekordu występujące w różnych źródłach,
- osoby i podmioty powiązane,
- daty obowiązywania relacji,
- różnice danych kontaktowych.

Adresy bazowe są budowane z kuratorowanej listy kombinacji województwa, powiatu, gminy, miejscowości, ulicy i kodu pocztowego. Generator celowo wprowadza niewielki odsetek anomalii, aby walidacja i matching nie były sprawdzane wyłącznie na danych idealnych.

Regenerację danych wykonuje skrypt:

`scripts/refine_synthetic_data.js`

Jakość wygenerowanych zbiorów jest kontrolowana w `tests/test_synthetic_data_quality.py`. Testy sprawdzają między innymi brak nieuzasadnionego współdzielenia dokumentów tożsamości, spójność identyfikatorów podmiotów, chronologię dat oraz zgodność części danych osobowych.

5.9. Odniesienie do implementacji

Obszar	Lokalizacja
Lista źródeł i formatów	<code>app/layers/ingestion/service.py</code>
Import plikowy i relacyjny	<code>app/layers/ingestion/api.py</code> , <code>app/layers/ingestion/service.py</code>
Automatyczny wybór źródła i encji	<code>airflow/dags/goldenizacja_pipeline.py</code>
Model mapowań	<code>app/layers/staging_validation/models.py</code>
Logika mapowania	<code>app/layers/staging_validation/mapper.py</code>
Definicje mapowań źródeł	<code>scripts/init_proposed_mssql_schema.sql</code>
Dane plikowe	<code>data/csv</code> , <code>data/json</code> , <code>data/xml</code> , <code>data/xlsx</code>
Manifest liczebności	<code>data/manifest.json</code>
Generator danych syntetycznych	<code>scripts/refine_synthetic_data.js</code>
Demonstracyjna baza Oracle	<code>scripts/init_oracle_insurance_core.sql</code>
Dane referencyjne TERYT	<code>data/teryt</code>
Testy importu relacyjnego	<code>tests/test_relational_ingestion.py</code>
Testy mapowania	<code>tests/test_staging_mapper.py</code>
Testy danych syntetycznych	<code>tests/test_synthetic_data_quality.py</code>
Testy TERYT	<code>tests/test_teryt_validation.py</code>

6. Architektura systemu

Platforma została zbudowana jako aplikacja warstwowa, w której kolejne etapy przetwarzania są udostępniane przez wspólny interfejs FastAPI i uruchamiane w określonej kolejności przez Apache Airflow. Podstawowym repozytorium jest Microsoft SQL Server. Przechowuje on metadane procesu, dane RAW, rekordy pośrednie oraz wynikowe encje Golden Record. Oracle pełni rolę demonstracyjnego źródła relacyjnego.

Architekturę należy rozpatrywać na trzech poziomach:

- logicznym, opisującym podział procesu na warstwy,
- aplikacyjnym, opisującym organizację kodu backendu,
- uruchomieniowym, opisującym współpracę kontenerów i baz danych.

6.1. Architektura logiczna

Główny przepływ danych składa się z pięciu funkcjonalnych obszarów. W kodzie część z nich została rozdzielona na bardziej szczegółowe moduły.

Pobieranie i rejestracja danych

Warstwa ingestion odpowiada za przyjęcie pliku albo wykonanie zapytania do źródła relacyjnego. Tworzy wpis systemu źródłowego i partii importu, zapisuje oryginalną zawartość w warstwie RAW oraz rozpoczyna rejestrowanie przebiegu procesu.

Przygotowanie i kontrola danych

Obszar stagingu i walidacji został podzielony na trzy moduły:

- `staging_validation` parsuje dane i mapuje je do wspólnego modelu,
- `preprocessing` normalizuje wartości wykorzystywane w walidacji i matchingu,
- `validation` wykonuje reguły jakości oraz sprawdza adresy z użyciem TERYT.

Rozdzielenie preprocessingu od walidacji pozwala wykonać reguły jakości na danych doprowadzonych do porównywalnej postaci, zachowując jednocześnie dane stagingowe i źródłowe.

Integracja i goldenizacja

Warstwa `integration_golden` odpowiada za:

- wyznaczanie kandydatów do połączenia,
- `scoring` Levenshteina,
- ponowną ocenę Jaro-Winklera,
- klasyfikację decyzji,
- grupowanie rekordów,
- wybór wartości wynikowych,
- tworzenie i aktualizowanie wymiarów GOLD,
- zapis lineage i zmian danych.

Jest to najbardziej rozbudowana warstwa aplikacji i główny obszar realizujący logikę biznesową projektu.

Analityka

Warstwa analytics została przewidziana jako miejsce dla metryk, raportów, projekcji grafowych i struktur analitycznych. W obecnym kodzie zawiera jedynie szkielet modułu i endpoint statusowy. Część modelu wymiarowego powstaje już podczas goldenizacji, ale nie wykonano osobnej logiki analitycznej.

Udostępnianie

Warstwa serving udostępnia odczytowe widoki danych wynikowych. Obejmuje listy i szczegóły Golden Record, wyszukiwanie osób i podmiotów, lineage, historię zmian, wyniki walidacji, kandydatów matchingu, porównanie rekordów oraz liczniki kolejnych etapów. Endpointy stosują paginację i opcjonalne filtry. Warstwa nie realizuje

mechanizmu PUSH ani webhooków.

6.2. Architektura aplikacji FastAPI

Punktem wejścia backendu jest `app/main.py`. Podczas uruchamiania aplikacji wykonywana jest funkcja `init_db()`, która:

1. oczekuje na dostępność SQL Servera,
2. tworzy bazę goldenizacja, jeżeli jeszcze nie istnieje,
3. tworzy tabele zarejestrowane w metadanych SQLAlchemy.

Aplikacja konfiguruje middleware CORS i dołącza dwa główne routery:

- `app.api.routes` zawiera techniczne endpointy demonstracyjne,
- `app.layers.router` łączy endpointy właściwego pipeline'u danych.

Router warstw używa wspólnego prefiksu `/layers` i dołącza moduły w kolejności odpowiadającej przepływowi danych:

1. ingestion,
2. staging validation,
3. preprocessing,
4. validation,
5. integration golden,
6. analytics,
7. serving.

Każda główna warstwa stosuje zbliżony podział plików:

Plik	Odpowiedzialność
<code>api.py</code>	endpointy HTTP, walidacja parametrów i mapowanie błędów na odpowiedzi HTTP
<code>service.py</code>	logika biznesowa warstwy
<code>repository.py</code>	operacje bazodanowe i oddzielenie logiki od SQLAlchemy
<code>models.py</code>	modele ORM i odwzorowanie tabel
<code>schemas.py</code>	modele żądań i odpowiedzi API

Warstwa stagingu posiada dodatkowo `mapper.py`, ponieważ mapowanie danych źródłowych do modelu kanonicznego stanowi oddzielny i rozbudowany obszar odpowiedzialności.

Taki podział umożliwia testowanie logiki biznesowej bez uruchamiania pełnego środowiska bazodanowego. Testy integracji i goldenizacji wykorzystują repozytoria zastępcze, które implementują kontrakt potrzebny serwisom.

6.3. Główne komponenty

FastAPI

Backend udostępnia operacje pipeline'u, zarządza logiką biznesową i połączeniem z Microsoft SQL Serverem. Do komunikacji z bazą wykorzystywane są SQLAlchemy i sterownik ODBC Driver 18 for SQL Server.

FastAPI generuje również opis OpenAPI oraz interfejs Swagger dostępny w środowisku lokalnym pod `/docs`.

Apache Airflow

Airflow jest orkiestratorem procesu. Nie przetwarza danych bezpośrednio w swojej bazie, lecz wywołuje endpointy FastAPI przez HTTP. Identyfikatory plików RAW oraz wyniki kroków są przekazywane pomiędzy zadaniami za pomocą XCom.

DAG jest uruchamiany ręcznie i nie posiada harmonogramu cyklicznego. Pozwala to wybrać plik, źródło, typ encji oraz parametry matchingu dla konkretnego przebiegu.

Microsoft SQL Server

SQL Server jest centralnym magazynem danych platformy. Schematy bazy rozdzielają odpowiedzialności:

- meta przechowuje źródła, partie, mapowania i logi,
- raw przechowuje oryginalne dane wejściowe,
- stg przechowuje dane stagingowe, preprocessingowe, walidacyjne i kandydatów matchingu,
- gold przechowuje wynikowe osoby, podmioty, adresy, identyfikatory, relacje, lineage i rejestr zmian.

Modele ORM są używane przez aplikację, natomiast `scripts/init_proposed_mssql_schema.sql` zawiera pełniejszą definicję struktury, ograniczeń, indeksów, słowników źródeł i mapowań kolumn.

Oracle

Oracle działa jako demonstracyjny system dziedziny. FastAPI łączy się z nim bezpośrednio podczas wywołania importu relacyjnego. Wynik zapytań jest serializowany do JSON i zapisywany w SQL Serverze jako snapshot RAW. Od tego momentu dane relacyjne przechodzą przez te same etapy co dane plikowe.

React

Frontend React jest osobną aplikacją komunikującą się z FastAPI przez HTTP. W środowisku lokalnym działa przez serwer developerski Vite. W przygotowanym obrazie wdrożeniowym aplikacja jest budowana statycznie i serwowana przez Nginx.

Aktualny frontend wykorzystuje endpointy odczytowe warstwy serving. Udostępnia dwa główne widoki:

- tabelę wyników walidacji z filtrowaniem i paginacją,
- tabelę kandydatów matchingu dla Levenshteina i Jaro-Winklera wraz z porównaniem szczegółów pary rekordów.

Interfejs nie obsługuje jeszcze pełnego profilu Golden Record, ręcznej obsługi przypadków REVIEW ani operacji zapisu do pipeline'u. Nie jest więc pełnym interfejsem biznesowym systemu, ale przestał być już wyłącznie testem technicznego połączenia z API.

Neo4j

Neo4j znajduje się w konfiguracji Docker Compose i posiada połączenie z demonstracyjnym modułem dokumentów w `app/api/routes.py`. Główny pipeline goldenizacji nie zapisuje jednak danych do Neo4j i nie odczytuje z niego relacji.

Komponent nie został przetestowany jako część procesu integracji osób i podmiotów. Z tego powodu na diagramie architektury jest oznaczony jako element demonstracyjny, a nie część działającej ścieżki przetwarzania.

6.4. Komunikacja pomiędzy komponentami

Podstawowa ścieżka wykonania wygląda następująco:

1. Użytkownik uruchamia DAG w Airflow i przekazuje parametry wejściowe.
2. Airflow odczytuje plik z zamontowanego katalogu data albo zleca API pobranie danych z Oracle.
3. Airflow wywołuje kolejne endpointy FastAPI przez HTTP.
4. FastAPI wykonuje logikę właściwej warstwy.
5. Repozytoria zapisują lub odczytują dane z Microsoft SQL Servera.
6. Wyniki operacji wracają do Airflow i są przekazywane do kolejnych zadań.

Frontend komunikuje się bezpośrednio z FastAPI, ale nie uczestniczy w wykonywaniu DAG. Neo4j jest osiągany wyłącznie przez demonstracyjne endpointy dokumentów.

6.5. Konfiguracja

Konfiguracja backendu jest zdefiniowana w klasie Settings w `app/core/config.py`. Wartości są pobierane ze zmiennych środowiskowych oraz pliku `.env`. Obejmują między innymi:

- adres i dane dostępowe SQL Servera,

- adres i dane dostępowe Neo4j,
- parametry Oracle,
- ścieżkę wolumenu pomocniczego,
- listę dozwolonych źródeł CORS,
- nazwę i port aplikacji.

Połączenie z SQL Serverem jest budowane przez `sqaalchemy.engine.URL`, co zapewnia poprawne kodowanie parametrów i hasła. Sesja SQLAlchemy jest tworzona osobno dla każdego żądania API.

Sekrety nie powinny być zapisywane w dokumentacji ani publikowane w repozytorium. W środowisku OpenShift przewidziano oddzielne zasoby ConfigMap i Secret.

6.6. Środowisko lokalne

`docker-compose.yml` definiuje sześć usług:

Usługa	Rola	Port lokalny
api	FastAPI i logika pipeline'u	8000
frontend	React/Vite	5173
airflow	orkiestracja DAG	8080
mssql	centralne repozytorium danych	1433
oracle	demonstracyjne źródło relacyjne	1521
neo4j	demonstracyjna baza grafowa	7474 i 7687

API oczekuje na poprawne uruchomienie SQL Servera, Oracle i Neo4j. Zależność od Neo4j wynika z konfiguracji środowiska i modułu demonstracyjnego, a nie z wymagań głównego pipeline'u.

Wolumeny zapewniają trwałość baz danych, logów Neo4j, środowiska Airflow i pomocniczych plików aplikacji. Katalog data jest montowany do kontenera Airflow tylko do odczytu.

Środowisko Docker Compose jest środowiskiem developerskim. API i frontend pracują w trybie umożliwiającym szybkie przeladowanie zmian, a Airflow używa własnego obrazu z `apache/airflow:3.2.2-python3.14`, menedżera uwierzytelniania FAB oraz uruchamia w jednym kontenerze procesy `api-server`, `scheduler`, `dag-processor` i `triggerer`. Lokalny executor pozostaje `SequentialExecutor`.

6.7. Przygotowane zasoby OpenShift

Katalog openshift zawiera manifesty:

- ConfigMap i Secret,
- PersistentVolumeClaim,
- Deployment i Service dla SQL Servera,
- Deployment i Service dla Neo4j,
- Deployment i Service dla API,
- Deployment i Service dla frontendu,
- Deployment i Service dla Airflow,
- Route dla usług udostępnianych przez HTTP,
- ConfigMap zawierającą przykładowy DAG.

Manifesty nie zostały przetestowane na docelowym klastrze. Nie obejmują także usługi Oracle używanej w aktualnym środowisku Docker Compose. Konfiguracja Airflow w OpenShift zawiera jedynie przykładowy DAG `goldenizacja_example`, a nie właściwy pipeline `goldenizacja_pipeline`. Zawierają również wartości wymagające zastąpienia dla konkretnego projektu i tras klastra. Stanowią więc podstawę wdrożenia, ale nie są potwierdzonym odwzorowaniem działającego środowiska lokalnego.

OpenShift nie jest uwzględniany jako część potwierdzonej ścieżki wykonania systemu. Szczegółowy opis manifestów i

rozbieżności zostanie przedstawiony w rozdziale dotyczącym wdrożenia.

6.8. Ograniczenia architektury

Najważniejsze ograniczenia obecnej architektury obejmują:

- brak uwierzytelniania i autoryzacji API,
- synchroniczne wykonywanie operacji warstw przez endpointy HTTP,
- SequentialExecutor w lokalnym Airflow,
- brak mechanizmu PUSH i webhooków dla zmian Golden Record,
- brak biznesowego interfejsu użytkownika,
- niewykorzystanie Neo4j w głównym pipeline'ie,
- nieprzetestowane manifesty OpenShift,
- brak Oracle w przygotowanych manifestach OpenShift,
- brak mechanizmu kolejkowego dla długotrwałych operacji,
- zależność stagingu od mapowań inicjalizowanych skryptem SQL.

Ograniczenia nie zmieniają podziału odpowiedzialności w działającym rdzeniu systemu, ale mają znaczenie przy ocenie gotowości rozwiązania do pracy poza środowiskiem demonstracyjnym.

6.9. Odniesienie do implementacji

Obszar	Lokalizacja
Inicjalizacja aplikacji	app/main.py
Konfiguracja	app/core/config.py
Router główny warstw	app/layers/router.py
Połączenie SQL Server	app/db/sql.py
Inicjalizacja bazy	app/db/init_db.py
Połączenie Neo4j	app/db/neo4j.py
Endpointy demonstracyjne	app/api/routes.py
Konsumenckie API danych	app/layers/serving/
Warstwy biznesowe	app/layers/
DAG	airflow/dags/goldenizacja_pipeline.py
Środowisko lokalne	docker-compose.yml
Obraz API	Dockerfile
Obraz frontendu	frontend/Dockerfile
Manifesty OpenShift	openshift/

7. Technologie i narzędzia

Dobór technologii podporządkowano warstwowemu przetwarzaniu danych, możliwości uruchomienia kompletnego środowiska lokalnego oraz oddzieleniu logiki biznesowej od sposobu składowania i udostępniania danych. Główny pipeline został wykonany w Pythonie, dane są przechowywane w Microsoft SQL Serverze, a kolejne etapy koordynuje Apache Airflow. Środowisko lokalne uruchamiane jest za pomocą Docker Compose.

Wersje bibliotek backendu są przypięte w `requirements.txt`. W przypadku frontendu `package.json` określa dopuszczalne zakresy wersji, natomiast `package-lock.json` zapisuje wersje faktycznie rozwiązane podczas instalacji.

7.1. Zestawienie technologii

Technologia	Wersja w projekcie	Zastosowanie	Status wykorzystania
Python	3.14	język backendu i zadań Airflow	główny element systemu
FastAPI	0.136.3	REST API i organizacja endpointów warstw	używane w głównym pipeline
Uvicorn	0.49.0	serwer ASGI aplikacji FastAPI	używane lokalnie i w obrazie API
Pydantic	2.13.4	modele danych wejściowych i odpowiedzi API	używane w głównym pipeline
pydantic-settings	2.14.1	konfiguracja ze zmiennych środowiskowych	używane w backendzie
SQLAlchemy	2.0.50	ORM, sesje i operacje bazodanowe	używane w głównym pipeline
PyODBC	5.3.0	komunikacja SQLAlchemy z SQL Serverem	używane w głównym pipeline
ODBC Driver 18 for SQL Server	instalowany w obrazie API	sterownik połączenia z Microsoft SQL Serverem	używane w obrazie backendu
Microsoft SQL Server	obraz 2022 latest	centralne repozytorium meta, raw, stg i gold	używane w głównym pipeline
Oracle Database Free	obraz 23 slim faststart	demonstracyjne źródło relacyjne	używane jako źródło testowe
python-oracledb	4.0.1	połączenie z Oracle bez klienta natywnego	używane przez import relacyjny
Apache Airflow	3.2.2, Python 3.14	orkiestracja kolejnych etapów procesu	używane w głównym pipeline
FAB Auth Manager	3.6.5	logowanie i zarządzanie kontami Airflow	używane przez interfejs Airflow
RapidFuzz	3.14.5	Levenshtein i Jaro-Winkler	używane w matchingu
python-stdnum	2.2	walidacja identyfikatorów	używane w walidacji
email-validator	2.3.0	kontrola składni adresów e-mail	używane w walidacji
dnspython	2.8.0	opcjonalna kontrola domen pocztowych	używane w walidacji
phonenumbers	9.0.32	normalizacja numerów telefonów	używane w preprocessingu
text-unidecode	1.3	uproszczenie tekstu na potrzeby porównań	używane w preprocessingu
openpyxl	3.1.5	odczyt plików XLSX	używane przez ingestion i staging
python-multipart	0.0.32	obsługa formularzy i uploadu plików w FastAPI	używane przez API
Neo4j Python Driver	6.2.0	komunikacja backendu z bazą grafową	używane przez demonstracyjną obsługę dokumentów
React	18.3.1	frontend prezentujący dane z warstwy serving	używane w zakresie widoków walidacji i matchingu
Vite	5.4.21	środowisko developerskie i budowanie frontendu	używane przez frontend
Node.js	obraz 20 Alpine	uruchamianie i budowanie frontendu	używane lokalnie i w buildzie
Nginx unprivileged	stable Alpine	serwowanie zbudowanego frontendu	przygotowane w obrazie wdrożeniowym
Neo4j	5.24	demonstracyjna baza grafowa	poza głównym pipeline, nieprzetestowane
Docker Compose	plik Compose w repozytorium	lokalne uruchomienie usług	podstawowe środowisko developerskie
OpenShift	manifesty YAML	przygotowanie zasobów wdrożeniowych	nieprzetestowane na docelowym klastrze

7.2. Backend

Python

Backend i DAG Airflow działają na Pythonie 3.14. Obraz API bazuje na python:3.14-slim-bookworm, zgodnym z repozytorium ODBC Driver 18 dla Debiana 12. Obraz Airflow rozszerza apache/airflow:3.2.2-python3.14 o FAB Auth Manager.

FastAPI i Uvicorn

FastAPI udostępnia operacje warstw przez REST, waliduje parametry oraz generuje OpenAPI i interfejs Swagger. Uvicorn pełni funkcję serwera ASGI. W Docker Compose uruchamiany jest z opcją automatycznego przeładowania kodu, co odpowiada środowisku developerskiemu. Obraz API posiada również standardową komendę uruchamiającą serwer bez opcji developerskich.

Pydantic i konfiguracja

Pydantic definiuje modele danych API, a pydantic-settings pobiera konfigurację ze zmiennych środowiskowych i pliku .env. Klasa Settings w app/core/config.py obejmuje parametry aplikacji, baz danych i CORS.

7.3. Dostęp do danych

SQLAlchemy

SQLAlchemy 2.0 odpowiada za modele ORM, sesje i operacje repozytoriów. Sesja jest tworzona dla każdego żądania, a pool_pre_ping sprawdza połączenie przed użyciem. Pełna definicja bazy, obejmująca również ograniczenia, indeksy, słowniki i mapowania, znajduje się w scripts/init_proposed_mssql_schema.sql.

Microsoft SQL Server

Microsoft SQL Server 2022 jest centralnym repozytorium schematów meta, raw, stg i gold. Połączenie wykorzystuje mssql+pyodbc i ODBC Driver 18, a dane RAW są przechowywane jako VARBINARY(MAX) mapowany przez LargeBinary.

Oracle

Oracle Database Free 23 jest demonstracyjnym źródłem relacyjnym inicjalizowanym przez scripts/init_oracle_insurance_core.sql. Backend łączy się przez ODBC albo bibliotekę oracledb, a wyniki zapytań zapisuje jako snapshoty JSON w warstwie RAW SQL Servera.

7.4. Orkiestracja

Apache Airflow wywołuje przez HTTP kolejne operacje FastAPI. Lokalnie używa SequentialExecutor, a API server, scheduler, DAG processor i triggerer działają w jednym kontenerze. FAB Auth Manager zapewnia logowanie kontem administratora skonfigurowanym przez zmienne środowiskowe. Szczegóły przepływu i parametrów opisano w rozdziale 9.

7.5. Biblioteki przetwarzania danych

RapidFuzz

RapidFuzz dostarcza implementacje Levenshteina i Jaro-Winklera używane do oceny podobieństwa tekstowego oraz ponownej oceny kandydatów matchingu.

Biblioteki walidacyjne

python-stdnum wspiera sprawdzanie identyfikatorów, natomiast część reguł kontrolnych została zaimplementowana również bezpośrednio w warstwie validation. Dotyczy to między innymi sum kontrolnych i zależności wynikających z numeru PESEL.

email-validator sprawdza składnię adresów e-mail. Jeżeli parametr procesu włącza kontrolę DNS, dnspython jest używany do wyszukania rekordów domeny.

Biblioteki normalizacyjne

phonenumbers standaryzuje numery telefonów do wspólnego formatu. text-unidecode upraszcza znaki tekstowe wykorzystywane podczas porównań. Funkcje aplikacji uzupełniają działanie bibliotek o reguły właściwe dla polskich

danych, nazw podmiotów, form prawnych i adresów.

openpyxl

openpyxl umożliwia odczyt plików XLSX. Arkusze są otwierane w trybie tylko do odczytu, co ogranicza niepotrzebne zużycie pamięci podczas parsowania większych plików.

7.6. Frontend

Frontend wykorzystuje React 18 i Vite. Lokalnie działa na obrazie node:20-alpine, natomiast wieloetapowy frontend/Dockerfile buduje pliki statyczne i przekazuje je do Nginx unprivileged. Aplikacja korzysta z endpointów warstwy serving i udostępnia widoki walidacji oraz matchingu, w tym filtrowanie, paginację i porównanie wybranej pary rekordów. Zakres nadal pozostaje odczytowy i nie obejmuje pełnego widoku Golden Record ani ręcznej obsługi decyzji integracyjnych.

7.7. Neo4j

Backend używa sterownika Neo4j 6.2.0, natomiast serwer działa na obrazie Neo4j 5.24. Komponent obsługuje demonstracyjny zapis dokumentów, ale nie jest częścią przetestowanego pipeline'u i nie przechowuje relacji Golden Record.

7.8. Konteneryzacja i wdrożenie

Docker Compose

Docker Compose jest podstawowym sposobem uruchamiania kompletnego środowiska lokalnego. Definiuje usługi, zależności startowe, healthchecki, wolumeny i zmienne środowiskowe.

OpenShift

Manifesty OpenShift obejmują ConfigMap, Secret, PVC, Deployment, Service i Route. Nie zostały przetestowane na docelowym klastrze i nie obejmują Oracle, dlatego są wariantem wymagającym dostosowania i weryfikacji.

7.9. Narzędzia developerskie

Skrypty w katalogu scripts inicjalizują bazy i wspierają przygotowanie danych syntetycznych. Testy znajdują się w tests, Swagger służy do ręcznego wywoływania API, a Airflow UI do uruchamiania i obserwowania DAG. DBEaver może być używany do przeglądania SQL Servera, ale nie jest częścią aplikacji.

7.10. Odniesienie do implementacji

Obszar	Lokalizacja
Zależności Python	requirements.txt
Zależności frontendowe	frontend/package.json, frontend/package-lock.json
Obraz backendu	Dockerfile
Obraz frontendu	frontend/Dockerfile
Obraz i inicjalizacja Airflow	airflow/Dockerfile, airflow/start-airflow.sh
Środowisko lokalne	docker-compose.yml
Konfiguracja backendu	app/core/config.py
Połączenie SQL Server	app/db/sql.py
Połączenie Neo4j	app/db/neo4j.py
Orkiestracja	airflow/dags/goldenizacja_pipeline.py
Matching	app/layers/integration_golden/service.py
Walidacja	app/layers/validation/service.py
Preprocessing	app/layers/preprocessing/service.py
Manifesty OpenShift	openshift/

8. Model danych

Model danych platformy został podzielony zgodnie z kolejnymi etapami przetwarzania. Centralnym repozytorium jest Microsoft SQL Server, w którym zastosowano cztery schematy: meta, raw, stg i gold. Podział ogranicza mieszanie danych źródłowych, technicznych informacji o procesie, rekordów roboczych oraz wyników encji Golden Record.

Pełna definicja fizyczna bazy znajduje się w `scripts/init_proposed_mssql_schema.sql`. Modele SQLAlchemy odwzorowują tabele wykorzystywane bezpośrednio przez aplikację i znajdują się w plikach `models.py` poszczególnych warstw.

8.1. Organizacja schematów

Schemat	Przeznaczenie	Przykładowe dane
meta	zarządzanie źródłami i przebiegiem procesu	systemy źródłowe, partie importu, mapowania kolumn, logi
raw	zachowanie danych w postaci otrzymanej ze źródła	treść pliku lub snapshot wyniku zapytania relacyjnego
stg	przygotowanie danych do integracji	rekordy kanoniczne, wartości znormalizowane, wyniki walidacji, kandydaci matchingu
gold	przechowywanie uzgodnionych danych wynikowych	osoby, podmioty, adresy, identyfikatory, relacje i lineage

Identyfikatory `ImportBatch_ID` i `RawFile_ID` łączą kolejne warstwy procesu. Umożliwiają wskazanie partii importu oraz materiału źródłowego, z którego pochodzi dany rekord roboczy lub wynik walidacji. W warstwie GOLD pochodzenie jest dodatkowo rejestrowane na poziomie wybranych atrybutów i relacji.

Konwencje modelu

W modelu zastosowano następujące konwencje:

- nazwa klucza głównego składa się z nazwy obiektu i przyrostka `_ID`;
- klucze techniczne są generowane przez SQL Server za pomocą `IDENTITY(1,1)`;
- klucze główne tabel procesowych i biznesowych mają przeważnie typ `BIGINT`;
- identyfikatory niewielkich słowników mają typ `INT`;
- daty zdarzeń technicznych są zapisywane jako `DATETIME2(0)` i domyślnie ustawiane przez `SYSUTCDATETIME()`;
- dane tekstowe są przechowywane w typach `NVARCHAR`, aby zachować znaki narodowe;
- zmienne struktury pochodzące ze źródeł są przechowywane w polach `NVARCHAR(MAX)` z ograniczeniem `ISJSON`;
- kolumny `Valid_From` i `Valid_To` określają okres obowiązywania relacji biznesowej, a nie czas technicznego zapisu rekordu;
- nazwy tabel faktów bezmiarowych rozpoczynają się od `Factless`, ponieważ rejestrują wystąpienie relacji, a nie wartość miary.

SQLAlchemy używa odpowiadających typów przenośnych, między innymi `BigInteger`, `Unicode`, `UnicodeText`, `Date`, `DateTime`, `Boolean` i `Numeric`. Szczegółowe ograniczenia i indeksy są definiowane w skrypcie SQL, dlatego skrypt pozostaje nadrzędnym źródłem fizycznej definicji bazy.

8.2. Schemat META

Schemat meta przechowuje słownik źródeł, partie importu, mapowania kolumn i logi procesu.

`SourceSystem.Trust_Level` jest wykorzystywany podczas survivorship, natomiast `ImportBatch_ID` przechodzi przez kolejne warstwy i łączy wyniki z konkretnym importem.

Tabela	Klucz główny	Najważniejsze kolumny	Relacje i ograniczenia
meta.SourceSystem	SourceSystem_ID INT	SourceSystem_Code	unikalny kod źródła;

Tabela	Klucz główny	Najważniejsze kolumny	Relacje i ograniczenia
		NVARCHAR(50), SourceSystem_Name NVARCHAR(255), Trust_Level TINYINT, Created_At DATETIME2(0)	Trust_Level w zakresie 0-100
meta.ImportBatch	ImportBatch_ID BIGINT	SourceSystem_ID, Import_Status, Import_Start_At, Import_End_At, Created_By, Error_Message	FK do SourceSystem; status ograniczony do zdefiniowanego zbioru
meta.ColumnMapping	ColumnMapping_ID INT	SourceSystem_ID, Entity_Type, Source_Column_Name, Canonical_Column_Name	FK do SourceSystem; unikalna kombinacja źródła, encji i kolumny wejściowej
meta.ProcessLog	ProcessLog_ID BIGINT	ImportBatch_ID, RawFile_ID, Staging_ID, Step_Name, Step_Status, liczniki i czasy	FK do ImportBatch i RawFile; kontrolowane nazwy i statusy kroków

Dozwolone statusy partii to NEW, PROCESSING, RAW_LOADED, STAGING_LOADED, COMPLETED i FAILED. ProcessLog rozróżnia kroki procesu oraz statusy STARTED, SUCCESS i FAILED, zapisując także liczniki i komunikaty błędów.

8.3. Schemat RAW

Schemat raw zawiera tabelę RawFile przechowującą plik albo snapshot JSON importu relacyjnego. Zawartość jest zapisana jako VARBINARY(MAX), a unikalny SHA-256 służy do wykrywania identycznych danych wejściowych.

Kolumna	Typ SQL	Wymagalność	Znaczenie
RawFile_ID	BIGINT IDENTITY	tak	techniczny klucz materiału RAW
ImportBatch_ID	BIGINT	tak	partia, w ramach której pobrano dane
File_Name	NVARCHAR(260)	tak	nazwa pliku albo techniczna nazwa snapshotu
File_Type	NVARCHAR(30)	tak	format zapisanej zawartości
File_Size	BIGINT	tak	rozmiar zawartości w bajtach; wartość nieujemna
File_Hash	CHAR(64)	tak	skrót SHA-256 używany do kontroli duplikatów
File_Content	VARBINARY(MAX)	nie	oryginalna zawartość pliku lub zserializowany wynik zapytania
Created_At	DATETIME2(0)	tak	techniczny czas zapisu

ImportBatch_ID wskazuje partię importu i posiada indeks. Szczegóły pobierania, kontroli duplikatów oraz wyboru VARBINARY(MAX) opisano w rozdziale 10.

8.4. Schemat STG

Schemat stg obejmuje dane kanoniczne, dane po normalizacji, wyniki kontroli jakości oraz struktury pomocnicze matchingu i grupowania.

Dane stagingowe

stg.Person_Staging i stg.Party_Staging przechowują dane po parsowaniu i mapowaniu. Obie tabele zachowują ImportBatch_ID, RawFile_ID, Source_Record_ID i kopię rekordu źródłowego w JSON.

Person_Staging

Grupa kolumn	Kolumny	Znaczenie
klucze i pochodzenie	Staging_ID, ImportBatch_ID, RawFile_ID, Source_Record_ID	identyfikacja rekordu i źródła
dokumenty i identyfikatory	PESEL, Serial_Number_ID_Card, Serial_Number_Passport	identyfikatory osoby w postaci otrzymanej po mapowaniu
dane osobowe	First_Name, Second_Name, Last_Name, Family_Name, Birth_Date, Place_Of_Birth, Sex, Citizenship	kanoniczny zestaw danych osoby
kontakt	Phone_Number, Email_Address	dane kontaktowe
adres	Street, Building_Number, Apartment_Number, City, Postal_City, Postal_Code, District, Province, Country	adres zapisany jeszcze przy rekordzie stagingowym
audyt mapowania	Raw_Record_JSON, Created_At	kopia rekordu wejściowego i czas zapisu

Raw_Record_JSON podlega kontroli poprawności JSON. Kolumny biznesowe są opcjonalne, ponieważ rekordy niekompletne są oceniane dopiero podczas walidacji.

Party_Staging

Grupa kolumn	Przykładowe kolumny	Znaczenie
klucze i pochodzenie	Staging_ID, ImportBatch_ID, RawFile_ID, Source_Record_ID	identyfikacja rekordu i źródła
dane podmiotu	Name, Short_Name, Legal_Entity_Type, Registration_Country, Establishment_Date	podstawowe dane opisowe
identyfikatory	Identifiers_JSON, Register_Number, Decision_Number	identyfikatory pochodzące z różnych rejestrów
adres	poła od Street do Country	adres podmiotu
informacje rejestrowe	Register_Status, Registration_Date, Deregistration_Date, Decision_Date	status i daty dotyczące wpisu
informacje działalności	Bank_Accounts_JSON, Has_Virtual_Accounts, Business_Scope, Ownership_Form, Municipality	dane właściwe wybranym źródłom publicznym
kontakt i kontekst branżowy	Phone_Number, Email_Address, Website, Agent_Type, Insurance_Company	dane kontaktowe i dodatkowe
relacje	Related_Persons_JSON, Related_Parties_JSON, poła rodzica bezpośredniego i ostatecznego	osoby i podmioty powiązane
dane GLEIF	Registration_Status, Last_Update_Date, Next_Renewal_Date, Managing_LOU, poła validation authority	informacje o rejestracji LEI
audyt mapowania	Raw_Record_JSON, Created_At	kopia rekordu wejściowego i czas zapisu

Pola Identifiers_JSON, Bank_Accounts_JSON, Related_Persons_JSON, Related_Parties_JSON i Raw_Record_JSON posiadają ograniczenia sprawdzające poprawność JSON. Rozbudowana struktura Party_Staging wynika z różnorodności źródeł podmiotowych. Nie wszystkie kolumny muszą być wypełnione dla każdego źródła.

Dane po preprocessingu

Tabele preprocessingowe przechowują wartości znormalizowane bez nadpisywania stagingu. Preprocessing_Rules_JSON opisuje zastosowane przekształcenia.

Tabela	Klucz główny	Powiązanie ze stagingiem	Główne grupy danych
stg.Person_Preprocessed	Preprocessed_ID	Staging_ID FK do Person_Staging, relacja unikalna 1:1	identyfikatory, pełna nazwa osoby, kontakt i adres po normalizacji
stg.Party_Preprocessed	Preprocessed_ID	Staging_ID FK do Party_Staging, relacja unikalna 1:1	nazwa, identyfikatory, rejestry, kontakt, adres i relacje po normalizacji

Bezpośrednie klucze do partii i RAW upraszczają filtrowanie przebiegu. Wybrane pola znormalizowane są objęte indeksami odpowiadającymi ścieżkom wyszukiwania kandydatów, na przykład kombinacja PESEL_Normalized i Full_Name_Normalized.

Walidacja

stg.Validation_Result zapisuje osobny wynik każdej reguły i pola, dzięki czemu jeden rekord może mieć wiele ocen jakości.

Kolumna lub grupa	Znaczenie
Validation_ID	klucz pojedynczego wyniku reguły
ImportBatch_ID, RawFile_ID	identyfikacja przebiegu i materiału wejściowego
Entity_Type	typ encji: PERSON albo PARTY
Staging_ID, Preprocessed_ID	wskazanie kontrolowanego rekordu
Validation_Level	etap kontroli: STAGING albo PREPROCESSING
Rule_Code, Field_Name	kod reguły i pole, którego dotyczy
Severity	poziom INFO, WARNING albo ERROR
Status	wynik PASS, WARNING albo ERROR
Message, Checked_Value	opis wyniku i sprawdzona wartość

Tabela posiada klucze obce do partii importu i materiału RAW. Staging_ID oraz Preprocessed_ID są identyfikatorami polimorficznymi zależnymi od Entity_Type, dlatego nie wskazują jedną stałą tabelę kluczem obcym.

Matching i grupowanie

Kandydaci Levenshteina i Jaro-Winklera zachowują porównywaną parę, wynik, decyzję i uzasadniające ją pola. Zaakceptowane powiązania tworzą grupy encji, a grupy niespełniające warunków materializacji mogą zostać zapisane w rejestrze odrzuceń.

Tabela	Klucz i najważniejsze relacje	Integralność
Match_Candidate_Levenshtein	para Left_Preprocessed_ID i Right_Preprocessed_ID; opcjonalne powiązanie z RawFile_ID	wynik 0-1; typ PERSON/PARTY; lewy identyfikator mniejszy od prawego; unikalna para w pliku
Match_Candidate_JaroWinkler	FK Levenshtein_Candidate_ID do pierwszego etapu; ta sama para rekordów	oba wyniki 0-1; kontrolowane decyzje; poprawne pola JSON
Entity_Group	Entity_Group_ID; unikalne Entity_Type i Group_Key	typ PERSON albo PARTY
Entity_Group_Member	FK złożony do grupy; FK do odpowiedniej tabeli preprocessingowej	jeden rekord preprocessingowy może należeć tylko do jednej grupy danego typu
Golden_Record_Reject	opcjonalny FK złożony do grupy oraz FK do RawFile	status OPEN, RESOLVED albo IGNORED; poprawność trzech pól JSON

Pola JSON zachowują podstawę decyzji AUTO_MERGE, REVIEW albo CANDIDATE. Group_Key jest stabilnym skrótem członków, a złożone klucze chronią przed połączeniem różnych typów encji.

8.5. Schemat GOLD

Model GOLD zatwierdzony w docs/dokumentacja/diagramy/Goldenizacja - model 22.04.pdf został odwzorowany w skrypcie SQL razem z kluczami, ograniczeniami, indeksami i słownikami.

Zestawienie struktur GOLD

Grupa	Tabele	Rola
główne wymiary	DimPerson, DimParty, DimAddress	uzgodnione dane osób, podmiotów i adresów
słowniki	DimAddressType, DimIdentityType, DimPartyRelationshipType, DimRegister, DimRegisterStatus, DimRoleType	kontrolowane typy relacji i identyfikatorów
fakty bezmiarowe	FactlessPartyIdentities, FactlessPersonAddress, FactlessPartyAddress, FactlessPartyRegisterEntry, FactlessPartyRelationship, FactlessPersonPartyRole	relacje pomiędzy wymiarami
audyt zmian	EntityChangeLog	historia zmian wartości encji GOLD
lineage atrybutów	GoldenPersonLineage, GoldenPartyLineage, GoldenAddressLineage, GoldenPartyIdentityLineage	pochodzenie wybranej wartości
lineage relacji	PersonAddressLineage, PartyAddressLineage, PersonPartyRoleLineage, PartyRegisterEntryLineage, PartyRelationshipLineage	pochodzenie utworzonego powiązania

Główne wymiary

Główne wymiary przechowują uzgodnione profile osób i podmiotów oraz współdzielone adresy. Adres jest łączony z encją tabelą relacyjną, a nie osadzony bezpośrednio w profilu.

DimPerson

Grupa	Kolumny	Uwagi
klucz	Person_ID BIGINT	klucz techniczny
identyfikatory	PESEL, Serial_Number_ID_Card, Serial_Number_Passport	wartości opcjonalne; dla każdej niepustej wartości utworzono unikalny indeks
dane osobowe	First_Name, Second_Name, Last_Name, Family_Name	wartości wybrane zgodnie z regułami survivorship
urodzenie i cechy	Birth_Date, Place_Of_Birth, Sex, Citizenship	dane profilu osoby
kontakt	Phone_Number, Email_Address	indeksy nieunikalne, ponieważ dane kontaktowe mogą być współdzielone
audyt techniczny	Created_At, Updated_At	czas utworzenia i ostatniej aktualizacji

Aktualizacja DimPerson nie tworzy nowej wersji wiersza; zmiany są zapisywane w EntityChangeLog.

DimParty

Kolumna	Typ SQL	Znaczenie
Party_ID	BIGINT IDENTITY	klucz podmiotu
Name	NVARCHAR(255) NOT NULL	uzgodniona pełna nazwa
Short_Name	NVARCHAR(255)	nazwa skrócona
Legal_Entity_Type	NVARCHAR(100)	forma prawna
Registration_Country	NVARCHAR(50)	kraj rejestracji
Establishment_Date	DATE	data utworzenia podmiotu
Created_At, Updated_At	DATETIME2(0)	czasy techniczne

Identyfikatory podmiotu nie są kolumnami DimParty. Są przechowywane w FactlessPartyIdentities, co pozwala przypisać wiele identyfikatorów różnych typów do jednego wymiaru.

DimAddress

Grupa	Kolumny
klucz	Address_ID
ulica i lokal	Street, Building_Number, Apartment_Number
miejsowość	City, Postal_City, Postal_Code
podział administracyjny	District, Province, Country
audyt techniczny	Created_At

Repozytorium wyszukuje istniejący adres po zestawie pól adresowych. Jeżeli nie znajdzie zgodnego wiersza, tworzy nowy wymiar, a następnie zapisuje powiązanie z osobą lub podmiotem.

Wymiary słownikowe

Tabela	Klucz	Wartość opisowa	Przykładowe wartości inicjalne
DimAddressType	AddressType_ID	AddressType_Name	REGISTERED, CORRESPONDENCE, RESIDENCE, BUSINESS
DimIdentityType	IdentityType_ID	IdentityType_Name	PESEL, NIP, REGON, KRS, LEI, ID_CARD, PASSPORT
DimPartyRelationshipType	RelationshipType_ID	Relationship_Name	DIRECT_PARENT, ULTIMATE_PARENT, SHAREHOLDER, RELATED_PARTY
DimRegister	Register_ID	Register_Name	wartości zależne od rejestrów zasilających
DimRegisterStatus	RegisterStatus_ID	Status_Name	ACTIVE, INACTIVE, SUSPENDED, DEREGISTERED, UNKNOWN
DimRoleType	RoleType_ID	Role_Name	OWNER, BOARD_MEMBER, PROXY, AGENT, EMPLOYEE

Nazwy słownikowe są unikalne, a wartości początkowe są ładowane przez skrypt inicjalizacyjny.

Identyfikatory i adresy

FactlessPartyIdentities przechowuje wiele identyfikatorów podmiotu, m.in. NIP, REGON, KRS i LEI.

FactlessPersonAddress i FactlessPartyAddress przypisują współdzielony adres do encji wraz z typem i okresem obowiązywania.

FactlessPartyIdentities

Kolumna	Znaczenie
PartyIdentity_ID	klucz techniczny przypisania
Party_ID	FK do DimParty
IdentityType_ID	FK do DimIdentityType
Identity_Value	wartość identyfikatora
Is_Valid	informacja o wyniku kontroli identyfikatora
Match_Confidence	ocena dopasowania w zakresie 0-1
Valid_From, Valid_To	opcjonalny okres obowiązywania

Kombinacja IdentityType_ID i Identity_Value jest unikalna. Oznacza to, że ten sam identyfikator określonego typu nie może zostać przypisany do dwóch podmiotów. Ograniczenie dat wymaga, aby Valid_To nie poprzedzało

Valid_From.

Relacje adresowe

FactlessPersonAddress i FactlessPartyAddress mają analogiczną budowę:

Element	Osoba	Podmiot
klucz relacji	PersonAddress_ID	PartyAddress_ID
właściciel relacji	Person_ID FK do DimPerson	Party_ID FK do DimParty
adres	Address_ID FK do DimAddress	Address_ID FK do DimAddress
typ	AddressType_ID FK do DimAddressType	AddressType_ID FK do DimAddressType
okres	Valid_From, Valid_To	Valid_From, Valid_To

Repozytorium zapobiega ponownemu tworzeniu identycznego aktywnego powiązania. Jeżeli dla osoby lub podmiotu zostanie wybrany inny adres tego samego typu, dotychczas aktywne powiązanie otrzymuje Valid_To, a nowe Valid_From. Datą graniczną jest dzień rozpoczęcia importu rekordu, z którego wybrano adres, a przy braku tej informacji dzień wykonania goldenizacji. Dzięki temu model zachowuje kolejność obowiązywania adresów bez tworzenia kolejnych wersji głównego wymiaru osoby lub podmiotu. Indeksy po właścicielu i adresie wspierają przechodzenie relacji w obu kierunkach.

Pozostałe relacje modelu GOLD

Model fizyczny zawiera również wpisy rejestrowe, relacje podmiotów i role osób. Aktualna materializacja zasila profile osób i podmiotów, adresy oraz identyfikatory; wymienione niżej relacje nie są jeszcze zapisywane przez serwis goldenizacji.

Tabela	Klucze obce	Dane dodatkowe	Kontrola integralności
FactlessPartyRegisterEntry	Party_ID, Register_ID, RegisterStatus_ID	Registration_Date, Deregistration_Date	data wykreślenia nie może poprzedzać rejestracji
FactlessPartyRelationship	Parent_Party_ID, Child_Party_ID, RelationshipType_ID	Valid_From, Valid_To	brak relacji podmiotu z samym sobą; poprawny zakres dat
FactlessPersonPartyRole	Person_ID, Party_ID, RoleType_ID	Valid_From, Valid_To	poprawny zakres dat

Odpowiadające im tabele lineage są obecne w modelu fizycznym, ale pozostają poza aktualną ścieżką automatycznego zapisu.

8.6. Audyt zmian i lineage

EntityChangeLog rejestruje zmianę wartości, natomiast tabele Golden*Lineage wskazują pochodzenie aktualnie wybranego atrybutu. Osobne lineage istnieje dla adresów, identyfikatorów i relacji przewidzianych w modelu.

Kolumna	Typ	Znaczenie
Change_ID	BIGINT	klucz wpisu historii
Entity_Type	NVARCHAR(20)	PERSON, PARTY, ADDRESS albo PARTY_IDENTITY
DimPerson_ID, DimParty_ID, DimAddress_ID, PartyIdentity_ID	BIGINT	dokładnie jedno wskazanie zmienionej encji
Attribute_Name	NVARCHAR(100)	nazwa zmienionego pola
Old_Value, New_Value	NVARCHAR(4000)	wartości przed i po zmianie
Change_Date	DATETIME2(0)	czas zarejestrowania zmiany
ImportBatch_ID	BIGINT	partia, która spowodowała zmianę

Ograniczenie CK_EntityChangeLog_Entity_Ref wymaga wskazania dokładnie jednego identyfikatora zgodnego z Entity_Type.

Tabele lineage atrybutów mają wspólny zestaw pól:

Pole	Znaczenie
identyfikator lineage	klucz techniczny wpisu
identyfikator encji	osoba, podmiot, adres lub identyfikator podmiotu
Attribute_Name	atrybut, którego pochodzenie opisuje wpis
SourceSystem_ID	system, z którego wybrano wartość
Source_Record_ID	identyfikator rekordu w źródle
ImportBatch_ID	przebieg, podczas którego wybrano wartość
Selection_Rule	reguła zastosowana podczas survivorship
Trust_Score	znormalizowany poziom zaufania do źródła
Quality_Score	ocena jakości wynikająca z walidacji
Validation_Status	status walidacji wybranej wartości
Recorded_At	czas zapisania informacji o pochodzeniu

Trust_Score i Quality_Score muszą mieścić się w zakresie 0-1. Indeksy wspierają odczyt po encji, atrybucie, źródle i partii.

W aktualnej implementacji zapis lineage jest wykonywany metodami `upsert_dimension_lineage()` i `upsert_address_link_lineage()` z `app/layers/integration_golden/repository.py`. Dla danego atrybutu wymiaru utrzymywane jest aktualne wskazanie wybranego źródła, a zmiany wartości biznesowej są niezależnie rejestrowane w `EntityChangeLog`.

8.7. Relacje między schematami

Ścieżka pochodzenia prowadzi od `SourceSystem` przez `ImportBatch`, `RawFile`, staging i preprocessing do grup encji oraz wymiarów GOLD. Lineage zachowuje źródło, rekord i partię wybranej wartości, a `EntityChangeLog` wiąże zmianę wymiaru z importem.

Model nie tworzy bezpośredniego klucza obcego z wymiaru GOLD do pojedynczego rekordu stagingowego. Jest to celowe, ponieważ jeden Golden Record może powstać z wielu rekordów i źródeł. Powiązanie wieloźródłowe jest reprezentowane przez tabele lineage.

Główne kardynalności

Relacja	Kardynalność
SourceSystem - ImportBatch	1:N
ImportBatch - RawFile	1:N
RawFile - rekordy stagingowe	1:N
rekord stagingowy - rekord preprocessingowy	1:0..1
Entity_Group - Entity_Group_Member	1:N
DimParty - FactlessPartyIdentities	1:N
DimPerson - FactlessPersonAddress	1:N
DimParty - FactlessPartyAddress	1:N
DimAddress - relacje adresowe	1:N
wymiar GOLD - wpisy lineage	1:N według atrybutów
wymiar GOLD - EntityChangeLog	1:N

8.8. Integralność i inicjalizacja

Skrypt SQL definiuje klucze, ograniczenia, indeksy, słowniki oraz konfigurację źródeł i mapowań.

Podczas uruchamiania FastAPI funkcja `init_db()` wykonuje `Base.metadata.create_all()`. Mechanizm tworzy tabele zarejestrowane w modelach ORM, ale nie zastępuje pełnego skryptu inicjalizacyjnego. Przygotowanie

kompletnej struktury, zgodnej z zatwierdzonym modelem GOLD, wymaga uruchomienia `scripts/init_proposed_mssql_schema.sql` zgodnie z instrukcją zawartą w `README.md`.

Najważniejsze ograniczenia i indeksy

Obszar	Zastosowane zabezpieczenie
systemy źródłowe	unikalny <code>SourceSystem_Code</code> , kontrola zakresu <code>Trust_Level</code>
mapowania	unikalna kolumna źródłowa w obrębie systemu i typu encji
RAW	unikalny SHA-256, nieujemny rozmiar pliku
staging	kontrola poprawności pól JSON
preprocessing	relacja 1:1 ze stagingiem, indeksy na polach matchingu
kandydaci	unikalna uporządkowana para rekordów, wyniki w zakresie 0-1
grupy	unikalny klucz grupy i jednoznaczne przypisanie członka
osoba GOLD	unikalne niepuste PESEL i numery dokumentów
identyfikatory podmiotu	unikalna para typu i wartości identyfikatora
relacje okresowe	data końcowa nie wcześniejsza niż początkowa
lineage	wyniki zaufania i jakości w zakresie 0-1
historia zmian	zgodność typu encji z dokładnie jednym identyfikatorem

Indeksy odpowiadają głównym ścieżkom dostępu: filtrowaniu przebiegu, wyszukiwaniu kandydatów i wymiarów oraz odczytowi historii i lineage.

8.9. Odniesienie do implementacji

Obszar	Lokalizacja
Zatwierdzony model GOLD	<code>docs/dokumentacja/diagramy/Goldenizacja - model 22.04.pdf</code>
Pełna definicja SQL Servera	<code>scripts/init_proposed_mssql_schema.sql</code>
Klasa bazowa ORM	<code>app/models/base.py</code>
META i RAW	<code>app/layers/ingestion/models.py</code>
Mapowania i staging	<code>app/layers/staging_validation/models.py</code>
Preprocessing	<code>app/layers/preprocessing/models.py</code>
Wyniki walidacji	<code>app/layers/validation/models.py</code>
Matching, grupy i GOLD	<code>app/layers/integration_golden/models.py</code>
Operacje na GOLD	<code>app/layers/integration_golden/repository.py</code>
Budowa Golden Record	<code>app/layers/integration_golden/service.py</code>
Inicjalizacja ORM	<code>app/db/init_db.py</code>

9. Przepływ i orkiestracja danych

Proces integracji danych jest wykonywany jako sekwencja operacji rozpoczynająca się od utworzenia partii importu i zapisu materiału RAW, a kończąca utworzeniem lub aktualizacją rekordów w schemacie GOLD. Orkiestratorem procesu jest Apache Airflow, natomiast logika poszczególnych etapów znajduje się w aplikacji FastAPI.

Airflow nie wykonuje bezpośrednio operacji na SQL Serverze. Zadania DAG-u wywołują endpointy HTTP odpowiednich warstw, a backend realizuje przetwarzanie za pomocą serwisów i repozytoriów. Dzięki temu te same operacje mogą być uruchamiane z DAG-u, interfejsu Swagger albo innego klienta API.

Główna definicja procesu znajduje się w `airflow/dags/goldenizacja_pipeline.py`. DAG ma identyfikator `goldenizacja_pipeline`, nie posiada harmonogramu cyklicznego i jest uruchamiany ręcznie z wybranymi parametrami.

9.1. Ogólny przebieg procesu

Przepływ obejmuje siedem zadań Airflow:

1. `raw_load`,
2. `staging_load`,
3. `preprocessing_load`,
4. `teryt_load`,
5. `validation_load`,
6. `integration_golden_match`,
7. `golden_load`.

Zadanie `integration_golden_match` obejmuje trzy kolejne operacje backendu: wyszukanie kandydatów metodą Levenshteina, ich ponowną ocenę metodą Jaro-Winklera oraz utworzenie grup encji. DAG zawiera więc siedem zadań Airflow, mimo że etap integracji wykonuje trzy wywołania API.

Zależności w DAG-u są liniowe. Następne zadanie rozpoczyna się dopiero po poprawnym zakończeniu poprzedniego:

```
raw_load
-> staging_load
-> preprocessing_load
-> teryt_load
-> validation_load
-> integration_golden_match
-> golden_load
```

Takie wykonanie odpowiada zależnościom danych: staging wymaga materiału RAW, preprocessing wymaga stagingu, walidacja korzysta z danych po preprocessingu, a goldenizacja wymaga wyników matchingu i grupowania.

9.2. Parametry uruchomienia DAG-u

DAG przyjmuje parametry opisujące źródło danych, typ encji oraz konfigurację walidacji i matchingu.

Parametr	Wartość domyślna	Znaczenie
<code>file_path</code>	<code>/opt/airflow/data/csv/pesel.csv</code>	ścieżka pliku dostępna w kontenerze Airflow
<code>input_type</code>	<code>FILE</code>	sposób pobrania danych: <code>FILE</code> albo <code>RELATIONAL</code>
<code>source_system_code</code>	<code>AUTO</code>	kod źródła lub automatyczne rozpoznanie
<code>query_name</code>	<code>insurance_core_export</code>	nazwa dozwolonego zapytania dla źródła relacyjnego
<code>entity_type</code>	<code>AUTO</code>	<code>PERSON</code> , <code>PARTY</code> albo automatyczny wybór
<code>created_by</code>	<code>airflow</code>	oznaczenie procesu inicjującego import
<code>check_email_dns</code>	<code>true</code>	włączenie kontroli DNS domen adresów e-mail
<code>matching_min_score</code>	<code>0.50</code>	minimalny wynik pierwszego etapu matchingu

Parametr	Wartość domyślna	Znaczenie
matching_max_pairs	2000000	limit liczby porównywanych par
jaro_winkler_min_score	0.78	minimalny wynik drugiego etapu matchingu

Wartości z konfiguracji konkretnego uruchomienia dag_run.conf mają pierwszeństwo przed wartościami zdefiniowanymi w params. Łączenie konfiguracji realizuje funkcja _conf().

Automatyczne rozpoznawanie źródła

Dla importu plikowego funkcja _source_system_code() ustala kod źródła z nazwy pliku zgodnie z mapą SOURCE_SYSTEM_BY_FILE_STEM. Obejmuje ona przygotowane zbiory CEIDG, GLEIF, KRS, PESEL, REGON, VAT oraz rejestry KNF używane w danych syntetycznych. Dla nieznanej nazwy wymagane jest jawne podanie source_system_code. Dla input_type=RELATIONAL domyślnym źródłem jest INSURANCE_CORE.

Automatyczne rozpoznawanie typu encji

Typ encji może zostać podany bezpośrednio jako PERSON lub PARTY. Przy wartości AUTO jest wybierany na podstawie źródła.

Źródła GLEIF, REGON, VAT, KNF_PIENIADZ_ELEKTRONICZNY oraz INSURANCE_CORE mają określony domyślny typ. Dla CEIDG, KRS i części rejestrów KNF proces wykonuje obie ścieżki: PARTY i PERSON.

Import relacyjny również domyślnie wykonuje oba typy encji. W takim przypadku backend pobiera dane osobno dla osoby i podmiotu, tworząc dwa materiały RAW i dwie partie importu.

9.3. Komunikacja Airflow z FastAPI

Wszystkie operacje są wywoływane metodą HTTP POST. Adres bazowy API w środowisku kontenerowym to: http://api:8000/layers

Funkcja _post_form() wysyła formularz, opcjonalnie wraz z plikami. Limit czasu pojedynczego żądania wynosi 300 sekund. Odpowiedź HTTP spoza zakresu powodzenia powoduje zgłoszenie RuntimeError zawierającego:

- metodę i endpoint,
- kod odpowiedzi,
- treść odpowiedzi backendu.

Poprawna odpowiedź jest deserializowana z JSON i zwracana przez zadanie. PythonOperator automatycznie zapisuje zwracaną wartość w XCom.

Zestawienie wywołań API

Operacja	Endpoint
import pliku	POST /layers/ingestion/raw-load
import relacyjny	POST /layers/ingestion/relational-load
staging	POST /layers/staging_validation/staging-load
preprocessing	POST /layers/preprocessing/preprocessing-load
załadowanie TERYT	POST /layers/validation/teryt-load
walidacja	POST /layers/validation/validation-load
Levenshtein	POST /layers/integration_golden/match-candidates
Jaro-Winkler	POST /layers/integration_golden/match-candidates/jaro-winkler
grupowanie	POST /layers/integration_golden/match-groups
materializacja GOLD	POST /layers/integration_golden/golden-load

Endpointsy przekazują logikę do funkcji serwisowych. Serwisy korzystają z repozytoriów SQLAlchemy, które wykonują odczyt i zapis w SQL Serverze.

9.4. Przekazywanie identyfikatorów

ImportBatch_ID

ImportBatch_ID jest tworzony przez backend podczas operacji RAW load. Reprezentuje pojedynczy import z określonego systemu źródłowego.

Identyfikator partii nie jest bezpośrednio przekazywany pomiędzy zadaniami Airflow. Kolejne warstwy otrzymują RawFile_ID i odczytują z raw.RawFile powiązany ImportBatch_ID. Ogranicza to liczbę parametrów przekazywanych przez DAG i zapewnia, że dalsze kroki używają partii przypisanej w bazie do konkretnego materiału RAW.

RawFile_ID

RawFile_ID jest podstawowym identyfikatorem przekazywanym pomiędzy zadaniami. raw_load zwraca go z odpowiedzi backendu, a kolejne zadania pobierają wartość przez:

```
context["ti"].xcom_pull(task_ids="raw_load")
```

Dla procesu obejmującego jeden materiał RAW wartością XCom jest liczba całkowita. Dla importu relacyjnego obejmującego obie encje wynik jest słownikiem:

```
{
  "PERSON": <RawFile_ID>,
  "PARTY": <RawFile_ID>
}
```

Każdy kolejny krok wybiera identyfikator właściwy dla aktualnie przetwarzanego typu encji.

Przy pliku zawierającym oba typy encji ten sam RawFile_ID jest używany dwukrotnie: raz do zbudowania stagingu PERSON, a drugi raz do stagingu PARTY. Rozdzielenie następuje przez parametr entity_type i osobne tabele docelowe.

9.5. Zakres zadań DAG-u

Zadanie	Odpowiedzialność orkiestracyjna	Wynik
raw_load	wybór trybu FILE lub RELATIONAL i wywołanie rejestracji RAW	pojedynczy RawFile_ID albo mapa identyfikatorów według typu encji
staging_load	uruchomienie ładowania STAGING dla ustalonych typów encji	wyniki wywołań API
preprocessing_load	uruchomienie standaryzacji i deduplikacji technicznej	wyniki wywołań API
teryt_load	przekazanie do API plików SIMC i ULIC	wynik ładowania danych referencyjnych
validation_load	uruchomienie walidacji, w tym opcjonalnej kontroli DNS poczty elektronicznej	wyniki walidacji
integration_golden_match	sekwencyjne uruchomienie Levenshteina, Jaro-Winklera i grupowania	zbiórny wynik trzech operacji
golden_load	uruchomienie materializacji złotych rekordów dla bieżącej partii	utworzone lub zaktualizowane rekordy GOLD

Dla zakresu obejmującego obie encje zadania warstwowe wykonują osobne wywołania dla PARTY i PERSON. Plik zawierający oba typy danych wykorzystuje ten sam RawFile_ID, natomiast wejście relacyjne zwraca osobne identyfikatory partii. Grupowanie działa dla całego typu encji i nie otrzymuje RawFile_ID.

Pliki /opt/airflow/data/teryt/SIMC.csv i /opt/airflow/data/teryt/ULIC.csv są przysyłane przy każdym uruchomieniu DAG-u. Brak jednego z nich powoduje błąd przed wywołaniem API. Szczegółowe reguły przetwarzania poszczególnych warstw opisano w kolejnych rozdziałach.

9.6. Statusy i logowanie

Stan procesu jest widoczny na dwóch poziomach:

- Airflow rejestruje stan zadań DAG-u i ich logi wykonania,

- SQL Server przechowuje stan partii oraz wpisy meta.ProcessLog.

Status partii

meta.ImportBatch.Import_Status obsługuje wartości:

- NEW,
- PROCESSING,
- RAW_LOADED,
- STAGING_LOADED,
- COMPLETED,
- FAILED.

Aktualna implementacja ustawia statusy podczas RAW load i staging load. Po poprawnym stagingu partia ma status STAGING_LOADED. Dalsze etapy nie zmieniają obecnie partii na COMPLETED; ich wykonanie jest widoczne w logach właściwych etapów oraz w stanie zadań Airflow.

Status partii FAILED jest ustawiany przy błędzie RAW load albo staging load. Błędy preprocessingu, walidacji i ładowania GOLD są zapisywane również w ProcessLog, natomiast błędy matchingu i grupowania pozostają widoczne w stanie i logach zadania Airflow.

ProcessLog

Wpis ProcessLog zawiera:

- ImportBatch_ID,
- opcjonalny RawFile_ID,
- nazwę kroku,
- status STARTED, SUCCESS albo FAILED,
- czas rozpoczęcia i zakończenia,
- liczniki wejścia i wyjścia,
- komunikat błędu.

Logi bazodanowe są tworzone dla:

- RAW_LOAD,
- STAGING_LOAD,
- STANDARDIZATION,
- VALIDATION,
- GOLDEN_LOAD.

Matching Levenshteina, Jaro-Winklera i grupowanie nie tworzą oddzielnych wpisów ProcessLog. Ich przebieg jest widoczny w logach zadania integration_golden_match oraz w tabelach wynikowych matchingu.

9.7. Transakcje i powtarzalność

Repozytoria wykonują zatwierdzenia po utworzeniu logów oraz po zapisaniu wyników etapów. W razie błędu serwis wycofuje bieżącą transakcję i, jeżeli log został już utworzony, kończy go statusem FAILED.

Etap GOLD nie działa jako jedna wspólna transakcja dla całego przebiegu golden_load. Implementacja zapisuje wynik osobno dla kolejnych grup encji. Oznacza to, że:

- grupy przetworzone przed błędem pozostają zapisane,
- błąd zatrzymuje dalsze grupy i oznacza krok jako FAILED,
- ponowne uruchomienie opiera się na logice create or update, więc nie wymaga ręcznego czyszczenia już zapisanych wymiarów.

Powtarzalność operacji zależy od etapu:

Etap	Zachowanie przy ponownym uruchomieniu
RAW load	unikalny hash ogranicza ponowny zapis identycznej zawartości
staging	ponowny zapis tego samego RAW i typu encji jest blokowany
preprocessing	ponowne przetworzenie istniejących rekordów jest blokowane
TERYT	pliki pomocnicze są nadpisywane
validation	wcześniejsze wyniki dla RAW i typu encji są zastępowane
matching	kandydaci są zastępowani dla tej samej kombinacji typu encji i RawFile_ID; ponowienie Levenshteina usuwa również zależne wyniki Jaro-Winklera z tego zakresu
grouping	członkostwa grup danego typu są budowane ponownie na podstawie wszystkich aktualnych kandydatów AUTO_MERGE tego typu
GOLD	repozytorium wyszukuje istniejące wymiary i relacje, a następnie wykonuje create albo update

Mechanizmy te ograniczają powstawanie duplikatów, ale pełne ponowienie całego DAG-u od początku tworzy nową próbę RAW load. W przypadku potrzeby powtórzenia tylko późniejszego etapu można wywołać odpowiedni endpoint bezpośrednio z istniejącym RawFile_ID, z uwzględnieniem zasad danego etapu.

9.8. Obsługa błędów

Backend rozróżnia błędy wejściowe i błędy nieoczekiwane:

- znane problemy danych lub konfiguracji zwracają HTTP 400,
- nieobsłużone wyjątki zwracają HTTP 500 z nazwą etapu.

Przykładowe błędy HTTP 400 obejmują:

- nieobsługiwany format lub system źródłowy,
- brak mapowania kolumn,
- brak materiału RAW,
- ponowne ładowanie stagingu,
- brak rekordów preprocessingowych,
- błędny typ encji,
- przekroczenie limitu par matchingu,
- brak kandydatów wymaganych przez drugi etap.

Airflow traktuje każdą odpowiedź spoza zakresu powodzenia jako wyjątek zadania. Ponieważ zależności są liniowe, niepowodzenie zatrzymuje uruchomienie przed kolejnymi etapami.

Operatory nie mają jawnie skonfigurowanej liczby ponowień ani opóźnienia pomiędzy próbami. Proces korzysta więc z ustawień domyślnych środowiska Airflow. W dostarczonej konfiguracji DAG-u nie zdefiniowano automatycznej polityki retry właściwej dla poszczególnych zadań.

9.9. Ograniczenia orkiestracji

Aktualna orkiestracja jest przeznaczona do kontrolowanego uruchamiania procesu demonstracyjnego. Należy uwzględnić następujące właściwości:

- DAG jest uruchamiany ręcznie i nie ma harmonogramu;
- wszystkie zadania są wykonywane sekwencyjnie;
- lokalny Airflow używa SequentialExecutor;
- limit czasu pojedynczego wywołania API wynosi 300 sekund;
- nie skonfigurowano indywidualnych retry zadań;
- TERYT jest przesyłany przy każdym przebiegu;

- trzy operacje matchingu są ukryte w jednym zadaniu Airflow;
- status ImportBatch nie jest kończony wartością COMPLETED przez ostatnie etapy;
- grupowanie jest wykonywane dla typu encji bez parametru RawFile_ID.

Nie uniemożliwia to wykonania pełnego procesu. Właściwości te wpływają przede wszystkim na obserwowalność, selektywne ponawianie kroków i skalowanie wielu równoległych importów.

9.10. Odniesienie do implementacji

Obszar	Lokalizacja
definicja DAG-u i parametry	airflow/dags/goldenizacja_pipeline.py
router warstw	app/layers/router.py
RAW load	app/layers/ingestion/api.py, service.py, repository.py
staging load	app/layers/staging_validation/api.py, service.py, repository.py
preprocessing	app/layers/preprocessing/api.py, service.py, repository.py
TERYT i walidacja	app/layers/validation/api.py, service.py, repository.py
matching, grupowanie i GOLD	app/layers/integration_golden/api.py, service.py, repository.py
modele statusów i logów	app/layers/ingestion/models.py
konfiguracja Airflow	docker-compose.yml, airflow/Dockerfile, airflow/start-airflow.sh

10. Pobieranie i składowanie danych RAW

Warstwa RAW stanowi pierwszy trwały etap przetwarzania danych. Jej zadaniem jest przyjęcie danych ze źródła, wykonanie podstawowej kontroli technicznej oraz zachowanie zawartości, na której będą pracowały kolejne warstwy. Dane nie są na tym etapie mapowane do modelu osoby ani podmiotu, normalizowane lub oceniane pod względem jakości biznesowej.

Platforma obsługuje dwa sposoby pozyskania danych:

- przesłanie pliku do API,
- wykonanie przygotowanego zapytania do demonstracyjnego źródła relacyjnego Oracle.

Niezależnie od sposobu pozyskania wynikiem jest rekord `raw.RawFile` powiązany z systemem źródłowym, partią importu i logiem `RAW_LOAD`. Warstwa sprawdza wejście, oblicza SHA-256 i zachowuje zawartość. Mapowanie do modeli `PERSON` i `PARTY`, normalizacja oraz walidacja odbywają się później.

10.1. Interfejs API

Metoda i endpoint	Przeznaczenie
GET /layers/ingestion/relational-queries	pobranie listy udostępnionych importów relacyjnych
POST /layers/ingestion/raw-load	przesłanie pliku do warstwy RAW
POST /layers/ingestion/relational-load	wykonanie przygotowanego importu z Oracle

Oba endpointy importowe zwracają `RawLoadResponse`.

Pole	Znaczenie
<code>import_batch_id</code> , <code>raw_file_id</code>	identyfikatory partii i materiału RAW
<code>file_name</code> , <code>file_type</code> , <code>file_size</code>	podstawowe metadane zapisanej zawartości
<code>file_hash</code>	skrót SHA-256
<code>records_in</code>	wstępny licznik rekordów
<code>import_status</code>	status partii, po sukcesie <code>RAW_LOADED</code>

`raw_file_id` jest podstawowym identyfikatorem przekazywanym do stageru i dalszych etapów.

10.2. Import plików

Endpoint przyjmuje plik, `source_system_code` oraz opcjonalne `created_by`. `UploadFile` jest odczytywany do postaci bajtowej i przekazywany do `import_raw_file()`.

Obsługiwane formaty

Dozwolone rozszerzenia są określone w stałej `SUPPORTED_FILE_TYPES`.

Format	Kontrola wykonywana podczas RAW load	Sposób ustalenia <code>records_in</code>
CSV	odczyt jako UTF-8 z opcjonalnym BOM	liczba wierszy pomniejszona o wiersz nagłówek
JSON	dekodowanie UTF-8 i sprawdzenie, czy korzeniem jest obiekt albo lista	1 dla obiektu albo długość listy
XML	parsowanie dokumentu XML	liczba elementów bezpośrednio pod elementem głównym
XLSX	otwarcie skoroszytu i sprawdzenie obecności niepustych wierszy	liczba niepustych wierszy pomniejszona o nagłówki

Rozszerzenie jest sprawdzane przed utworzeniem partii, a `File_Type` zapisywany wielkimi literami. Kontrola ma charakter techniczny: nie sprawdza wymaganych kolumn ani zgodności z modelem kanonicznym. Plik zapisany w RAW może więc zostać później odrzucony w stageru.

`records_in` jest licznikiem technicznym. Dla CSV i XLSX zakłada jeden wiersz nagłówek, dla XML liczy elementy

pierwszego poziomu, a dla JSON elementy listy lub pojedynczy obiekt. Nie potwierdza, że rekordy zostaną poprawnie zmapowane.

Sprawdzanie systemu źródłowego

source_system_code jest normalizowany do wielkich liter i musi występować w SUPPORTED_SOURCE_SYSTEMS. Pełny katalog źródeł opisano w rozdziale 5. Brakujący wpis meta.SourceSystem jest tworzony automatycznie, z pustym poziomem zaufania.

10.3. Import ze źródła relacyjnego

Endpoint przyjmuje source_system_code, query_name, opcjonalny entity_type i created_by. Nie pozwala przekazać dowolnego SQL: zapytanie musi występować w RELATIONAL_QUERY_DEFINITIONS i być zgodne z systemem źródłowym oraz typem encji.

Definicje zapytań

Endpoint GET /relational-queries zwraca publiczną nazwę insurance_core_export z typem AUTO. Podczas uruchomienia jest ona rozwijana według entity_type do jednej z dwóch definicji wewnętrznych:

Nazwa wewnętrzna	Typ encji	Nazwa snapshotu
insurance_core_party_export	PARTY	insurance_core_party_export.json
insurance_core_person_export	PERSON	insurance_core_person_export.json

Ogólna nazwa wymaga podania PERSON albo PARTY. W DAG-u Airflow przy automatycznym wyborze obu typów wykonywane są dwa osobne wywołania, dlatego powstają dwie partie i dwa materiały RAW.

Definicja jest odrzucana, jeżeli nazwa jest nieznana, brakuje wymaganego typu encji albo zapytanie nie odpowiada wskazanemu typowi lub źródłu.

Wybór z zamkniętej listy zapobiega wykonaniu przez endpoint dowolnego SQL przekazanego przez klienta. Nie zastępuje jednak ograniczeń uprawnień konta Oracle, które powinno mieć wyłącznie dostęp potrzebny do odczytu danych objętych eksportem.

Połączenie z Oracle

Połączenie korzysta z ORACLE_ODBC_CONNECTION_STRING i PyODBC albo, gdy ciągu nie podano, ze sterownika oracledb w trybie thin. Nazwy kolumn pochodzą z cursor.description, a wiersze są zamieniane na słowniki.

Eksport PARTY wykonuje zapytanie główne oraz zapytania pobierające role osób i relacje między podmiotami. Dane dodatkowe są dołączane do odpowiedniego rekordu klienta w polach RELATED_PERSONS_JSON i RELATED_PARTIES_JSON.

Eksport PERSON zwraca osoby z przypisań agentów oraz osoby zapisane jako klienci systemu demonstracyjnego. Wyniki są odczytywane do pamięci, a połączenie jest zamykane również po wystąpieniu błędu.

Utworzenie snapshotu

Wynik importu relacyjnego jest serializowany do dokumentu JSON:

```
lista rekordów Oracle
-> serializacja JSON w UTF-8
-> obliczenie SHA-256
-> zapis jako raw.RawFile
```

Serializacja zachowuje znaki narodowe, a wartości nieobsługiwane bezpośrednio przez JSON, na przykład daty, konwertuje do tekstu. Snapshot jest dalej przetwarzany jak plik JSON, dzięki czemu staging pracuje na stanie danych pobranym dla konkretnej partii.

10.4. Wspólny zapis materiału RAW

Import plikowy i relacyjny kończą się wywołaniem persist_raw_content(), które tworzy metadane, oblicza skrót i zapisuje log.

Kolejność operacji

1. Repozytorium pobiera albo tworzy meta.SourceSystem.
2. Tworzona jest partia ze statusem NEW.
3. Status partii zmienia się na PROCESSING.
4. Powstaje log RAW_LOAD ze statusem STARTED.
5. Dla pełnej zawartości obliczany jest SHA-256.
6. Powstaje rekord raw.RawFile.
7. Log otrzymuje status SUCCESS, RawFile_ID oraz liczniki.
8. Partia otrzymuje status RAW_LOADED i czas zakończenia.

Partia jest tworzona przed próbą zapisu materiału RAW. Dzięki temu błąd zapisu może zostać przypisany do konkretnego ImportBatch_ID.

10.5. Struktura RawFile

raw.RawFile przechowuje zawartość wejściową oraz metadane potrzebne do jej identyfikacji.

Kolumna	Typ fizyczny	Znaczenie
RawFile_ID	BIGINT IDENTITY	identyfikator materiału RAW
ImportBatch_ID	BIGINT	partia importu
File_Name	NVARCHAR(260)	nazwa pliku albo snapshotu
File_Type	NVARCHAR(30)	format zawartości
File_Size	BIGINT	liczba bajtów
File_Hash	CHAR(64)	SHA-256 zapisany szesnastkowo
File_Content	VARBINARY(MAX)	zawartość wejściowa
Created_At	DATETIME2(0)	czas zapisu

ImportBatch_ID jest kluczem obcym do meta.ImportBatch i posiada indeks. Rozmiar nie może być ujemny. File_Hash ma ograniczenie unikalności.

Model SQLAlchemy odwzorowuje File_Content typem LargeBinary. Chociaż definicja fizyczna dopuszcza wartość NULL, ścieżki importu wymagają niepustej zawartości i przekazują do repozytorium wartość typu bytes.

Krok jest rejestrowany w meta.ProcessLog. Po sukcesie log zawiera RawFile_ID, status SUCCESS i wstępne liczniki rekordów; po błędzie status FAILED i komunikat. Szczegółowy model metadanych i statusów opisano w rozdziałach 9 i 10.

10.6. Kontrola duplikatów

Skrót jest obliczany dla pełnej zawartości bajtowej:

SHA-256(File_Content) -> 64 znaki szesnastkowe

Ograniczenie UQ_RawFile_File_Hash zapewnia unikalność w całej tabeli, niezależnie od nazwy, źródła, typu encji i daty. Identyczne bajty przesłane pod inną nazwą również są duplikatem. Zasada obejmuje także snapshoty relacyjne.

Porównanie jest wykonywane na poziomie bajtów, a nie znaczenia danych. Dwa dokumenty JSON zawierające te same wartości, ale różniące się kolejnością pól lub formatowaniem, otrzymają inne skróty i mogą zostać zapisane osobno. Analogicznie zmiana kodowania, separatorów końca linii lub BOM zmienia SHA-256 pliku.

W przypadku snapshotu Oracle stabilność skrótu zależy również od kolejności rekordów zwróconych przez zapytanie i sposobu serializacji wartości. Mechanizm wykrywa zatem identyczny obraz wejścia, a nie biznesową równowagę dwóch zbiorów.

Konflikt ograniczenia unikalności jest przechwytywany jako IntegrityError. Serwis wycofuje bieżącą transakcję, kończy log i partię statusem FAILED, zapisuje komunikat File with this hash already exists. i zgłasza kontrolowany

błąd zawartości. Endpoint zwraca wtedy HTTP 400 bez identyfikatora istniejącego materiału RAW.

Globalny zakres unikalności oznacza, że system nie przechowuje dwóch osobnych rekordów RAW dla identycznej zawartości pochodzącej z różnych deklarowanych źródeł.

10.7. Obsługa błędów

HTTP 400 jest zwracany dla rozpoznanych problemów wejścia i konfiguracji:

- nieobsługiwanego formatu lub kodu źródła;
- pustej zawartości, nieprawidłowego XLSX albo nieobsługiwanej struktury korzenia JSON;
- błędnej nazwy lub kombinacji parametrów zapytania relacyjnego;
- duplikatu zawartości.

Pozostałe wyjątki zwracają HTTP 500 z oznaczeniem etapu. Błąd składni JSON lub XML oraz błąd dekodowania tekstu nie są obecnie zamieniane na `InvalidFileContentError`, dlatego również trafiają do tej kategorii.

Po błędzie, który wystąpił już po utworzeniu partii, serwis próbuje:

- wycofać niezakończoną transakcję;
- zakończyć rozpoczęty log statusem `FAILED`;
- ustawić partię na `FAILED`;
- zapisać treść wyjątku.

Repozytorium zatwierdza osobno utworzenie źródła, partii, logu i materiału RAW. Nie jest to jedna transakcja obejmująca cały krok. Pozwala to zachować metadane błędu, ale diagnostyka powinna uwzględniać zarówno `ImportBatch`, jak i `ProcessLog`.

10.8. Uzasadnienie użycia VARBINARY(MAX)

Pierwotna koncepcja przewidywała SQL Server FILESTREAM. W wykonanej implementacji zastosowano VARBINARY(MAX) mapowany jako LargeBinary, ponieważ w czasie realizacji możliwe było uruchomienie i zweryfikowanie tego wariantu bez dodatkowej konfiguracji magazynu FILESTREAM, ścieżek systemowych i uprawnień. Decyzja nie wynika z założenia, że środowisko docelowe nie obsługuje FILESTREAM.

Przyjęte rozwiązanie zachowuje pełną zawartość bajtową, umożliwia kontrolę SHA-256 i wiąże materiał z partią oraz logami w SQL Serverze. Nie zapewnia natomiast strumieniowego dostępu ani korzyści FILESTREAM przy obsłudze dużych obiektów.

Aplikacja odczytuje cały plik do pamięci i przekazuje wartość bytes do SQLAlchemy. Wariant został zweryfikowany dla danych projektu, ale nie wykonano pomiarów dla bardzo dużych plików ani wielu równoległych importów.

10.9. Bezpieczeństwo i integralność

Parametry połączeń są pobierane z konfiguracji środowiska. Ponieważ `File_Content` może zawierać dane osobowe i biznesowe, dostęp do schematu raw i kopii zapasowych powinien być ograniczony. Integralność wspierają klucze obce, unikalny hash, kontrola rozmiaru i logi procesu. SHA-256 wykrywa identyczną zawartość, ale nie potwierdza jej autentyczności.

10.10. Ograniczenia

W obecnej implementacji:

- cały przesyłany plik jest odczytywany do pamięci przed zapisem;
- snapshot relacyjny również jest tworzony w pamięci jako pełna lista i dokument JSON;
- nie skonfigurowano limitu rozmiaru pliku na poziomie endpointu;
- nie zastosowano skanowania antywirusowego przesyłanych plików;
- unikalność skrótu obowiązuje globalnie, a nie w ramach źródła;
- nie ma endpointu zwracającego istniejący `RawFile_ID` po wykryciu duplikatu;
- lista importów relacyjnych jest zdefiniowana w kodzie, a nie skonfigurowana administracyjnie;

- records_in jest licznikiem technicznym i nie zastępuje wyniku stagingu;
- błędy składni JSON i XML mogą zostać sklasyfikowane jako HTTP 500.

10.11. Realizacja w kodzie

Obszar	Plik lub element
endpointy importu	app/layers/ingestion/api.py
kontrola i zapis plików	_get_file_type(), _validate_and_count_records(), import_raw_file(), persist_raw_content()
import Oracle	RELATIONAL_QUERY_DEFINITIONS, resolve_relational_query_definition(), extract_relational_records()
operacje bazodanowe	app/layers/ingestion/repository.py
modele ORM	app/layers/ingestion/models.py
fizyczny schemat SQL Server	scripts/init_proposed_mssql_schema.sql

tests/test_relational_ingestion.py sprawdza pobieranie danych Oracle, dołączanie relacji, serializację snapshotu oraz wybór wariantu PERSON lub PARTY. Testy nie obejmują pełnej integracji endpointu plikowego z rzeczywistym SQL Serverem ani wydajności VARBINARY(MAX).

11. Staging i mapowanie danych

Warstwa staging przekształca materiał zapisany w RAW do wspólnej struktury osoby albo podmiotu. Na tym etapie dane są parsowane, przypisywane do kolumn kanonicznych i zapisywane w `stg.Person_Staging` lub `stg.Party_Staging`. Oryginalny materiał RAW nie jest modyfikowany.

Staging nie wykonuje pełnej standaryzacji ani oceny jakości biznesowej. Zachowuje wartości możliwie blisko postaci źródłowej, wykonując jedynie sanitację tekstu, konwersję typów wymaganych przez model SQL oraz składanie wybranych struktur do JSON. Właściwa normalizacja danych odbywa się w warstwie preprocessing.

11.1. Interfejs API

Ładowanie stagingu udostępnia endpoint:

POST `/layers/staging_validation/staging-load`

Wywołanie wymaga:

Parametr	Znaczenie
<code>raw_file_id</code>	identyfikator materiału zapisanego w <code>raw.RawFile</code>
<code>entity_type</code>	docelowy model PERSON albo PARTY

Odpowiedź `StagingLoadResponse` zawiera identyfikatory partii i RAW, typ encji, liczbę rekordów wejściowych i zapisanych, statusy procesu oraz zestawienia brakujących i nierozpoznanych kolumn.

11.2. Przebieg ładowania

Funkcja `load_raw_file_to_staging()` wykonuje następujące operacje:

1. normalizuje typ encji;
2. pobiera `RawFile` i powiązaną partię importu;
3. sprawdza, czy ten sam materiał nie został już załadowany do wskazanej tabeli stagingowej;
4. ustawia partię na `PROCESSING` i tworzy log `STAGING_LOAD`;
5. parsuje zawartość RAW;
6. pobiera mapowania kolumn dla systemu źródłowego i typu encji;
7. buduje rekordy kanoniczne;
8. uzupełnia klucze techniczne, kopię rekordu źródłowego i podstawowe typy danych;
9. zapisuje rekordy do właściwej tabeli;
10. kończy log statusem `SUCCESS` i ustawia partię na `STAGING_LOADED`.

Przy plikach zawierających dane obu typów ten sam `RawFile_ID` może zostać załadowany osobno do `Person_Staging` i `Party_Staging`. Kontrola ponownego wykonania jest prowadzona niezależnie dla każdej tabeli.

11.3. Parsowanie materiału RAW

Parser jest wybierany na podstawie `RawFile.File_Type`.

Format	Sposób odczytu
CSV	<code>csv.DictReader</code> , kodowanie UTF-8 z obsługą BOM, pierwszy wiersz jako nagłówek
JSON	obiekt jako jeden rekord albo lista obiektów
XML	elementy <code><record></code> zawierające pola <code><field name="..."></code> albo bezpośrednie elementy potomne
XLSX	aktywny arkusz otwierany przez <code>openpyxl</code> w trybie <code>read_only</code> ; pierwszy niepusty wiersz jako nagłówek

Każdy rekord jest reprezentowany jako słownik nazw kolumn i wartości. JSON może zachować struktury

zagnieżdżone, natomiast CSV i XLSX dostarczają strukturę płaską. XML bez elementów <record> oraz tablica JSON zawierająca wartości inne niż obiekty są odrzucane.

11.4. Mapowania kolumn

Mapowania znajdują się w meta.ColumnMapping i są wybierane według:

- SourceSystem_ID,
- Entity_Type,
- Source_Column_Name.

Kolumna Canonical_Column_Name wskazuje pole modelu PERSON lub PARTY. Mapowania dla źródeł projektu są inicjalizowane przez scripts/init_proposed_mssql_schema.sql. Brak jakiegokolwiek mapowania dla danej kombinacji źródła i encji powoduje przerwanie stagingu.

Przykładowe mapowania:

Źródło	Kolumna źródłowa	Typ encji	Kolumna kanoniczna
PESEL	Imie	PERSON	First_Name
PESEL	DataUrodzenia	PERSON	Birth_Date
CEIDG	firma.wlasciciel.nazwisko	PERSON	Last_Name
CEIDG	firma.nazwa	PARTY	Name
KRS	numerKRS	PARTY	Identifiers_JSON
GLEIF	LegalName	PARTY	Name
VAT	accountNumbers	PARTY	Bank_Accounts_JSON
INSURANCE_CORE	PARTY_LABEL	PARTY	Name
INSURANCE_CORE	NATIONAL_REF	PERSON	PESEL

Nazwy kolumn są porównywane bez uwzględnienia wielkości liter. Notacja kropkowa obsługuje zarówno zagnieżdżone obiekty, na przykład firma.wlasciciel.imie, jak i płaskie nagłówki CSV zawierające kropki.

Jeżeli kilka kolumn źródłowych prowadzi do tego samego pola kanonicznego, wybierana jest pierwsza niepusta wartość według kolejności mapowań pobranych z bazy.

11.5. Raport mapowania

Dla każdego rekordu mapper ustala:

- kolumny zdefiniowane w mapowaniu, których nie było w rekordzie źródłowym;
- kolumny obecne w źródle, które nie zostały rozpoznane przez mapowanie.

Odpowiedź API agreguje te informacje jako słowniki missing_columns i unrecognized_columns, w których wartością jest liczba rekordów dotkniętych danym problemem. Brak pojedynczej mapowanej kolumny nie zatrzymuje ładowania. Pole docelowe pozostaje puste, a brak jest raportowany.

Kolumny użyte jako techniczny identyfikator rekordu oraz rozpoznane szerokie pola relacji KRS nie są wykazywane jako nierozpoznane. Raport służy do wykrywania zmian struktury źródła, ale nie zastępuje późniejszej walidacji wartości.

11.6. Budowa rekordu stagingowego

Każdy zapisany rekord otrzymuje:

- ImportBatch_ID,
- RawFile_ID,
- Source_Record_ID,
- pola kanoniczne właściwe dla typu encji,
- Raw_Record_JSON,

- techniczny czas utworzenia.

Raw_Record_JSON jest kopią pojedynczego rekordu po usunięciu niedrukowalnych znaków z wartości tekstowych. Pozwala odnieść dane kanoniczne do struktury wejściowej bez ponownego parsowania całego materiału RAW.

Identyfikator rekordu źródłowego

Source_Record_ID jest wybierany w kolejności:

1. wartość przypisana przez mapowanie do Source_Record_ID;
2. jedna z rozpoznawanych kolumn źródłowych, m.in. id, firma.id, numerKRS, PESEL albo LEI;
3. numer rekordu w przetwarzanym materiale.

Ostatni wariant zapewnia techniczne wskazanie rekordu, ale nie jest stabilnym identyfikatorem biznesowym po zmianie kolejności danych wejściowych.

11.7. Model PERSON

Person_Staging przechowuje identyfikatory osoby, imiona i nazwiska, dane urodzenia, płeć, obywatelstwo, kontakt oraz elementy adresu.

Podczas budowy rekordu:

- Birth_Date jest konwertowane do typu DATE;
- Sex jest mapowane do wartości BIT, gdzie rozpoznawane oznaczenia kobiety dają true, a mężczyzny false;
- niepoprawna lub nierozpoznana wartość daty albo płci jest zapisywana jako NULL.

Dla szerokich danych KRS mapper wybiera jedną spójną grupę osoby powiązanej, zamiast łączyć imię, nazwisko i PESEL pochodzące z różnych ról lub slotów.

11.8. Model PARTY

Party_Staging obejmuje profil podmiotu, identyfikatory, adres, dane rejestrowe, kontakt, rachunki bankowe oraz informacje o osobach i podmiotach powiązanych.

Wybrane wartości są przygotowywane do zapisu:

- daty rejestrowe i daty relacji są konwertowane do DATE;
- Has_Virtual_Accounts jest konwertowane do BIT;
- rachunki bankowe są zapisywane jako tablica JSON;
- identyfikatory pochodzące z wielu kolumn są łączone w Identifiers_JSON.

Rozpoznawane klucze identyfikatorów obejmują m.in. NIP, REGON, KRS, LEI i numer UKNF. Dzięki temu źródła używające różnych nazw, takich jak nip, TAX_REF lub Numer NIP, tworzą wspólną strukturę JSON.

Szerokie kolumny KRS opisujące członków zarządu, prokurentów, wspólników, likwidatorów i członków rady nadzorczej są grupowane do Related_Persons_JSON. Dane wspólników będących podmiotami trafiają do Related_Parties_JSON. Struktury zachowują rolę lub rodzaj relacji, numer slotu, dane identyfikacyjne oraz dostępny okres obowiązywania.

11.9. Zakres konwersji

Staging wykonuje wyłącznie konwersje potrzebne do utworzenia spójnego rekordu technicznego. Nie rozdziela złożonych linii adresowych na ulicę, numer budynku, kod i miejscowość. Jeżeli źródło przekazuje cały adres w jednym polu, wartość pozostaje w kolumnie wskazanej przez mapowanie.

Nie są również wykonywane:

- normalizacja wielkości liter i znaków diakrytycznych;
- standaryzacja numerów telefonów i adresów e-mail;
- kontrola sum identyfikatorów;
- walidacja danych z TERYT;
- ocena jakości rekordów.

Operacje te należą do kolejnych warstw procesu.

11.10. Statusy, błędy i powtarzalność

Ponowne załadowanie tego samego RawFile_ID do tej samej tabeli stagingowej jest blokowane. Taki przypadek nie ustawia wcześniej poprawnej partii na FAILED.

HTTP 400 jest zwracany m.in. przy:

- braku materiału RAW lub partii;
- braku mapowań kolumn;
- nieobsługiwanym albo uszkodzonym formacie;
- ponownej próbie załadowania tego samego zakresu.

Pozostałe wyjątki kończą log STAGING_LOAD i partię statusem FAILED, o ile zostały już utworzone. Niepoprawny entity_type nie jest obecnie przechwytywany przez listę kontrolowanych wyjątków endpointu i może zostać zwrócony jako HTTP 500.

Repozytorium zatwierdza osobno zmianę statusu partii, utworzenie logu, zapis rekordów i zakończenie procesu. Operacja nie jest jedną transakcją obejmującą cały staging.

11.11. Ograniczenia

Aktualna implementacja:

- odczytuje całe rekordy źródłowe do pamięci przed mapowaniem;
- wymaga mapowań zdefiniowanych w bazie;
- nie udostępnia administracyjnego API do zarządzania mapowaniami;
- zapisuje nierozpoznane daty i wartości logiczne jako NULL bez osobnego błędu;
- nie aktualizuje istniejącego stagingu, lecz blokuje ponowne ładowanie;
- zachowuje tylko jeden rekord PERSON z szerokiego rekordu KRS, mimo że może on zawierać wiele osób powiązanych.

Pełne listy kolumn tabel stagingowych i ich ograniczenia opisano w rozdziale 8.

11.12. Odniesienie do implementacji

Obszar	Lokalizacja
endpoint stagingu	app/layers/staging_validation/api.py
przebieg ładowania i konwersje typów	app/layers/staging_validation/service.py
mapowanie do modelu kanonicznego	app/layers/staging_validation/mapper.py
operacje bazodanowe	app/layers/staging_validation/repository.py
modele ColumnMapping, Person_Staging, Party_Staging	app/layers/staging_validation/models.py
definicje tabel i mapowania źródeł	scripts/init_proposed_mssql_schema.sql
testy mapowania	tests/test_staging_mapper.py

12. Preprocessing i standaryzacja

Warstwa preprocessing przekształca dane zapisane w tabelach staging do ujednoliconej postaci wykorzystywanej przez walidację, wyszukiwanie duplikatów i proces goldenizacji. Oryginalne wartości pozostają w tabelach staging, natomiast ich znormalizowane odpowiedniki są zapisywane w tabelach Person_Preprocessed albo Party_Preprocessed.

Preprocessing nie rozstrzyga, czy wartość jest poprawna biznesowo. Przykładowo ujednolici zapis numeru telefonu lub adresu, ale nie potwierdza ich istnienia. Takie kontrole należą do kolejnych etapów przetwarzania.

12.1. Uruchomienie procesu

Proces jest udostępniony przez endpoint:

POST /layers/preprocessing/preprocessing-load

Żądanie zawiera:

- raw_file_id – identyfikator pliku, którego rekordy mają zostać przetworzone,
- entity_type – typ encji: PERSON albo PARTY.

Serwis pobiera z warstwy staging wszystkie rekordy powiązane z podanym plikiem i typem encji. Dla każdego rekordu tworzy dokładnie jeden rekord warstwy preprocessed. Relacja ta jest zabezpieczona ograniczeniem unikalności na identyfikatorze rekordu staging.

Jeżeli plik nie ma odpowiednich rekordów staging albo został już przetworzony dla danego typu encji, żądanie jest odrzucane z kodem HTTP 400. Ponowne uruchomienie nie nadpisuje istniejących danych i nie tworzy ich kopii.

12.2. Przebieg przetwarzania

Przetwarzanie obejmuje następujące kroki:

1. normalizację i sprawdzenie typu encji,
2. pobranie rekordów staging dla wskazanego pliku,
3. sprawdzenie, czy dane nie zostały już przetworzone,
4. utworzenie wpisu ProcessLog dla kroku STANDARDIZATION,
5. zastosowanie reguł właściwych dla osoby albo podmiotu,
6. zapis rekordów w odpowiedniej tabeli preprocessed,
7. zakończenie wpisu procesu statusem SUCCESS lub FAILED.

Preprocessing nie zmienia statusu całej partii importowej, ponieważ odpowiada wyłącznie za jeden z etapów jej przetwarzania. Aktualna implementacja nie wykonuje całego etapu jako jednej wspólnej transakcji. Repozytorium zatwierdza osobno utworzenie logu procesu, zapis rekordów preprocessed i zakończenie logu. W przypadku błędu niezakończona część operacji jest wycofywana, ale wcześniej zatwierdzone kroki techniczne mogą pozostać zapisane.

12.3. Reguły wspólne

Podstawowe reguły standaryzacji są stosowane zależnie od rodzaju pola.

Rodzaj danych	Sposób standaryzacji
Tekst	Usunięcie nadmiarowych spacji i zamiana liter na wielkie. Polskie znaki są zachowywane.
Identyfikator	Zamiana liter na wielkie oraz usunięcie separatorów i znaków innych niż litery i cyfry.
Adres e-mail	Usunięcie spacji na początku i końcu oraz zamiana liter na małe.
Numer telefonu	Próba zapisu w formacie E.164, z domyślnym regionem Polski; w razie braku możliwości pełnego rozpoznania stosowana jest prostsza normalizacja cyfr.
Adres WWW	Zamiana liter na małe oraz usunięcie prefiksów http://, https:// i www..

Rodzaj danych	Sposób standaryzacji
Kod pocztowy	Sprowadzenie do cyfr i zapis w formacie NN-NNN, jeżeli wartość ma pięć cyfr.
Dane JSON	Próba odczytania JSON, uporządkowanie kluczy i zapis w ujednoliconej postaci; niepoprawny JSON jest traktowany jako tekst.

Wartości puste lub zawierające wyłącznie białe znaki są zapisywane jako NULL. Preprocessing nie uzupełnia brakujących danych wartościami domyślnymi, z wyjątkiem opisanych niżej reguł wyprowadzania pól.

12.4. Standaryzacja danych osoby

Dla encji PERSON przetwarzane są w szczególności:

- PESEL,
- imię, drugie imię i nazwisko,
- numer dowodu osobistego i paszportu,
- miejsce urodzenia i obywatelstwo,
- numer telefonu i adres e-mail.

Na podstawie dostępnych części imienia i nazwiska powstaje pole `Full_Name`. Nieobecne elementy są pomijane, dzięki czemu w wartości nie pojawiają się dodatkowe separatory ani wielokrotne spacje.

Data urodzenia, płeć i pola logiczne są przenoszone z warstwy staging w postaci nadanej podczas wcześniejszego ładowania danych. Preprocessing nie wykonuje ponownie ich konwersji.

12.5. Standaryzacja danych podmiotu

Dla encji PARTY ujednolicane są między innymi:

- nazwa pełna i skrócona,
- forma prawna,
- NIP, REGON, KRS i LEI,
- dane rejestrowe i numery decyzji,
- dane kontaktowe,
- informacje o relacjach z innymi podmiotami.

Identyfikatory są odczytywane z pola `Identifiers_JSON`. Jeżeli źródło GLEIF zawiera identyfikator organu walidacyjnego odpowiadający rejestrowi KRS albo REGON, numer rejestracyjny może zostać przypisany do właściwego pola. Wartość podana bezpośrednio w identyfikatorach źródłowych ma pierwszeństwo przed wartością wyprowadzoną.

Jeżeli brakuje nazwy skróconej lub formy prawnej, serwis może wyprowadzić je z nazwy pełnej. Rozpoznawane są najczęściej występujące polskie formy, między innymi spółka akcyjna, spółka z ograniczoną odpowiedzialnością, spółka jawna, spółka komandytowa, fundacja i stowarzyszenie. Określenia opisujące profil działalności, takie jak PPHU, nie są traktowane jako forma prawna.

Reguła ta ma charakter pomocniczy. Nie zastępuje danych rejestrowych i nie przesądza o poprawności prawnej rozpoznanego oznaczenia.

12.6. Rozdzielanie i składanie adresu

Adres źródłowy może być zapisany jako jedno pole albo jako kilka części. Preprocessing próbuje rozpoznać:

- ulicę,
- numer budynku,
- numer lokalu,
- kod pocztowy,
- miejscowość,

- kraj.

Obsługiwane są między innymi zapisy, w których ulica występuje przed kodem i miejscowością albo po nich. Numery lokali oznaczone jako m., lok. lub lokal są sprowadzane do zapisu z ukośnikiem. Prefiksy ulic są ujednolicane, przykładowo do UL, AL, OS i PL.

Po rozdzieleniu elementów tworzona jest również wartość `Full_Address`, zawierająca dostępne części adresu w ustalonej kolejności. Ułatwia to późniejsze porównywanie rekordów, natomiast osobne kolumny pozwalają dopasowywać tylko wybrane elementy.

Mechanizm jest oparty na regułach i wyrażeniach regularnych. Nietypowe lub niejednoznaczne adresy mogą nie zostać rozdzielone w pełni. Weryfikacja miejscowości i kodu pocztowego względem danych TERYT jest wykonywana w warstwie walidacji, a nie podczas preprocessingu.

12.7. Informacja o zastosowanych regułach

Każdy rekord `preprocessed` zawiera pole `Preprocessing_Rules_JSON`. Przechowuje ono ogólne oznaczenia zastosowanych grup reguł:

- standaryzacji tekstu,
- normalizacji telefonu,
- rozdzielania adresu,
- czasu wykonania operacji.

Pole opisuje sposób przetwarzania na poziomie ogólnym. Nie stanowi szczegółowego rejestru każdej zmiany wykonanej na poszczególnych wartościach. Zachowanie wartości źródłowych w warstwie staging umożliwia jednak porównanie danych przed i po standaryzacji.

12.8. Przygotowanie danych do dopasowywania

Tabele `preprocessed` zawierają zarówno ujednolicone pola pojedyncze, jak i pola złożone używane przez kolejne etapy. Dla osoby są to przykładowo pełne imię i nazwisko oraz zestawienia imienia, nazwiska, daty i miejsca urodzenia. Dla podmiotu wykorzystywane są przede wszystkim identyfikatory rejestrowe, nazwa, adres i dane kontaktowe.

W bazie utworzono indeksy wspierające wyszukiwanie po najważniejszych identyfikatorach i kombinacjach pól. Obejmują one między innymi PESEL, dokumenty tożsamości, NIP, REGON, KRS, LEI, dane kontaktowe oraz wybrane zestawienia nazw i adresów.

Preprocessing przygotowuje dane do porównania, ale nie łączy rekordów i nie wybiera rekordu golden. Decyzje te są podejmowane przez dalsze warstwy systemu.

12.9. Ograniczenia

Najważniejsze ograniczenia etapu to:

- regułowy charakter rozpoznawania adresów i form prawnych,
- zależność dokładności normalizacji telefonów od możliwości biblioteki `phonenumbers`,
- brak kontroli poprawności biznesowej identyfikatorów i danych kontaktowych,
- ogólny, a nie szczegółowy charakter pola `Preprocessing_Rules_JSON`,
- przetwarzanie wszystkich rekordów danego pliku i typu encji w ramach jednego wywołania.

Ograniczenia te nie powodują utraty danych źródłowych, ponieważ rekord staging pozostaje dostępny i jest wskazywany przez rekord `preprocessed`.

12.10. Odniesienia do implementacji

Najważniejsze elementy implementacji znajdują się w plikach:

- `app/layers/preprocessing/api.py` – endpoint uruchamiający preprocessing,
- `app/layers/preprocessing/service.py` – reguły standaryzacji i przebieg procesu,
- `app/layers/preprocessing/repository.py` – odczyt i zapis danych,

- `app/layers/preprocessing/models.py` – modele tabel preprocessed,
- `app/layers/preprocessing/schemas.py` – model odpowiedzi API,
- `tests/test_preprocessing.py` – testy reguł normalizacji,
- `scripts/init_proposed_mssql_schema.sql` – definicje tabel, ograniczeń i indeksów.

13. Walidacja

Warstwa walidacji ocenia jakość danych po preprocessingu. Nie modyfikuje sprawdzanych rekordów i nie uzupełnia ich danymi zewnętrznymi. Dla każdej wykonanej reguły zapisuje osobny wynik, który wskazuje sprawdzane pole, wartość, status oraz komunikat.

Błąd danych nie przerywa przetwarzania pozostałych rekordów. Dzięki temu po zakończeniu procesu dostępny jest zestaw wszystkich wykrytych niezgodności, a nie tylko pierwszy napotkany problem.

13.1. Uruchomienie procesu

Walidację uruchamia endpoint:

POST /layers/validation/validation-load

Żądanie zawiera:

- raw_file_id – identyfikator pliku,
- entity_type – typ encji PERSON albo PARTY,
- check_email_dns – informację, czy oprócz składni adresu e-mail należy sprawdzić jego domenę; domyślna wartość to true.

Walidacja wymaga wcześniejszego utworzenia rekordów preprocessed. Serwis pobiera odpowiadające sobie rekordy z tabel staging i preprocessed. Brak takich danych lub niepoprawny typ encji powoduje odpowiedź HTTP 400.

13.2. Przebieg walidacji

Proces obejmuje:

1. pobranie rekordów staging i preprocessed dla wskazanego pliku,
2. usunięcie wcześniejszych wyników walidacji tego pliku i typu encji,
3. utworzenie wpisu ProcessLog dla kroku VALIDATION,
4. wykonanie reguł odpowiednich dla osoby albo podmiotu,
5. zapis wyników w tabeli stg.Validation_Result,
6. zakończenie wpisu procesu statusem SUCCESS lub FAILED.

Ponowne uruchomienie zastępuje wcześniejszy zestaw wyników, dlatego w tabeli pozostają rezultaty ostatniego wykonania walidacji. Może się on różnić od poprzedniego między innymi po zmianie danych, reguł, plików TERYT albo ustawienia kontroli DNS.

Status procesu opisuje wykonanie techniczne etapu. Wykrycie niepoprawnych wartości nie oznacza awarii procesu. Jeżeli wszystkie reguły zostały wykonane i zapisane, ProcessLog otrzymuje status SUCCESS, nawet gdy część wyników ma status ERROR.

13.3. Model wyniku

Każdy wpis Validation_Result dotyczy jednej reguły zastosowanej do jednego rekordu. Zawiera między innymi:

- identyfikatory partii, pliku, rekordu staging i rekordu preprocessed,
- typ encji,
- poziom danych: STAGING albo PREPROCESSING,
- kod reguły i nazwę sprawdzanego pola,
- sprawdzaną wartość,
- status, poziom istotności i komunikat.

Poprawny wynik ma status PASS, poziom INFO i komunikat OK. Niezgodność otrzymuje status oraz poziom istotności ERROR, a komunikat zawiera kod rozpoznanego błędu. Podsumowanie odpowiedzi API podaje liczbę wszystkich wyników oraz liczbę reguł zakończonych jako PASS i ERROR. Nie są to liczby poprawnych i błędnych rekordów, ponieważ jeden rekord jest sprawdzany przez wiele reguł.

Zapisane wyniki można później odczytać przez endpoint GET /layers/serving/validation-results, z

opcjonalnym filtrowaniem po typie encji, systemie źródłowym i kodzie reguły.

13.4. Walidacja osoby

Dla encji PERSON wykonywane są następujące kontrole:

Reguła	Zakres
PERSON_PESSEL_CHECKSUM	Format i suma kontrolna numeru PESEL.
PERSON_PESSEL_BIRTH_DATE_MATCH	Zgodność daty urodzenia zapisanej w PESEL z datą w rekordzie.
PERSON_PESSEL_SEX_MATCH	Zgodność płci zapisanej w PESEL z wartością źródłową.
PERSON_PESSEL_BIRTH_DATE_NOT_FUTURE	Sprawdzenie, czy data zakodowana w PESEL nie jest przyszła.
PERSON_BIRTH_DATE_NOT_FUTURE	Sprawdzenie, czy data urodzenia z rekordu nie jest przyszła.
PERSON_EMAIL_SYNTAX	Kontrola adresu e-mail i opcjonalnie jego domeny.
PERSON_ID_CARD_CHECKSUM	Format i suma kontrolna polskiego dowodu osobistego.
Reguły nazw	Dopuszczalny zapis imion i nazwisk, w tym polskich znaków, spacji, apostrofu i łącznika.

Przy porównaniu płci serwis w pierwszej kolejności próbuje odczytać jej wartość z Raw_Record_JSON. Jest to zabezpieczenie przed błędną konwersją wartości źródłowej do typu logicznego podczas wcześniejszego etapu. Jeżeli wartość nie jest dostępna w surowym rekordzie, używane jest pole staging.

Brak PESEL jest traktowany jako błąd podstawowej reguły jego poprawności. Większość pozostałych pól opcjonalnych nie powoduje błędów wyłącznie z powodu braku wartości. Przykładowo suma kontrolna dowodu jest sprawdzana dopiero wtedy, gdy numer został podany.

13.5. Walidacja podmiotu

Dla encji PARTY sprawdzane są:

Reguła	Zakres
PARTY_NIP_CHECKSUM	Format i suma kontrolna NIP.
PARTY_REGON_CHECKSUM	Format i suma kontrolna REGON o długości 9 albo 14 cyfr.
PARTY_KRS_FORMAT	Obecność dokładnie 10 cyfr w numerze KRS.
PARTY_LEI_CHECKSUM	Format i suma kontrolna identyfikatora LEI.
PARTY_EMAIL_SYNTAX	Kontrola adresu e-mail i opcjonalnie jego domeny.
PARTY_NAME_STRING	Sprawdzenie, czy nazwa jest niepustym tekstem.

NIP, REGON, KRS i LEI są polami opcjonalnymi na tym etapie. Brak identyfikatora daje wynik PASS, natomiast podana wartość musi spełnić odpowiednią regułę. W przypadku KRS kontrolowany jest format, a nie zgodność numeru z rejestrem.

Walidacja podmiotu obejmuje również porządek dat:

- data utworzenia nie może być późniejsza od daty wyrejestrowania,
- data rejestracji nie może być późniejsza od daty wyrejestrowania,
- data kolejnego odnowienia nie może być wcześniejsza od daty ostatniej aktualizacji,
- początek relacji z bezpośrednim podmiotem dominującym nie może być późniejszy od jej końca,
- początek relacji z najwyższym podmiotem dominującym nie może być późniejszy od jej końca.

Jeżeli jedna z dat pary nie występuje, kolejność nie jest kwestionowana. Podana, lecz nierozpoznana wartość daty powoduje wynik ERROR.

13.6. Kontrola adresu z wykorzystaniem TERYT

Walidacja adresu korzysta z plików SIMC.csv i ULIC.csv. Dane mogą znajdować się w katalogu wskazanym przez zmienną TERYT_DIR albo w katalogu teryt pod ścieżką FILESTREAM_PATH. W repozytorium dostępne są pliki w

data/teryt.

Pliki można również przekazać przez endpoint:

POST /layers/validation/teryt-load

Endpoint zapisuje przekazane pliki w systemie plików. Nie wzbogaca rekordów i nie importuje TERYT do tabel bazy danych.

Jeżeli oba wymagane pliki są dostępne, dla osoby i podmiotu wykonywane są dwie reguły:

- ADDR_TERYT_CITY_EXISTS – występowanie miejscowości w rejestrze SIMC,
- ADDR_TERYT_STREET_EXISTS – występowanie ulicy w rejestrze ULIC dla rozpoznanej miejscowości.

Nazwy są porównywane po ujednoliceniu wielkości liter i odstępów. Dla ulic uwzględniane są prefiksy UL, AL, OS i PL. Serwis obsługuje również wariant, w którym kolejność ostatnich członów nazwy ulicy jest odwrócona względem zapisu TERYT.

Brak miejscowości lub ulicy nie jest sam w sobie oznaczany jako błąd TERYT. Jeżeli miejscowość została podana, ale nie występuje w SIMC, zarówno kontrola miejscowości, jak i powiązanej ulicy otrzymuje ERROR. Gdy pliki TERYT nie są dostępne, reguły adresowe nie są wykonywane i nie powstają dla nich wyniki.

13.7. Walidacja adresu e-mail

Adres e-mail jest najpierw sprawdzany składniowo. Jeżeli parametr `check_email_dns` ma wartość `true`, po poprawnej kontroli składni serwis sprawdza również istnienie domeny.

Kontrola domeny próbuje rozwiązać jej adres sieciowy, a następnie rekord MX. Zapytania DNS mają krótki limit czasu, aby ograniczyć wpływ niedostępnej sieci na czas przetwarzania. Wynik tej reguły może zatem zależeć od aktualnej dostępności DNS. Dla środowiska bez dostępu do sieci można wyłączyć tę część kontroli, pozostawiając samą walidację składni.

13.8. Wykorzystanie wyników

Wyniki walidacji są wykorzystywane przez warstwę goldenizacji przy wyborze wartości przeżywającej. Status jest pobierany dla konkretnego pola rekordu `preprocessed`. Wartość z wynikiem `PASS` może otrzymać pierwszeństwo przed kandydatem, który nie przeszedł walidacji.

Potwierdzenie adresu przez TERYT stanowi dodatkową przesłankę przy wyborze adresu. Walidacja nie wykonuje jednak matchingu, nie grupuje rekordów i nie tworzy rekordu `golden`.

13.9. Ograniczenia

Najważniejsze ograniczenia walidacji to:

- brak sprawdzania identyfikatorów bezpośrednio w rejestrach zewnętrznych,
- kontrola KRS ograniczona do formatu,
- zależność kontroli domeny e-mail od dostępności DNS,
- wykonywanie reguł TERYT tylko wtedy, gdy oba pliki referencyjne są dostępne,
- brak weryfikacji numerów telefonu i adresów WWW,
- zastępowanie wcześniejszych wyników podczas ponownego uruchomienia.

Warstwa rejestruje wykryte problemy, lecz nie poprawia wartości automatycznie. Zachowane powiązania z rekordami `staging` i `preprocessed` pozwalają ustalić, której wartości oraz reguły dotyczy wynik.

13.10. Odniesienia do implementacji

Najważniejsze elementy implementacji znajdują się w plikach:

- `app/layers/validation/api.py` – endpointy walidacji i ładowania plików TERYT,
- `app/layers/validation/service.py` – reguły walidacyjne i obsługa TERYT,
- `app/layers/validation/repository.py` – pobieranie danych i zapis wyników,
- `app/layers/validation/models.py` – model `Validation_Result`,

- `app/layers/validation/schemas.py` – modele odpowiedzi API,
- `tests/test_validation.py` – testy reguł osoby i podmiotu,
- `tests/test_teryt_validation.py` – testy kontroli adresów,
- `scripts/init_proposed_mssql_schema.sql` – definicja tabeli i indeksów walidacji.

14. Matching, grupowanie i budowa Golden Record

Warstwa `integration_golden` odpowiada za rozpoznawanie rekordów opisujących tę samą osobę albo ten sam podmiot, budowę stabilnych grup encji oraz materializację wynikowych rekordów w schemacie `gold`. Jest to etap, w którym kończy się przetwarzanie porównawcze, a rozpoczyna utrwalanie wspólnej reprezentacji danych.

Implementacja rozdziela ten obszar na cztery kroki:

1. wyszukanie kandydatów metodą Levenshteina,
2. ponowną ocenę kandydatów metodą Jaro-Winklera,
3. grupowanie par `AUTO_MERGE`,
4. utworzenie albo aktualizację rekordów `GOLD`.

14.1. Uruchamianie procesu

Warstwa udostępnia cztery endpointy:

```
POST /layers/integration_golden/match-candidates
POST /layers/integration_golden/match-candidates/jaro-winkler
POST /layers/integration_golden/match-groups
POST /layers/integration_golden/golden-load
```

Zakres parametrów:

- `entity_type` przyjmuje `PERSON` albo `PARTY`,
- `raw_file_id` zawęża matching albo ładowanie `GOLD` do wskazanego materiału,
- `min_score` pozwala sterować progiem kandydata w obu etapach matchingu,
- `max_pairs` ogranicza liczbę porównań w etapie Levenshteina,
- `entity_group_id` pozwala uruchomić materializację `GOLD` dla jednej grupy.

Matching wymaga wcześniejszego przygotowania rekordów `preprocessed`. Grupowanie wymaga zapisanych wyników Jaro-Winklera. Materializacja `GOLD` wymaga istnienia grup encji.

14.2. Model reguł dopasowania

Matching jest regułowy. Każde pole używane do porównania ma:

- wagę,
- rolę,
- sposób porównania,
- listę aliasów odczytu z rekordu `preprocessed`,
- opcjonalny status pola rozstrzygającego.

W implementacji używane są role:

- `STRONG` - silne identyfikatory,
- `FIXED` - pola oczekiwane jako zgodne,
- `SEMI_FIXED` - pola zwykle stabilne, ale dopuszczające rozbieżności,
- `DYNAMIC` - pola kontaktowe i adresowe,
- `CONTEXT` - pola pomocnicze, kontekstowe.

Reguły dla osoby

Najwyższą wagę mają identyfikatory i pola tożsamości:

- `PESEL`,
- numer dowodu,
- numer paszportu,
- data urodzenia,

- imię i nazwisko,
- pełne imię i nazwisko,
- miejsce urodzenia.

Niższą wagę mają pola dynamiczne, takie jak:

- telefon,
- e-mail,
- adres,
- kod pocztowy,
- województwo.

Testy potwierdzają między innymi, że zgodny PESEL pozwala na automatyczne scalenie mimo zmian danych dynamicznych, natomiast konflikt wielu pól stabilnych obniża wynik do CANDIDATE albo NO_MATCH.

Reguły dla podmiotu

Najsilniejsze identyfikatory to:

- NIP,
- REGON,
- KRS,
- LEI,
- w wybranych źródłach także numer decyzji i numer rejestrowy.

Dużą wagę mają również:

- nazwa,
- data utworzenia,
- nazwy i relacje podmiotów dominujących,
- wybrane pola rejestrowe i statusowe.

Pola adresowe, kontaktowe i kontekstowe wpływają na wynik, ale nie mają siły samodzielnego rozstrzygnięcia. Dzięki temu podobne adresy lub wspólne cechy branżowe nie powodują automatycznego scalenia niepowiązanych podmiotów.

14.3. Etap Levenshteina

Pierwszy etap matchingu wykorzystuje funkcję `score_match()`, która dla każdej pary rekordów:

1. odczytuje wartości pól zdefiniowanych w regułach,
2. liczy podobieństwo Levenshteina,
3. mnoży wynik przez wagę pola,
4. buduje łączny wynik ważony,
5. rejestruje zgodne pola silne oraz pola konfliktowe.

Domyślne progi etapu są następujące:

- próg wejścia kandydata: 0.50,
- próg REVIEW: 0.70,
- próg AUTO_MERGE: 0.90,
- dodatkowy próg 0.95 dla bardzo wysokiego podobieństwa bez krytycznych konfliktów.

Wynik klasyfikowany jest do jednej z decyzji:

- AUTO_MERGE,
- REVIEW,

- CANDIDATE,
- NO_MATCH.

Sama zgodność silnego identyfikatora nie wystarcza zawsze do AUTO_MERGE. Jeżeli równocześnie wystąpią konflikty pól stabilnych, wynik może zostać obniżony do CANDIDATE albo NO_MATCH. Implementacja traktuje krytycznie zwłaszcza konflikty identyfikatorów takich jak PESEL, NIP, REGON, KRS, numer dowodu i numer paszportu.

Etap zapisuje wyniki do tabeli `stg.Match_Candidate_Levenshtein`. W tabeli pozostają tylko kandydaci, którzy przekroczyli minimalny próg i nie zostali sklasyfikowani jako NO_MATCH.

14.4. Ograniczenie liczby porównań

Aby uniknąć niekontrolowanego wzrostu liczby porównań, etap Levenshteina posiada limit bezpieczeństwa `matching_max_pairs`. Domyślna wartość to `2_000_000`.

Po przekroczeniu limitu serwis zwraca błąd wejściowy. Jest to zabezpieczenie techniczne, a nie reguła biznesowa. W razie potrzeby limit można zwiększyć albo wyłączyć przez ustawienie 0.

14.5. Etap Jaro-Winklera

Drugi etap nie przegląda całego zbioru rekordów od nowa. Ocenia wyłącznie kandydatów zapisanych po etapie Levenshteina.

Funkcja `score_jaro_winkler_match()`:

- koncentruje się na polach tekstowych,
- zachowuje informację o silnych polach i wcześniejszych konfliktach,
- ponownie ocenia podobieństwo nazw, nazwisk, miejsc urodzenia, adresów i wybranych pól opisowych.

Domyślne progi tego etapu są wyższe:

- próg kandydata: 0.78,
- próg REVIEW: 0.86,
- próg AUTO_MERGE: 0.94.

Wyniki są zapisywane do `stg.Match_Candidate_JaroWinkler`. Jeżeli kandydat nie spełnia progu i nie ma silnych podstaw do dalszej analizy, jest odrzucany na tym etapie.

14.6. Znaczenie decyzji matchingowych

Decyzje są interpretowane następująco:

- AUTO_MERGE - para może zostać użyta do budowy wspólnej grupy encji,
- REVIEW - para jest podobna, ale wymagałaby oceny ręcznej,
- CANDIDATE - para została wykryta jako potencjalnie interesująca, lecz nie spełnia warunków połączenia,
- NO_MATCH - para nie powinna być dalej brana pod uwagę.

Aktualna implementacja wykorzystuje automatycznie tylko pary AUTO_MERGE. Przypadki REVIEW są zachowywane w wynikach matchingu, ale system nie zawiera pełnego procesu ręcznej obsługi takich decyzji.

14.7. Grupowanie rekordów

Grupowanie opiera się na parach AUTO_MERGE po etapie Jaro-Winklera. Funkcja `build_entity_groups()` buduje spójne składowe grafu rekordów przy użyciu algorytmu typu union-find.

W praktyce oznacza to, że:

- jeżeli rekord A łączy się z B,
- a rekord B łączy się z C,
- to wszystkie trzy rekordy trafiają do jednej grupy, nawet jeśli para A-C nie była oceniana bezpośrednio jako niezależny przypadek połączenia.

Każda grupa otrzymuje stabilny `Group_Key`, wyliczony jako SHA-256 z uporządkowanej listy identyfikatorów `Preprocessed_ID`.

Po zbudowaniu grup wieloelementowych funkcja `add_singleton_entity_groups()` dopisuje również rekordy niepołączone z żadnym innym rekordem. Dzięki temu każda encja `preprocessed` należy do dokładnie jednej grupy:

- grupy scalonej,
- albo grupy jednoelementowej.

Wynik grupowania jest zapisywany w tabelach:

- `stg.Entity_Group`,
- `stg.Entity_Group_Member`.

Ponowne uruchomienie grupowania zastępuje aktualne członkostwa grup dla danego typu encji na podstawie bieżących kandydatów `AUTO_MERGE`.

14.8. Budowa Golden Record osoby

Materializacja osoby odbywa się funkcją `create_or_update_golden_person()`. Dla wskazanej grupy serwis:

1. pobiera wszystkich członków grupy i odpowiadające im rekordy `preprocessed`,
2. wybiera wartości przeżywające dla pól osoby,
3. wyszukuje istniejącą osobę po identyfikatorach tożsamości,
4. tworzy nowy rekord `gold.DimPerson` albo aktualizuje istniejący,
5. buduje lub ponownie wykorzystuje adres,
6. zakłada albo ponownie wykorzystuje powiązanie osoby z adresem.

Tożsamość osoby jest wyszukiwana po:

- PESEL,
- numerze dowodu osobistego,
- numerze paszportu.

Jeżeli rekord już istnieje, aktualizacja nie tworzy nowej osoby, tylko nadpisuje wartości bieżące i zapisuje zmiany atrybutów.

14.9. Budowa Golden Record podmiotu

Materializacja podmiotu działa analogicznie, ale z uwzględnieniem szerszego zestawu identyfikatorów. Funkcja `create_or_update_golden_party()`:

1. buduje wartości przeżywające dla atrybutów podmiotu,
2. wyszukuje istniejący rekord po NIP, REGON, KRS i LEI,
3. tworzy albo aktualizuje `gold.DimParty`,
4. tworzy albo ponownie wykorzystuje adres rejestrowy,
5. zapisuje powiązanie podmiotu z adresem,
6. zapisuje identyfikatory podmiotu jako osobne rekordy `faktless`.

Dla podmiotu utrwalane mogą być między innymi:

- NIP,
- REGON,
- KRS,
- LEI,
- numer rejestrowy KNF,
- numer decyzji.

Liczba zapisanych identyfikatorów zależy od danych dostępnych w zwycięskich wartościach grupy.

14.10. Reguły survivorship

Wybór wartości końcowej dla atrybutu realizuje funkcja `select_survivor_value()`. Kolejność kryteriów jest

stała:

1. obecność niepustej wartości,
2. pozytywny wynik walidacji,
3. potwierdzenie adresu w TERYT dla pól adresowych,
4. priorytet źródła dla danego atrybutu,
5. poziom zaufania źródła,
6. czas rozpoczęcia importu,
7. kolejność wejściowa jako awaryjny tie-break.

Źródła mają różne priorytety zależnie od encji i pola. Przykładowo:

- dla danych tożsamości osoby preferowany jest PESEL,
- dla kontaktu osoby preferowane są CEIDG i INSURANCE_CORE,
- dla adresów podmiotu wysokie priorytety mają REGON, VAT, CEIDG i KRS,
- dla pola LEI preferowane jest źródło GLEIF.

Dobór wartości nie jest więc jednym globalnym rankingiem źródeł, tylko zestawem reguł zależnych od atrybutu.

14.11. Adresy i powiązania adresowe

Adres GOLD jest budowany osobno od rekordu osoby lub podmiotu. Funkcja `create_golden_address_for_records()`:

- wybiera zwycięskie pola adresowe,
- sprawdza, czy taki adres już istnieje,
- tworzy nowy rekord `gold.DimAddress` albo ponownie używa istniejącego.

Powiązanie z adresem również nie jest bezwarunkowo tworzone od nowa. Funkcja `ensure_golden_address_link()`:

- dla osoby używa typu adresu `RESIDENCE`,
- dla podmiotu używa typu `REGISTERED`,
- może pozostawić istniejące aktywne powiązanie,
- może zamknąć wcześniejsze aktywne powiązanie przez ustawienie `Valid_To`,
- może utworzyć nowe aktywne powiązanie z `Valid_From`.

Dzięki temu zmiana adresu nie niszczy wprost poprzedniego powiązania, lecz kończy jego obowiązywanie i zapisuje nowe.

14.12. Odrzucanie grup

Nie każda grupa prowadzi do powstania Golden Record. Aktualna implementacja zawiera jawny mechanizm odrzucenia grupy podmiotu, jeżeli po zastosowaniu reguł survivorship nie da się wyznaczyć wymaganych pól.

Obecnie dla PARTY polem wymaganym jest:

- Name.

Jeżeli nazwa pozostaje pusta, serwis:

- nie tworzy rekordu `DimParty`,
- zapisuje odrzucenie w `stg.Golden_Record_Reject`,
- zwraca informację o brakujących polach i identyfikatorach członków grupy.

Mechanizm ten służy do zachowania pełnej ścieżki technicznej również dla przypadków, w których rekord wynikowy nie może zostać utworzony poprawnie.

14.13. Transakcyjność etapu GOLD

Ładowanie GOLD nie jest realizowane jako jedna wspólna transakcja dla wszystkich grup objętych wywołaniem.

Implementacja przetwarza grupy kolejno i zatwierdza zapis po wykonaniu logiki dla konkretnej grupy.

Oznacza to, że:

- część grup może zostać poprawnie utrwalona jeszcze przed wystąpieniem błędu,
- awaria późniejszej grupy nie usuwa wcześniej zapisanych wyników,
- ponowne uruchomienie opiera się na logice create-or-update.

Taki model ogranicza ryzyko pełnej utraty postępu przy dłuższym przebiegu, ale oznacza też brak atomowości dla całego wywołania golden-load.

14.14. Ograniczenia

Najważniejsze ograniczenia obecnej implementacji to:

- tylko decyzje AUTO_MERGE wpływają na grupowanie i budowę GOLD,
- brak pełnego procesu ręcznej obsługi przypadków REVIEW,
- grupowanie działa dla całego typu encji, a nie dla pojedynczego RawFile_ID,
- lineage przechowuje stan aktualnie wybranych wartości, a nie pełną historię kolejnych wersji wyboru,
- odrzucenie grup wprost zaimplementowano obecnie dla wymaganego pola nazwy podmiotu,
- budowa GOLD wykonuje zapis etapami dla kolejnych grup, a nie jako jedną transakcję globalną.

Ograniczenia te nie zmieniają faktu, że rdzeń automatycznego matchingu, grupowania i materializacji Golden Record działa w obecnym kodzie end-to-end.

14.15. Odniesienia do implementacji

Najważniejsze elementy implementacji znajdują się w plikach:

- `app/layers/integration_golden/api.py` - endpointy matchingu, grupowania i ładowania GOLD,
- `app/layers/integration_golden/service.py` - scoring, klasyfikacja, grupowanie, survivorship i materializacja wymiarów,
- `app/layers/integration_golden/repository.py` - operacje odczytu i zapisu w schemacie stg i gold,
- `app/layers/integration_golden/models.py` - modele kandydatów, grup, odrzuceń i wymiarów GOLD,
- `app/layers/integration_golden/schemas.py` - modele odpowiedzi API,
- `tests/test_integration_golden_matching.py` - testy reguł matchingu, progów i survivorship,
- `tests/test_integration_golden_dimensions.py` - testy budowy rekordów GOLD i aktualizacji wymiarów,
- `tests/test_integration_golden_load.py` - testy przebiegu golden_load, odrzuceń i logiki grup,
- `tests/test_integration_golden_repository.py` - testy podstawowych operacji repozytorium,
- `scripts/init_proposed_mssql_schema.sql` - definicje tabel dla kandydatów, grup i wymiarów.

15. Audytowalność, lineage i historia zmian

System przechowuje informacje pozwalające odtworzyć, z jakiego źródła pochodzą aktualne wartości Golden Record, kiedy wykonano poszczególne kroki procesu oraz jakie atrybuty osób i podmiotów uległy zmianie przy kolejnych aktualizacjach. Mechanizm ten nie jest jednak pełnym wersjonowaniem całego stanu encji w czasie. Obejmuje trzy uzupełniające się obszary:

1. logi technicznego przebiegu procesu,
2. lineage aktualnie wybranych wartości i relacji,
3. rejestr zmian atrybutów wymiarów.

15.1. Logi przebiegu procesu

Podstawowym technicznym śladem wykonania jest tabela meta.ProcessLog. Rekord logu jest tworzony dla wybranych etapów i zawiera:

- ImportBatch_ID,
- opcjonalny RawFile_ID,
- nazwę kroku,
- status,
- czas rozpoczęcia i zakończenia,
- liczbę rekordów wejściowych i wyjściowych,
- komunikat błędu.

W modelu danych dozwolone są kroki takie jak:

- RAW_LOAD,
- STAGING_LOAD,
- STANDARDIZATION,
- VALIDATION,
- GOLDEN_LOAD.

Statusy techniczne obejmują:

- STARTED,
- SUCCESS,
- FAILED.

Log procesu służy do odpowiedzi na pytania:

- czy krok został uruchomiony,
- czy zakończył się poprawnie,
- ile rekordów albo grup objął,
- jaki błąd zatrzymał etap.

Nie jest to jednak dziennik wszystkich decyzji biznesowych podejmowanych podczas matchingu i survivorship. Na przykład matching i grupowanie nie zapisują osobnych wpisów ProcessLog dla każdej pary lub każdej decyzji.

15.2. Logowanie etapu GOLD

Warstwa integration_golden tworzy wpis ProcessLog dla kroku GOLDEN_LOAD, jeżeli wywołanie odbywa się w kontekście konkretnego RawFile_ID.

Przebieg jest następujący:

1. pobranie ImportBatch_ID powiązanego z RawFile_ID,
2. utworzenie logu ze statusem STARTED,
3. wykonanie materializacji grup,

4. zakończenie logu statusem SUCCESS albo FAILED.

Przy powodzeniu zapisywane są:

- liczba grup w zakresie wykonania,
- liczba grup przetworzonych do GOLD.

Przy błędzie zapisywany jest komunikat wyjątku. Log ten opisuje wynik techniczny całego wywołania golden-load, a nie wynik każdej pojedynczej reguły survivorship.

15.3. Czym jest lineage w tym systemie

Lineage opisuje pochodzenie aktualnie wybranej wartości Golden Record albo aktualnie utrwalonej relacji. Dla każdej zapisanej wartości system stara się zachować:

- system źródłowy,
- identyfikator rekordu źródłowego,
- partię importu,
- regułę wyboru,
- poziom zaufania,
- wynik jakości i walidacji.

Lineage nie przechowuje całej ścieżki wszystkich rozważanych kandydatów dla każdego atrybutu. Przechowuje wyłącznie to źródło, które zostało wykorzystane do zapisania bieżącej wartości.

15.4. Tabele lineage

Aktualna implementacja wykorzystuje następujące tabele:

- gold.GoldenPersonLineage,
- gold.GoldenPartyLineage,
- gold.GoldenAddressLineage,
- gold.GoldenPartyIdentityLineage,
- gold.PersonAddressLineage,
- gold.PartyAddressLineage.

Struktura tych tabel jest podobna. Zawierają one między innymi:

- identyfikator obiektu docelowego,
- nazwę atrybutu albo relacji,
- SourceSystem_ID,
- Source_Record_ID,
- ImportBatch_ID,
- Selection_Rule,
- Trust_Score,
- Quality_Score,
- Validation_Status,
- Recorded_At.

W praktyce oznacza to, że dla bieżącej wartości można ustalić nie tylko źródło, lecz także podstawę wyboru, na przykład:

- NON_EMPTY_VALUE,
- PASSED_VALIDATION,
- TERYT_CONFIRMED_ADDRESS,

- SOURCE_PRIORITY,
- TRUST_LEVEL,
- NEWEST_IMPORT,
- INPUT_ORDER_FALLBACK.

15.5. Lineage osób, podmiotów i adresów

Lineage atrybutów wymiarów jest zapisywane funkcją `write_dimension_lineage()`. Mechanizm działa po utworzeniu albo aktualizacji:

- gold.DimPerson,
- gold.DimParty,
- gold.DimAddress,
- gold.FactlessPartyIdentities.

Dla każdego atrybutu, który otrzymał niepustą wartość i dla którego dostępne są dane źródłowe, zapisywany jest osobny wpis lineage.

Przykładowo można ustalić:

- z którego źródła pochodzi imię osoby,
- z której partii importu pochodzi nazwa podmiotu,
- które źródło dostarczyło zwycięski kod pocztowy,
- z jakiego materiału pochodzi zapisany numer LEI.

Lineage adresu jest zapisywane osobno od lineage osoby albo podmiotu. Oznacza to, że adres jako obiekt DimAddress posiada własne pochodzenie swoich atrybutów, niezależne od tego, z kim został później powiązany.

15.6. Lineage relacji adresowych

Oprócz lineage samych wymiarów system zapisuje lineage relacji adresowych:

- PersonAddressLineage dla relacji osoby z adresem,
- PartyAddressLineage dla relacji podmiotu z adresem.

Nie jest to kopia całego zestawu pól adresowych. Implementacja wybiera jeden reprezentatywny wybór źródła dla relacji, zgodnie z priorytetem:

1. Full_Address,
2. Street,
3. City,
4. Postal_Code,
5. Building_Number,
6. Apartment_Number,
7. Postal_City,
8. District,
9. Province,
10. Country.

Na tej podstawie system zapisuje, z którego źródła i importu pochodzi utworzenie albo aktualizacja konkretnego powiązania adresowego. Reguła wyboru jest dodatkowo oznaczana prefiksem ADDRESS_LINK_FROM....

15.7. Wykorzystanie walidacji w lineage

Warstwa goldenizacji nie zapisuje do lineage surowego wyniku wszystkich reguł walidacyjnych. Zamiast tego:

- pobiera zagregowany status dla pola,
- przekształca go do uproszczonego Validation_Status,

- wylicza uproszczony `Quality_Score`.

Agregacja działa następująco:

- jeżeli dla pola wystąpi `ERROR`, wynik pola jest `ERROR`,
- w przeciwnym razie `WARNING` ma pierwszeństwo nad `PASS`,
- przy braku problemów wynik to `PASS`.

`Quality_Score` jest następnie redukowany do prostego sygnału:

- 1.0 dla wartości uznanej za poprawną,
- 0.0 dla wartości niepoprawnej.

Mechanizm ten wzmacnia użyteczność lineage, ale nie zastępuje pełnego wglądu w tabelę `stg.Validation_Result`.

15.8. Rejestr zmian wymiarów

Zmiany atrybutów są zapisywane w tabeli `gold.EntityChangeLog`. Rekord zmiany zawiera:

- typ encji,
- odniesienie do zmienionego obiektu,
- nazwę atrybutu,
- starą wartość,
- nową wartość,
- datę zmiany,
- opcjonalny `ImportBatch_ID`.

Model przewiduje obsługę zmian dla typów:

- `PERSON`,
- `PARTY`,
- `ADDRESS`,
- `PARTY_IDENTITY`.

W aktualnej logice warstwy `integration_golden` rejestr zmian jest wykorzystywany bezpośrednio przy aktualizacji:

- `DimPerson`,
- `DimParty`.

Jeżeli nowa wartość atrybutu różni się od poprzedniej, funkcja `record_dimension_changes()` zapisuje wpis z poprzednią i nową wartością. Gdy wartości są równoważne, zmiana nie jest dopisywana.

15.9. Co dokładnie daje historia zmian

Rejestr zmian pozwala odpowiedzieć na pytania typu:

- która wartość atrybutu osoby została zmieniona,
- jaka była poprzednia i nowa wartość,
- przy której partii importu nastąpiła zmiana,
- kiedy ta zmiana została zapisana.

Jest to więc historia ewolucji pól wymiaru, a nie pełna migawka całego rekordu po każdym przebiegu procesu. System nie zapisuje kompletnej wersji encji po każdym imporcie.

15.10. Ograniczenia historii lineage i zmian

Najważniejsze ograniczenia są następujące:

- wpis lineage dla danego atrybutu reprezentuje aktualne pochodzenie, a nie pełną sekwencję wcześniejszych

źródeł tego samego atrybutu,

- przy ponownym zapisie lineage dla tego samego obiektu i atrybutu wcześniejszy wpis jest zastępowany nowym,
- EntityChangeLog przechowuje zmianę wartości, ale nie przechowuje pełnej logiki wyboru wszystkich kandydatów rozważanych w survivorship,
- aktualna warstwa zapisuje historię zmian bezpośrednio dla osób i podmiotów; model dopuszcza więcej typów encji, lecz nie wszystkie są obecnie materializowane w analogicznym zakresie,
- logi techniczne procesu nie są pełnym dziennikiem wszystkich decyzji matchingowych.

W praktyce oznacza to, że system dobrze pokazuje:

- skąd pochodzi aktualna wartość,
- jak zmieniała się wartość atrybutu,
- kiedy i w jakiej partii zaszła zmiana.

Nie pozwala natomiast odtworzyć kompletnego, historycznego obrazu wszystkich wcześniejszych wariantów lineage dla tego samego atrybutu.

15.11. Przykłady informacji możliwych do odtworzenia

Na podstawie aktualnych struktur można ustalić między innymi:

- z którego systemu źródłowego pochodzi bieżące imię osoby GOLD,
- która partia importu dostarczyła aktualny numer REGON podmiotu,
- czy zwycięski adres był wybrany dzięki priorytetowi źródła czy dzięki potwierdzeniu w TERYT,
- jakie było poprzednie nazwisko albo poprzednia nazwa podmiotu,
- kiedy utworzono lub zaktualizowano relację osoby z adresem.

Takie informacje są wystarczające do audytu bieżącego wyniku oraz do prześledzenia najważniejszych zmian biznesowych, ale nie zastępują pełnego mechanizmu temporalnego.

15.12. Wykorzystanie przez warstwę serving

Zapisane dane audytowe nie pozostają wyłącznie wewnętrznym artefaktem warstwy GOLD. Zgodnie z zakresem projektu są później udostępniane przez warstwę serving, która potrafi zwrócić między innymi:

- lineage wybranych rekordów GOLD,
- historię zmian,
- wyniki walidacji powiązane z rekordami.

Oznacza to, że audytowalność nie jest wyłącznie cechą bazy danych, ale także częścią interfejsu odczytowego systemu.

15.13. Odniesienia do implementacji

Najważniejsze elementy implementacji znajdują się w plikach:

- `app/layers/ingestion/models.py` - model ProcessLog,
- `app/layers/integration_golden/models.py` - EntityChangeLog oraz tabele lineage,
- `app/layers/integration_golden/repository.py` - zapis i aktualizacja lineage, zapis historii zmian, log GOLDEN_LOAD,
- `app/layers/integration_golden/service.py` - logika wyboru wpisów lineage, zapis zmian i wykorzystanie statusów walidacji,
- `app/layers/validation/models.py` - wyniki walidacji używane do budowy jakości pola,
- `tests/test_integration_golden_dimensions.py` - testy lineage, zmian atrybutów i relacji adresowych,
- `tests/test_integration_golden_load.py` - testy logowania GOLDEN_LOAD,

- `scripts/init_proposed_mssql_schema.sql` - definicje tabel i indeksów dla `ProcessLog`, `EntityChangeLog` i tabel `lineage`.

16. REST API

Interfejs REST jest głównym sposobem uruchamiania kolejnych warstw systemu oraz odczytu wyników procesu. API zostało zbudowane w FastAPI i obejmuje dwa obszary:

1. techniczne endpointy pomocnicze poza głównym pipeline'em,
2. właściwe endpointy warstwowe pod wspólnym prefiksem `/layers`.

Taki podział rozdziela funkcje demonstracyjne od właściwego procesu goldenizacji i ułatwia późniejsze powiązanie operacji z poszczególnymi warstwami kodu.

16.1. Organizacja aplikacji

Punktem wejścia jest `app/main.py`. Aplikacja:

- uruchamia inicjalizację bazy przy starcie,
- konfiguruje middleware CORS,
- dołącza router techniczny `app.api.routes`,
- dołącza router warstwowy `app.layers.router`.

Router warstwowy posiada prefiks:

`/layers`

W jego ramach podłączone są routery:

- `ingestion`,
- `staging_validation`,
- `preprocessing`,
- `validation`,
- `integration_golden`,
- `analytics`,
- `serving`.

16.2. Styl komunikacji API

API używa dwóch głównych sposobów przekazywania danych:

- formularzy `multipart/form-data` albo `application/x-www-form-urlencoded` dla operacji uruchamiających pipeline,
- parametrów zapytania dla endpointów odczytowych warstwy `serving`.

W praktyce oznacza to, że:

- endpointy importu plików i ładowania warstw przyjmują parametry przez `Form` i `File`,
- endpointy `list`, wyszukiwania, historii i porównań korzystają z `Query`.

Odpowiedzi są zwracane jako JSON zgodny z modelami Pydantic z plików `schemas.py`.

16.3. Endpointy techniczne poza `/layers`

Router `app.api.routes` zawiera pomocnicze trasy demonstracyjne:

```
GET /health
POST /documents
GET /documents
GET /documents/search
POST /files/line-count
```

Ich rola jest pomocnicza:

- `GET /health` służy do prostego sprawdzenia dostępności API,
- endpointy `/documents` pokazują demonstracyjną integrację SQL i Neo4j,

- POST /files/line-count jest niezależnym testowym endpointem uploadu pliku CSV.

Endpointy te nie należą do głównego pipeline'u goldenizacji i nie powinny być traktowane jako interfejs biznesowy systemu.

16.4. Endpointy statusowe warstw

Każda główna warstwa posiada prosty endpoint:

GET /layers/<warstwa>/status

Dotyczy to warstw:

- ingestion,
- staging_validation,
- preprocessing,
- validation,
- integration_golden,
- analytics,
- serving.

Odpowiedź ma jednolity schemat LayerStatus i zawiera nazwę warstwy oraz status ready. Endpointy te służą głównie do szybkiej diagnostyki i weryfikacji dostępności modułu.

16.5. Endpointy sterujące pipeline'em

Główna ścieżka przetwarzania jest udostępniona przez endpointy POST pod /layers. Najważniejsze z nich to:

```
POST /layers/ingestion/raw-load
POST /layers/ingestion/relational-load
POST /layers/staging_validation/staging-load
POST /layers/preprocessing/preprocessing-load
POST /layers/validation/teryt-load
POST /layers/validation/validation-load
POST /layers/integration_golden/match-candidates
POST /layers/integration_golden/match-candidates/jaro-winkler
POST /layers/integration_golden/match-groups
POST /layers/integration_golden/golden-load
```

Są to endpointy uruchamiające logikę kolejnych warstw, a nie tylko zapisujące prosty rekord w bazie. Każde wywołanie kończy się uruchomieniem serwisu i zwróceniem podsumowania wykonania.

Ingestion

Warstwa ingestion udostępnia:

- GET /layers/ingestion/relational-queries - listę dozwolonych zapytań relacyjnych,
- POST /layers/ingestion/raw-load - import pliku do warstwy RAW,
- POST /layers/ingestion/relational-load - import danych z Oracle do RAW.

Import plikowy przyjmuje:

- file,
- source_system_code,
- opcjonalne created_by.

Import relacyjny przyjmuje:

- source_system_code,
- query_name,
- opcjonalne entity_type,
- opcjonalne created_by.

Staging i preprocessing

Warstwy pośrednie używają prostszych formularzy:

- POST /layers/staging_validation/staging-load,
- POST /layers/preprocessing/preprocessing-load.

Obie operacje przyjmują:

- raw_file_id,
- entity_type.

Validation

Warstwa validation udostępnia:

- POST /layers/validation/teryt-load,
- POST /layers/validation/validation-load.

teryt-load przyjmuje dwa pliki:

- SIMC.csv,
- ULIC.csv.

validation-load przyjmuje:

- raw_file_id,
- entity_type,
- check_email_dns.

Integration Golden

Warstwa integration_golden udostępnia:

- POST /layers/integration_golden/match-candidates,
- POST /layers/integration_golden/match-candidates/jaro-winkler,
- POST /layers/integration_golden/match-groups,
- POST /layers/integration_golden/golden-load.

Etapy matchingu przyjmują:

- entity_type,
- opcjonalny raw_file_id,
- parametr proggu min_score,
- a dla Levenshteina także max_pairs.

match-groups przyjmuje wyłącznie entity_type, ponieważ grupowanie działa dla całego typu encji.

golden-load przyjmuje:

- entity_type,
- opcjonalny raw_file_id,
- opcjonalny entity_group_id.

16.6. Endpointy odczytowe warstwy serving

Warstwa serving udostępnia interfejs odczytowy wyników procesu. Wszystkie jej endpointy są oparte o GET i modele Pydantic ze schematami paginacji, list i szczegółów.

Najważniejsze trasy to:

```
GET /layers/serving/golden-records
GET /layers/serving/persons/{person_id}
GET /layers/serving/persons/search/by-pesel
GET /layers/serving/parties/search
```

```
GET /layers/serving/parties/{party_id}
GET /layers/serving/lineage/{entity_type}/{record_id}
GET /layers/serving/history/{entity_type}/{record_id}
GET /layers/serving/validation-results
GET /layers/serving/match-results/levenshtein
GET /layers/serving/match-results/jaro-winkler
GET /layers/serving/match-results/comparison
GET /layers/serving/counts
```

Zakres funkcjonalny tych endpointów obejmuje:

- listę rekordów GOLD,
- szczegóły osoby i podmiotu,
- wyszukiwanie po identyfikatorach i nazwie,
- lineage,
- historię zmian,
- listę wyników walidacji,
- listę kandydatów matchingu,
- porównanie dwóch rekordów preprocessed,
- liczniki etapów procesu.

16.7. Typowe odpowiedzi warstwy serving

Warstwa serving korzysta z kilku powtarzalnych wzorców odpowiedzi.

Listy paginowane

Listy takie jak:

- golden-records,
- validation-results,
- match-results/levenshtein,
- match-results/jaro-winkler

zwracają:

- items - właściwe rekordy,
- page - metadane paginacji (limit, offset, total).

Szczegóły rekordu

Endpointy szczegółów osoby i podmiotu zwracają:

- podstawowe atrybuty wymiaru,
- listy adresów,
- dla podmiotu także listę identyfikatorów.

Lineage i historia zmian

Endpointy:

- /lineage/{entity_type}/{record_id},
- /history/{entity_type}/{record_id}

zwracają:

- identyfikację encji,
- listę wpisów lineage albo historii zmian.

Porównanie dwóch rekordów

Endpoint `/match-results/comparison` zwraca:

- `entity_type`,
- `left_preprocessed_id`,
- `right_preprocessed_id`,
- szczegóły kandydata Levenshteina,
- szczegóły kandydata Jaro-Winklera,
- dwa rekordy źródłowe w formie słowników.

Jest to endpoint szczególnie istotny dla frontendu, ponieważ zasilą modal porównania par rekordów.

16.8. Konwencja błędów

API rozróżnia dwa podstawowe typy błędów:

- błędy wejściowe zwracane jako HTTP 400,
- błędy nieoczekiwane zwracane jako HTTP 500.

W warstwie `serving` pojawiają się również odpowiedzi HTTP 404, gdy żądany rekord GOLD nie istnieje.

Typowy wzorzec jest następujący:

- wyjątki domenowe i walidacyjne są mapowane na `HTTPException(status_code=400, detail=...)`,
- brak rekordu szczegółowego jest mapowany na 404,
- w wielu endpointach pipeline'u nieobsłużony wyjątek jest opakowany w 500 z nazwą kroku, natomiast część endpointów odczytowych `serving` pozostawia nieobsłużone wyjątki do domyślnej obsługi FastAPI.

Przykłady:

- błędny typ encji,
- brak danych wejściowych do kroku,
- przekroczenie limitu par matchingu,
- nieistniejący `RawFile_ID`,
- nieistniejący `Person_ID` albo `Party_ID`.

16.9. CORS i współpraca z frontendem

W `app/main.py` skonfigurowano middleware CORS z listą źródeł pobieraną z konfiguracji. Dzięki temu frontend działający na innym porcie może wywoływać API lokalnie bez dodatkowego proxy.

Ma to bezpośrednie znaczenie dla frontendu React, który pobiera dane z:

- `/layers/serving/validation-results`,
- `/layers/serving/match-results/levenshtein`,
- `/layers/serving/match-results/jaro-winkler`,
- `/layers/serving/match-results/comparison`.

16.10. Swagger i OpenAPI

Ponieważ API jest zbudowane w FastAPI, dokumentacja OpenAPI i interfejs Swagger są generowane automatycznie. Pozwala to:

- przeglądać dostępne endpointy,
- sprawdzać parametry i modele odpowiedzi,
- uruchamiać wywołania testowe bez zewnętrznego klienta.

Swagger jest szczególnie przydatny przy ręcznym uruchamianiu etapów pipeline'u niezależnie od Airflow.

16.11. Zakres i ograniczenia API

Najważniejsze ograniczenia obecnej wersji są następujące:

- API nie implementuje uwierzytelniania i autoryzacji,
- endpointy warstwowe są synchroniczne z perspektywy klienta HTTP,
- brak mechanizmu PUSH i webhooków dla zmian danych,
- brak endpointów manualnej obsługi decyzji REVIEW,
- techniczne endpointy demonstracyjne współistnieją z głównym API, ale nie są częścią właściwego pipeline'u.

Warstwa serving jest odczytowa. Nie służy do edycji Golden Record ani do ręcznego zatwierdzania decyzji integracyjnych.

16.12. Odniesienia do implementacji

Najważniejsze elementy implementacji znajdują się w plikach:

- `app/main.py` - inicjalizacja aplikacji, CORS i podłączenie routerów,
- `app/api/routes.py` - techniczne endpointy demonstracyjne,
- `app/layers/router.py` - wspólny router warstw pod `/layers`,
- `app/layers/ingestion/api.py` - import plików i źródeł relacyjnych,
- `app/layers/staging_validation/api.py` - ładowanie stagingu,
- `app/layers/preprocessing/api.py` - preprocessing,
- `app/layers/validation/api.py` - TERYT i walidacja,
- `app/layers/integration_golden/api.py` - matching, grupowanie i ładowanie GOLD,
- `app/layers/serving/api.py` - odczyt wyników procesu,
- `app/layers/serving/schemas.py` - modele odpowiedzi list, szczegółów, lineage i matchingu,
- `tests/test_serving_api.py` - testy endpointów warstwy serving,
- `tests/test_health.py` - test prostego endpointu zdrowia aplikacji.

17. Interfejs użytkownika

Frontend jest osobną aplikacją React, która komunikuje się z backendem przez HTTP i korzysta z odczytowych endpointów warstwy serving. W aktualnej wersji nie służy do uruchamiania pipeline'u ani do ręcznego zatwierdzania decyzji integracyjnych. Jego zadaniem jest prezentacja wybranych wyników procesu w czytelnej formie tabelarycznej.

W praktyce frontend pełni rolę lekkiej konsoli odczytowej dla dwóch obszarów:

1. wyników walidacji,
2. wyników matchingu.

17.1. Technologia i sposób uruchomienia

Interfejs został wykonany w:

- React 18,
- Vite,
- JavaScript z modułami ES.

Skrypt dev uruchamia środowisko developerskie Vite, build przygotowuje produkcyjny bundle, a preview wystawia zbudowaną aplikację na porcie 4173.

Kod wejściowy znajduje się w:

- frontend/src/main.jsx,
- frontend/src/App.jsx.

W main.jsx aplikacja jest montowana przez ReactDOM.createRoot(...), a style ładowane są z frontend/src/styles/index.css.

17.2. Struktura aplikacji

W kodzie widoczny jest prosty podział na warstwy frontendowe:

- api/ - komunikacja z backendem,
- features/ - widoki funkcjonalne,
- components/ui/ - małe komponenty wielokrotnego użytku,
- constants/ - stałe widoków i filtrowania,
- utils/ - formatowanie i funkcje pomocnicze,
- styles/ - podział arkuszy CSS według obszarów interfejsu.

Aktualna struktura nie korzysta z globalnego store ani routingu URL. Nawigacja pomiędzy widokami jest obsługiwana lokalnym stanem komponentu App.

17.3. Komunikacja z backendem

Komunikacja z API została wydzielona do pliku:

- frontend/src/api/serving.js.

Warstwa ta:

- pobiera bazowy adres API z VITE_API_URL,
- domyślnie używa http://localhost:8000,
- buduje parametry zapytania na podstawie obiektu filtrów,
- wykonuje fetch,
- próbuje odczytać detail albo message z odpowiedzi błędu,
- zwraca zdeserializowany JSON.

Frontend korzysta obecnie z trzech funkcji:

- getValidationResults(...),

- `getMatchResults(...)`,
- `getMatchComparison(...)`.

Oznacza to, że interfejs jest ściśle związany z odczytowymi endpointami `serving`, a nie z operacjami zapisu czy wywołaniami `pipeline'u`.

17.4. Główny układ aplikacji

Komponent `App.jsx` definiuje wspólny szkielet interfejsu:

- sekcję nagłówkową z tytułem aktywnego widoku,
- prezentację aktualnego adresu API,
- przycisk odświeżenia danych,
- przełącznik widoku Walidacja / Matching,
- obszar roboczy z aktywnym komponentem funkcjonalnym.

Stan główny obejmuje:

- `activeView` - wybór aktualnego widoku,
- `refreshToken` - prosty licznik wymuszający ponowne pobranie danych.

Frontend nie utrzymuje złożonego drzewa zależności pomiędzy widokami. Każdy widok sam odpowiada za pobranie i prezentację swoich danych.

17.5. Widok walidacji

Widok walidacji został zaimplementowany w:

- `frontend/src/features/validation/ValidationView.jsx`.

Komponent pobiera dane z endpointu:

`GET /layers/serving/validation-results`

Obsługiwane filtry obejmują:

- `entity_type`,
- `source_system_code`,
- `rule_code`,
- `status`,
- `severity`.

Widok udostępnia:

- formularz filtrów,
- tabelę wyników,
- obsługę paginacji,
- komunikat ładowania,
- komunikat błędu,
- komunikat pustego wyniku.

W tabeli prezentowane są między innymi:

- identyfikator wyniku walidacji,
- źródło,
- typ encji,
- kod reguły,
- pole,
- status,

- poziom istotności,
- sprawdzana wartość,
- komunikat,
- data utworzenia.

Dodatkowo komponent `ruleHighlight.js` nadaje części reguł wyróżnienia wizualne. Nie zmienia to logiki danych, ale poprawia czytelność najważniejszych problemów walidacyjnych.

17.6. Widok matchingu

Widok matchingu znajduje się w:

- `frontend/src/features/matching/MatchingView.jsx`.

Komponent może pracować w dwóch trybach:

- levenshtein,
- jaro-winkler.

Zmiana trybu powoduje przełączenie zestawu danych i ponowne pobranie listy kandydatów z endpointów:

```
GET /layers/serving/match-results/levenshtein
GET /layers/serving/match-results/jaro-winkler
```

Widok udostępnia:

- przełącznik algorytmu,
- tabelę kandydatów,
- paginację,
- obsługę błędów,
- przycisk Porównaj dla każdej pary.

W tabeli matchingu prezentowane są między innymi:

- identyfikator kandydata,
- typ encji,
- wynik liczbowy,
- decyzja,
- silne pola,
- pola konfliktowe,
- informacja o przejściu do drugiego sita,
- pola tekstowe,
- para `left_preprocessed_id / right_preprocessed_id`,
- data utworzenia.

Interfejs pokazuje więc nie tylko samą decyzję, ale także podstawowe przesłanki porównania.

17.7. Porównanie pary rekordów

Szczegółowe porównanie dwóch rekordów zostało wydzielone do komponentu:

- `frontend/src/features/matching/MatchingComparisonPanel.jsx`.

Po kliknięciu Porównaj frontend wywołuje:

```
GET /layers/serving/match-results/comparison
```

z parametrami:

- `entity_type`,

- `left_preprocessed_id`,
- `right_preprocessed_id`.

W odpowiedzi otrzymuje:

- szczegóły kandydata Levenshteina,
- szczegóły kandydata Jaro-Winklera,
- dwa rekordy źródłowe jako słowniki pól.

Modal porównania:

- zestawia lewą i prawą wartość tego samego pola,
- oznacza zgodność albo różnicę,
- rozpoznaje pola silne, konfliktowe i tekstowe,
- pozwala zamknąć okno kliknięciem tła albo klawiszem Escape.

Jest to obecnie najbardziej analityczny element frontendu, ponieważ pozwala przejść od listy kandydatów do porównania atrybut po atrybucie.

17.8. Komponenty wspólne

Na potrzeby wielu widoków wydzielono dwa małe komponenty interfejsu:

- `Pager.jsx`,
- `StatusBadge.jsx`.

`Pager` obsługuje ruch po stronach wyników i pokazuje zakres aktualnie prezentowanych rekordów.

`StatusBadge` nadaje wspólny wygląd statusom takim jak:

- PASS,
- ERROR,
- AUTO_MERGE,
- REVIEW,
- TAK,
- NIE.

Dzięki temu oba główne widoki zachowują wspólną konwencję wizualną.

17.9. Formatowanie i stałe

Widoki korzystają z warstwy pomocniczej:

- `frontend/src/utils/formatters.js`,
- `frontend/src/utils/matching.js`.

Funkcje te odpowiadają za:

- formatowanie dat,
- bezpieczne wyświetlanie pustych wartości,
- dobór wariantu wizualnego znacznika statusu,
- prezentację wyniku punktowego matchingu.

Stałe pomocnicze znajdują się w:

- `frontend/src/constants/validation.js`,
- `frontend/src/constants/matching.js`,
- `frontend/src/constants/validationFilters.js`.

Zawierają one między innymi:

- limity stron,

- puste struktury danych,
- listy opcji filtrów,
- listy algorytmów dostępnych w widoku matchingu.

17.10. Warstwa stylów

Style zostały podzielone na kilka plików:

- `styles/base.css`,
- `styles/layout.css`,
- `styles/components.css`,
- `styles/views.css`,
- `styles/index.css`.

Podział jest funkcjonalny:

- `base` definiuje podstawy typografii i zmiennych,
- `layout` opisuje główny układ aplikacji,
- `components` obejmuje wspólne elementy interfejsu,
- `views` zawiera klasy specyficzne dla widoków,
- `index.css` spina całość.

Takie rozdzielenie wystarcza dla obecnej skali aplikacji i ogranicza konieczność utrzymywania jednego dużego arkusza stylów.

17.11. Zakres funkcjonalny obecnej wersji

Aktualnie frontend realizuje:

- odczyt wyników walidacji,
- filtrowanie wyników walidacji,
- paginację,
- odczyt kandydatów matchingu dla dwóch algorytmów,
- porównanie szczegółów dwóch rekordów,
- ręczne odświeżenie danych.

Nie realizuje natomiast:

- widoku list i szczegółów Golden Record,
- wyszukiwania osób i podmiotów po endpointach szczegółowych `servicing`,
- prezentacji lineage i historii zmian,
- uruchamiania etapów pipeline'u,
- ręcznej obsługi przypadków REVIEW,
- edycji danych.

Jest to więc interfejs odczytowy dla wybranych obszarów systemu, a nie pełny panel operacyjny goldenizacji.

17.12. Miejsce frontendu w architekturze

Frontend nie komunikuje się bezpośrednio z SQL Serverem, Airflow ani Oracle. Jego jedynym punktem dostępu do danych jest REST API backendu.

Z architektonicznego punktu widzenia oznacza to, że:

- wszystkie dane prezentowane użytkownikowi przechodzą przez warstwę `servicing`,
- frontend pozostaje cienką warstwą prezentacji,
- reguły biznesowe, walidacyjne i matchingowe pozostają po stronie backendu.

To podejście upraszcza spójność systemu i pozwala rozwijać interfejs bez przenoszenia logiki domenowej do przeglądarki.

17.13. Ograniczenia

Najważniejsze ograniczenia obecnego frontendu to:

- brak routingu URL i głębszej nawigacji pomiędzy ekranami,
- brak mechanizmu uwierzytelniania użytkownika,
- brak funkcji zapisu lub akcji na pipeline,
- brak widoków Golden Record, lineage i historii zmian,
- brak interfejsu do ręcznej pracy z decyzjami REVIEW,
- zależność od dostępności backendu i poprawnej konfiguracji VITE_API_URL.

Nie są to braki przypadkowe. Frontend w aktualnej wersji został zawężony do czytelnego podglądu danych servingowych dla walidacji i matchingu.

17.14. Odniesienia do implementacji

Najważniejsze elementy implementacji znajdują się w plikach:

- `frontend/src/main.jsx` - punkt wejścia aplikacji,
- `frontend/src/App.jsx` - główny układ i przełączanie widoków,
- `frontend/src/api/serving.js` - komunikacja z backendem,
- `frontend/src/features/validation/ValidationView.jsx` - widok wyników walidacji,
- `frontend/src/features/validation/ruleHighlight.js` - wyróżnianie wybranych reguł,
- `frontend/src/features/matching/MatchingView.jsx` - widok kandydatów matchingu,
- `frontend/src/features/matching/MatchingComparisonPanel.jsx` - modal porównania rekordów,
- `frontend/src/components/ui/Pager.jsx` - paginacja,
- `frontend/src/components/ui/StatusBadge.jsx` - wspólny komponent statusów,
- `frontend/src/utils/formatters.js` - formatowanie wartości i dat,
- `frontend/src/utils/matching.js` - prezentacja wyników matchingu,
- `frontend/src/constants/validation.js` i `frontend/src/constants/matching.js` - limity i puste struktury danych,
- `frontend/src/styles/` - style aplikacji,
- `frontend/package.json` - konfiguracja zależności i skryptów uruchomieniowych.

18. Środowisko uruchomieniowe, konfiguracja i wdrożenie

Projekt został przygotowany przede wszystkim do uruchamiania lokalnego w środowisku kontenerowym.

Repozytorium zawiera również zestaw manifestów OpenShift, jednak mają one charakter przygotowanej podstawy wdrożeniowej, a nie potwierdzonego odwzorowania środowiska lokalnego.

W praktyce należy rozróżnić dwa poziomy:

1. środowisko lokalne Docker Compose, które stanowi główną i rzeczywiście używaną ścieżkę uruchomienia,
2. środowisko OpenShift, dla którego przygotowano manifesty wymagające dostosowania i weryfikacji.

18.1. Środowisko lokalne Docker Compose

Główny plik uruchomieniowy to:

- `docker-compose.yml`.

Definiuje on sześć usług:

Usługa	Rola	Port lokalny
api	FastAPI i logika pipeline'u	8000
frontend	React/Vite	5173
airflow	orkiestracja DAG	8080
mssql	centralne repozytorium danych	1433
oracle	demonstracyjne źródło relacyjne	1521
neo4j	demonstracyjna baza grafowa	7474, 7687

Środowisko to jest developerskie. API i frontend działają w trybie umożliwiającym szybkie przeładowanie zmian, a Airflow uruchamiany jest w uproszczonej konfiguracji z jednym kontenerem i SequentialExecutor.

18.2. Kontenery i obrazy

Backend API

Backend budowany jest z:

- Dockerfile.

Obraz bazuje na:

- `python:3.14-slim-bookworm`.

Podczas budowy:

- instalowany jest ODBC Driver 18 dla SQL Servera,
- instalowane są zależności z `requirements.txt`,
- kopiowany jest katalog `app/`.

W Compose backend uruchamiany jest poleceniem:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload --reload-dir /workspace/app
```

Oznacza to, że lokalny kontener API pracuje w trybie developerskim z hot reloadem dla kodu backendu.

Frontend

W środowisku lokalnym frontend korzysta z gotowego obrazu:

- `node:20-alpine`.

Zamiast własnego obrazu developerskiego używany jest montowany katalog `./frontend` oraz polecenie:

```
npm install && npm run dev -- --host 0.0.0.0 --port 5173
```

Dzięki temu zmiany w kodzie React są od razu widoczne bez ręcznej odbudowy obrazu.

Równolegle repozytorium zawiera:

- frontend/Dockerfile

do budowy wersji statycznej. Ten obraz:

1. wykonuje npm ci,
2. buduje aplikację przez vite build,
3. przekazuje wynik do nginxinc/nginx-unprivileged:stable-alpine.

Jest to wariant przygotowany pod wdrożenie, a nie tryb używany lokalnie podczas rozwoju.

Airflow

Airflow budowany jest z:

- airflow/Dockerfile.

Obraz bazuje na:

- apache/airflow:3.2.2-python3.14.

Rozszerzenie obejmuje instalację:

- apache-airflow-providers-fab==3.6.5.

Uruchomienie odbywa się przez:

- airflow/start-airflow.sh.

Skrypt wykonuje:

1. airflow db migrate,
2. airflow fab-db migrate,
3. utworzenie albo reset hasła administratora,
4. start procesów api-server, scheduler, dag-processor i triggerer.

W środowisku lokalnym wszystkie te procesy działają w jednym kontenerze.

Bazy danych

Lokalne środowisko korzysta z gotowych obrazów:

- mcr.microsoft.com/mssql/server:2022-latest,
- gvenzl/oracle-free:23-slim-faststart,
- neo4j:5.24.

SQL Server jest głównym repozytorium procesu.

Oracle służy jako źródło demonstracyjne dla importu relacyjnego.

Neo4j pozostaje komponentem demonstracyjnym i nie uczestniczy w głównym pipeline.

18.3. Wolumeny i trwałość danych

Docker Compose definiuje wolumeny:

- mssql_data,
- oracle_data,
- neo4j_data,
- neo4j_logs,
- filestream_data,
- frontend_node_modules,
- airflow_db.

Ich rola jest następująca:

- trwałość danych SQL Servera i Oracle,
- trwałość danych i logów Neo4j,

- przechowywanie plików pomocniczych pod `FILESTREAM_PATH`,
- zachowanie zależności frontendu,
- zachowanie metadanych i stanu Airflow.

Dodatkowo montowane są katalogi robocze z repozytorium:

- `./app` do kontenera API,
- `./frontend` do kontenera frontendu,
- `./airflow/dags`, `./airflow/logs`, `./airflow/plugins`, `./airflow/config` do kontenera Airflow,
- `./data` do Airflow tylko do odczytu.

18.4. Zależności startowe i healthchecki

Usługa api oczekuje na zdrowy stan:

- mssql,
- oracle,
- neo4j.

Sprawdzanie gotowości realizowane jest przez healthchecki:

- SQL Server: zapytanie `SELECT 1` przez `sqlcmd`,
- Oracle: skrypt `healthcheck.sh`,
- Neo4j: żądanie HTTP do portu 7474.

Airflow zależy od API, ponieważ wykonuje wywołania HTTP do warstw backendu.

Frontend zależy od API, ponieważ pobiera dane z endpointów `serve`.

18.5. Konfiguracja backendu

Konfiguracja backendu jest zdefiniowana w:

- `app/core/config.py`.

Wartości są ładowane z:

- pliku `.env`,
- zmiennych środowiskowych kontenera.

Najważniejsze grupy parametrów obejmują:

- nazwę i port aplikacji,
- parametry połączenia z SQL Serverem,
- parametry połączenia z Neo4j,
- ścieżkę `FILESTREAM_PATH`,
- listę `CORS_ORIGINS`,
- parametry Oracle.

Przykładowe kluczowe zmienne:

- `MSSQL_PASSWORD`,
- `NEO4J_PASSWORD`,
- `CORS_ORIGINS`,
- `ORACLE_HOST`,
- `ORACLE_PORT`,
- `ORACLE_SERVICE_NAME`.

Hasła baz danych są wymagane z `.env` albo środowiska uruchomieniowego. Nie powinny być zapisywane w dokumentacji ani publikowane w repozytorium jako wartości produkcyjne.

18.6. Konfiguracja frontendu

Frontend korzysta z jednej najważniejszej zmiennej środowiskowej:

- VITE_API_URL.

W Docker Compose ustawiana jest ona na:

`http://localhost:8000`

Oznacza to, że aplikacja React kieruje odczyt danych do lokalnego backendu FastAPI. Jeżeli adres API się zmienia, frontend wymaga odpowiedniej aktualizacji tej wartości przy budowie albo uruchomieniu.

18.7. Konfiguracja Airflow

W Compose Airflow otrzymuje między innymi:

- AIRFLOW_HOME=/opt/airflow,
- AIRFLOW__CORE__LOAD_EXAMPLES=False,
- AIRFLOW__CORE__EXECUTOR=SequentialExecutor,
- AIRFLOW__CORE__AUTH_MANAGER=airflow.providers.fab.auth_manager.fab_auth_manager.FabAuthManager,
- AIRFLOW__API__SECRET_KEY,
- AIRFLOW__API__AUTH__JWT_SECRET,
- AIRFLOW_USERNAME,
- AIRFLOW_PASSWORD.

Konfiguracja ta oznacza, że:

- Airflow działa lokalnie bez rozproszonego executora,
- interfejs webowy jest chroniony prostym logowaniem FAB,
- konto administratora jest zakładane albo aktualizowane podczas startu.

Domyślne wartości developerskie dla logowania to admin/admin, jeśli nie zostaną zastąpione przez zmienne środowiskowe.

18.8. Lokalne uruchomienie

Podstawowe uruchomienie developerskie odbywa się przez:

```
docker compose up -d
```

Zatrzymanie środowiska:

```
docker compose down
```

Pełny reset wolumenów:

```
docker compose down -v
```

Najważniejsze lokalne adresy dostępu:

- Swagger API: `http://localhost:8000/docs`,
- frontend React: `http://localhost:5173`,
- Airflow UI: `http://localhost:8080`,
- Neo4j Browser: `http://localhost:7474`.

18.9. Inicjalizacja baz danych

SQL Server korzysta z przygotowanego skryptu:

- `scripts/init_proposed_mssql_schema.sql`.

Skrypt:

- tworzy bazę goldenizacja,
- zakłada schematy meta, raw, stg, gold,
- tworzy tabele procesu, stagingu i GOLD,
- jest przygotowany jako idempotentny.

Oracle korzysta ze skryptu:

- `scripts/init_oracle_insurance_core.sql`

zamontowanego do kontenera podczas startu. Służy on do zainicjalizowania demonstracyjnego źródła relacyjnego.

Backend przy starcie wykonuje dodatkowo własną inicjalizację bazy przez `init_db()`, aby kontener API był gotowy bez osobnego ręcznego przygotowania struktur ORM.

18.10. OpenShift

Repozytorium zawiera katalog:

- `openshift/`

z manifestami dla:

- ConfigMap,
- Secret,
- PVC,
- SQL Server,
- Neo4j,
- API,
- frontendu,
- Airflow,
- Route,
- ConfigMap z przykładowym DAG.

Najważniejsze pliki to:

- `01-configmap.yaml`,
- `02-secrets.yaml`,
- `03-pvc.yaml`,
- `04-mssql.yaml`,
- `05-neo4j.yaml`,
- `06-api.yaml`,
- `08-routes.yaml`,
- `09-frontend.yaml`,
- `10-airflow.yaml`,
- `11-airflow-dags-configmap.yaml`.

Manifesty zakładają użycie obrazów w wewnętrznym rejestrze OpenShift i wymagają podmiany placeholderów takich jak:

- `REPLACE_WITH_PROJECT`,
- `REPLACE_API_ROUTE`,
- `REPLACE_FRONTEND_ROUTE`.

18.11. Różnice między Docker Compose i OpenShift

Najważniejsze rozbieżności są następujące:

- OpenShift nie obejmuje Oracle, mimo że środowisko lokalne go używa,
- konfiguracja Airflow w OpenShift montuje jedynie przykładowy DAG `goldenizacja_example`,
- manifesty nie odzwierciedlają pełnej struktury lokalnych wolumenów developerskich,
- frontend w OpenShift jest wdrażany jako zbudowany obraz statyczny, a lokalnie działa w trybie Vite dev,
- OpenShift wymaga wcześniejszego zbudowania i opublikowania obrazów API, frontendu i Airflow.

Z tego powodu środowiska OpenShift nie można traktować jako gotowego, przetestowanego odpowiednika Compose.

18.12. Bezpieczeństwo konfiguracji

W repozytorium rozdzielono konfigurację jawną od sekretów:

- ConfigMap przechowuje ustawienia niesekretne,
- Secret przechowuje hasła i klucze.

Dotyczy to zwłaszcza:

- hasła SQL Server,
- hasła Neo4j,
- loginu i hasła Airflow,
- sekretów API i JWT Airflow.

Samo rozdzielenie konfiguracji nie oznacza jednak pełnego zabezpieczenia systemu. API aplikacji nadal nie implementuje własnego mechanizmu uwierzytelniania i autoryzacji.

18.13. Ograniczenia środowiskowe

Najważniejsze ograniczenia obecnego sposobu uruchomienia to:

- Docker Compose jest środowiskiem developerskim, a nie produkcyjnym,
- Airflow lokalnie działa w jednym kontenerze i z SequentialExecutor,
- Neo4j jest obecny w środowisku, ale nie jest częścią głównego pipeline'u,
- OpenShift nie został przetestowany na docelowym klastrze,
- manifesty OpenShift nie obejmują Oracle,
- ConfigMap Airflow w OpenShift zawiera przykładowy DAG zamiast właściwego pipeline'u.

Ograniczenia te nie uniemożliwiają działania lokalnego rdzenia systemu, ale mają znaczenie przy ocenie gotowości rozwiązania do wdrożenia poza środowiskiem demonstracyjnym.

18.14. Odniesienia do implementacji

Najważniejsze elementy implementacji znajdują się w plikach:

- `docker-compose.yml` - lokalne środowisko usług,
- `README.md` - podstawowe instrukcje uruchomienia,
- `Dockerfile` - obraz backendu API,
- `frontend/Dockerfile` - obraz wdrożeniowy frontendu,
- `airflow/Dockerfile` - obraz Airflow,
- `airflow/start-airflow.sh` - inicjalizacja i uruchomienie Airflow,
- `app/core/config.py` - konfiguracja aplikacji ze zmiennych środowiskowych,
- `openshift/README.md` - opis wdrożenia na OpenShift,
- `openshift/01-configmap.yaml` i `openshift/02-secrets.yaml` - konfiguracja i sekrety,

- openshift/06-api.yaml, openshift/09-frontend.yaml, openshift/10-airflow.yaml - główne deploymenty usług,
- openshift/11-airflow-dags-configmap.yaml - przykładowy DAG Airflow,
- scripts/init_proposed_mssql_schema.sql - inicjalizacja schematu SQL Server,
- scripts/init_oracle_insurance_core.sql - inicjalizacja demonstracyjnego źródła Oracle.

19. Weryfikacja rozwiązania

Weryfikacja rozwiązania została oparta przede wszystkim na testach automatycznych backendu oraz na uruchomieniu lokalnego środowiska Docker Compose.

19.1. Zakres przyjętej weryfikacji

W aktualnej wersji projektu weryfikacja obejmuje trzy poziomy:

1. testy jednostkowe i komponentowe dla logiki backendowej,
2. testy endpointów odczytowych i wybranych endpointów pipeline'u,
3. lokalne uruchomienie środowiska developerskiego z API, bazami danych, frontendem i Airflow.

Najszersze pokrycie testowe dotyczy warstw:

- staging_validation,
- preprocessing,
- validation,
- integration_golden,
- serving.

19.2. Testy automatyczne backendu

Katalog tests/ zawiera zestaw testów obejmujących kluczowe elementy przetwarzania danych.

Import i RAW

Pliki:

- tests/test_relational_ingestion.py,
- tests/test_file_line_count.py.

Sprawdzane są między innymi:

- ekstrakcja danych relacyjnych i budowa snapshotu JSON,
- dobór zapytań dla PERSON i PARTY,
- walidacja poprawności parametrów wejściowych,
- pomocniczy endpoint liczenia wierszy CSV.

Mapowanie do stagingu

Plik:

- tests/test_staging_mapper.py.

Zakres testów obejmuje:

- mapowanie rekordów PERSON i PARTY,
- obsługę różnych struktur wejściowych CSV, JSON i XML,
- budowę Identifiers_JSON,
- normalizację dat i wartości logicznych,
- zachowanie linii adresowych i danych relacyjnych,
- ochronę przed ponownym załadowaniem tego samego RawFile_ID do tego samego obszaru staging.

Preprocessing i standaryzacja

Plik:

- tests/test_preprocessing.py.

Sprawdzane są między innymi:

- normalizacja tekstu, telefonu i adresu e-mail,

- wydzielanie nazwy i formy prawnej podmiotu,
- rozbiecie adresu na pola strukturalne,
- przygotowanie rozszerzonych pól wykorzystywanych później w matchingu.

Walidacja danych

Pliki:

- tests/test_validation.py,
- tests/test_teryt_validation.py.

Testy obejmują:

- sumy kontrolne PESEL, NIP, REGON, LEI i dowodu osobistego,
- zgodność PESEL z datą urodzenia i płcią,
- walidację składni e-mail,
- walidację nazw osobowych,
- reguły zakresów dat dla rekordów PARTY,
- weryfikację miasta i ulicy względem słowników TERYT.

Matching, grupowanie i GOLD

Pliki:

- tests/test_integration_golden_matching.py,
- tests/test_integration_golden_dimensions.py,
- tests/test_integration_golden_load.py,
- tests/test_integration_golden_repository.py.

Zakres obejmuje:

- decyzje AUTO_MERGE, REVIEW, CANDIDATE i NO_MATCH,
- działanie sit Levenshteina i Jaro-Winklera,
- reguły doboru wartości zwycięskiej,
- grupowanie rekordów do EntityGroup,
- zapis rekordów GOLD,
- idempotencję zapisów adresów, tożsamości i lineage,
- zapis historii zmian oraz odrzutów przy niespełnieniu warunków biznesowych,
- poprawność obsługi raw_file_id w golden-load.

Serving i warstwa API

Pliki:

- tests/test_serving_api.py,
- tests/test_health.py.

Sprawdzone są:

- odpowiedzi endpointów odczytowych serving,
- filtrowanie i paginacja,
- szczegóły rekordów PERSON i PARTY,
- endpointy wyników walidacji i matchingu,
- endpoint porównania par rekordów,
- endpoint liczników etapów,

- obecność konfiguracji CORS potrzebnej frontendowi,
- podstawowy endpoint zdrowia /health.

Jakość danych syntetycznych

Plik:

- tests/test_synthetic_data_quality.py.

Testy te nie sprawdzają logiki API, ale kontrolują jakość przygotowanych danych wejściowych, między innymi przez:

- ograniczenie konfliktów identyfikatorów,
- spójność danych osobowych i firmowych,
- zgodność chronologii wybranych dat.

19.3. Weryfikacja lokalnego uruchomienia

Poza testami kod został weryfikowany w lokalnym środowisku kontenerowym opisanym w rozdziale 18.

W praktyce potwierdzony został następujący zakres:

- start backendu FastAPI z inicjalizacją bazy,
- dostępność Swaggera,
- start środowiska Airflow po dostosowaniu obrazu i skryptu startowego do aktualnej wersji,
- rejestracja DAG goldenizacja_pipeline,
- działanie lokalnego frontendu React komunikującego się z endpointami serving,
- wykonanie pipeline'u z użyciem endpointów warstw backendowych.

Oznacza to, że zweryfikowano nie tylko pojedyncze funkcje, ale również podstawowy przepływ lokalny: API -> warstwy przetwarzania -> zapis do SQL Server -> odczyt przez serving i frontend.

19.4. Zakres nieobjęty pełną automatyzacją

Obecna weryfikacja ma charakter głównie backendowy.

W szczególności:

- frontend nie ma osobnego zestawu testów automatycznych w repozytorium,
- testy Airflow nie są prowadzone jako osobna warstwa testów jednostkowych DAG-a, lecz przez lokalne uruchomienie i wykonanie pipeline'u,
- manifesty openshift/ nie stanowią obecnie części potwierdzonego, regularnie testowanego procesu wdrożeniowego.

Nie zmienia to faktu, że rdzeń logiki przetwarzania danych jest objęty testami w kodzie w znacznie większym stopniu niż elementy interfejsu i wdrożenia.

19.5. Odniesienia do implementacji

Najważniejsze pliki związane z weryfikacją rozwiązania:

- tests/test_relational_ingestion.py,
- tests/test_staging_mapper.py,
- tests/test_preprocessing.py,
- tests/test_validation.py,
- tests/test_teryt_validation.py,
- tests/test_integration_golden_matching.py,
- tests/test_integration_golden_dimensions.py,
- tests/test_integration_golden_load.py,
- tests/test_integration_golden_repository.py,

- tests/test_serving_api.py,
- tests/test_health.py,
- tests/test_synthetic_data_quality.py,
- docker-compose.yml,
- airflow/dags/goldenizacja_pipeline.py,
- app/main.py.

20. Podsumowanie realizacji

W ramach projektu wykonano działającą platformę integracji i goldenizacji danych osób oraz podmiotów, opartą na warstwowym przetwarzaniu danych i centralnym repozytorium SQL Server. Rozwiązanie obejmuje import danych z wielu źródeł, zachowanie warstwy RAW, mapowanie do modeli stagingowych, preprocessing, walidację, matching, grupowanie encji oraz budowę i aktualizację rekordów GOLD wraz z lineage i rejestrem zmian.

Istotnym rezultatem projektu jest to, że proces nie kończy się na pojedynczej transformacji danych, ale tworzy spójny przepływ od danych wejściowych do warstwy udostępniania wyników. Backend FastAPI udostępnia endpointy zarówno do uruchamiania etapów pipeline'u, jak i do odczytu wyników przez warstwę serving. Całość może być wykonywana lokalnie przez Docker Compose oraz orkiestratora Apache Airflow, a frontend React pozwala przeglądać wyniki walidacji i matchingu bez bezpośredniej pracy na bazie danych.

Zrealizowana wersja odpowiada więc na główny cel projektu: pokazuje kompletny, technicznie spójny mechanizm dochodzenia od danych heterogenicznych do aktualnego Golden Record z zachowaniem audytowalności procesu. Jednocześnie projekt zachowuje charakter wersji inżynierskiej i demonstracyjnej, a nie zamkniętego produktu wdrożeniowego. Dotyczy to zwłaszcza obszarów takich jak pełne testy środowiska OpenShift, rozbudowany interfejs operatorski, mechanizmy ręcznej obsługi decyzji REVIEW czy pełne zabezpieczenie API.

Na poziomie dokumentacyjnym najważniejsze jest to, że opisany system odpowiada stanowi faktycznie zaimplementowanemu w repozytorium. Dokumentacja nie opisuje rozwiązania docelowego w sensie koncepcyjnym, tylko wersję rzeczywiście wykonaną, uruchomioną i zweryfikowaną w zakresie przedstawionym w poprzednich rozdziałach.

20.1. Końcowe odniesienie do implementacji

Najważniejsze obszary kodu zamykające zakres realizacji projektu:

- `app/layers/ingestion`,
- `app/layers/staging_validation`,
- `app/layers/preprocessing`,
- `app/layers/validation`,
- `app/layers/integration_golden`,
- `app/layers/serving`,
- `airflow/dags/goldenizacja_pipeline.py`,
- `frontend/src/`,
- `docker-compose.yml`,
- `tests/`.